Explainable Financial Alerts for Retail Investors: Integrating Statistical Anomaly Detection and News–Market Impact Correlation

Henrique José da Silva Santos

Student No.: 1180934

A dissertation submitted in partial fulfillment of the requirements for the degree of Master of Science, Specialisation Area of

Supervisor: Luís Gomes
Co-Supervisor: Rafael Silva

Evaluation Committee:
President:
[A definir]

Members:
[A definir]

Porto, August 10, 2026

Statement Of Integrity

I hereby declare that I have conducted this academic work with integrity.

I have not plagiarised or applied any form of undue use of information or falsification of results along the process leading to its elaboration.

Therefore, the work presented in this document is original and was authored by me, having not been previously used for any other purpose.

I declare that I have fully acknowledged the Code of Ethical Conduct of P.PORTO.

Use of Artificial Intelligence. In accordance with the P.PORTO/ISEP guidelines, I declare that artificial intelligence tools were used during the development of this work to assist with the drafting and editing of the text and with software development. I directed the work and critically reviewed and validated all outputs; the ideas, decisions and final content are my own and are my responsibility.

ISEP, Porto, August 10, 2026

Dedicatory

Abstract

This dissertation presents InvestiGator, an explainable (XAI-first) financial-alert system for retail investors in the US equity market (NYSE and NASDAQ). The system raises Telegram alerts from two independent triggers (abrupt market movements, detected as statistical anomalies, and incoming financial news) and, for every alert, exposes a fully traceable explanation: the detected event, the reasoning applied, the sources, and historical precedents. Its core is a news–market correlation engine that, given a news item, retrieves analogous historical news from a knowledge base and measures the impact those precedents had on prices, event-study style and without lookahead. As an Artificial Intelligence Engineering contribution, the work integrates, applies and critically evaluates existing components (sentence-embedding retrieval, statistical anomaly detection, supervised triage and transparent explanation) in a functional, explainable and reproducible system. On real data, semantic retrieval clearly outperformed lexical and trivial baselines (cross-ticker precision@5 of 0.51 against 0.35 lexical and 0.24 random), and the volatility-normalised detector fired far more consistently across stocks than a fixed threshold. A materiality-triage model trained on 79,753 multi-year news–market examples nearly quadrupled within-budget alert precision on held-out data, yet no text model beat the volatility-only baseline, and in deployment the same gate showed no significant benefit on the differently distributed population it actually sees. Both findings are reported as they stand. Limitations and future work are stated explicitly.

Keywords: Explainable AI, Financial Alerts, Anomaly Detection, News Impact, Retail Investors, Financial NLP

Resumo

Esta dissertação apresenta o InvestiGator, um sistema de alertas financeiros explicável (XAI-first) para investidores de retalho no mercado acionista dos Estados Unidos (NYSE e NASDAQ). O sistema envia alertas via Telegram a partir de dois gatilhos independentes (movimentos abruptos de mercado, detetados como anomalias estatísticas, e novas notícias financeiras) e, para cada alerta, expõe uma explicação totalmente rastreável: o evento detetado, o raciocínio aplicado, as fontes e os precedentes históricos. O seu núcleo é um motor de correlação notícia–mercado que, dada uma notícia, recupera notícias históricas análogas de uma base de conhecimento e mede o impacto que esses precedentes tiveram nos preços, à maneira de um estudo de evento e sem lookahead. Enquanto contribuição de Engenharia de Inteligência Artificial, o trabalho integra, aplica e avalia criticamente componentes existentes (recuperação por embeddings de frases, deteção estatística de anomalias, triagem supervisionada e explicação transparente) num sistema funcional, explicável e reproduzível. Em dados reais, a recuperação semântica superou claramente as baselines lexical e triviais (precision@5 cross-ticker de 0,51, face a 0,35 de uma baseline lexical e 0,24 do acaso) e o detetor normalizado pela volatilidade disparou de forma muito mais consistente entre ações do que um limiar fixo. Um modelo de triagem de materialidade treinado em 79 753 exemplos notícia–mercado de vários anos quase quadruplicou a precisão dos alertas dentro do orçamento diário, mas nenhum modelo com texto bateu a baseline de só-volatilidade, resultado reportado tal como é. As limitações e o trabalho futuro ficam explícitos.

Palavras-chave: Explainable AI, Financial Alerts, Anomaly Detection, News Impact, Retail Investors, Financial NLP

Acknowledgement

Contents

List of Figures

List of Tables

List of Algorithms

Listings

List of Symbols

r_t	log-return of an asset on day t	—
μ_t	rolling mean of returns	—
σ_t	rolling standard deviation of returns	—
z_t	z-score of the return on day t	—
k	anomaly threshold (in standard deviations)	—
w	rolling-window length	d

List of Acronyms

AI	Artificial Intelligence.
NASDAQ	National Association of Securities Dealers Automated Quotations.
NYSE	New York Stock Exchange.
RQ	Research Question.

Chapter 1

Introduction

1.1 Contextualization

Most Americans now own stock, directly or through retirement accounts and mutual funds (Gallup 2025), and they invest in the world's largest equity market, worth US$62.2 trillion at the end of 2024 (Securities Industry and Financial Markets Association (SIFMA) 2025) (Figure 1.1). Trading happens mainly on the New York Stock Exchange (NYSE) and National Association of Securities Dealers Automated Quotations (NASDAQ), and a growing share of it now comes from ordinary people using commission-free mobile apps, not from professional institutions.

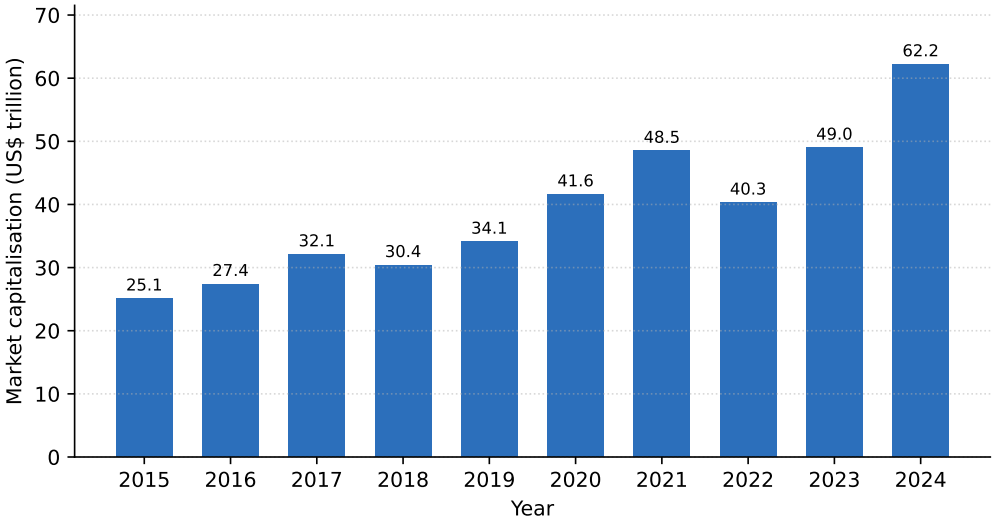

Figure 1.1: US equity market capitalisation, 2015–2024 (US$ trillion). Drawn from data reported in the SIFMA 2025 Capital Markets Fact Book (Securities Industry and Financial Markets Association (SIFMA) 2025).

These non-professional, or "retail", investors are the audience of this work, and their position is hard. Markets move continuously across thousands of securities, and relevant news arrives all through the trading day, yet retail investors have none of the tools that banks and funds take for granted. Ownership is uneven too: it reaches 87% of households earning above US$100,000 but only 28% below US$50,000 (Gallup 2025).

Meanwhile, artificial intelligence has spread quickly through finance: a 2026 survey found 81% of financial firms adopting Artificial Intelligence (AI) and 71% already using generative AI (Cambridge Centre for Alternative Finance 2026). But most of this technology is opaque,

rarely showing how it reaches a conclusion (Adadi and Berrada 2018; Barredo Arrieta et al. 2020). For someone who must act on an alert and live with the result, that opacity is the obstacle this dissertation removes: it lets the investor see *why* an alert was raised.

1.2 Problem Statement

When a holding moves, a retail investor asks the same three questions, in the same order, and they map onto three distinct technical problems:

1. **Is this actually unusual, or normal for this stock?** A 3% day is unremarkable for one company and a rare event for another, so the judgement is statistical and has to be made against each security's own recent behaviour.
2. **Is it this company, or the whole market?** A fall that simply tracks a falling index carries different information from a fall the company produced on its own, and separating the two requires estimating how strongly the security normally follows the market and its sector.
3. **Has something like this happened before, and what followed?** Answering this means finding past events that are genuinely comparable, which is a retrieval problem over meaning rather than keywords, and then measuring what prices did afterwards.

Free consumer tools answer none of the three. They report the percentage move, which is the starting point of the first question rather than its answer. Professional terminals do answer all three, at a subscription cost far beyond a non-professional investor. The gap this dissertation addresses is therefore not a lack of data, which is abundant and largely free, but a lack of *context* that an ordinary investor can obtain and, more importantly, can check.

Who this is for

Two user profiles frame the design, and they want opposite things from the same system.

The first is the **long-term holder**: someone with a small number of positions who looks at them infrequently and feels anxiety when the screen is red. For this person the most valuable output is often permission to do nothing, delivered as evidence that a move was market-wide and unremarkable. The failure mode to avoid is alarm without information.

The second is the **active retail investor**: someone who checks several times a day and already receives alerts that say only that a price moved. For this person the value is context arriving with the alert rather than after a search elsewhere. The failure mode to avoid is alert fatigue, where volume trains the reader to ignore the channel.

The two profiles pull in different directions on cadence, and the system resolves that tension by being deliberately selective and by stating its selection rules openly, a decision revisited in Chapter 4.

Position taken

This work takes a deliberately different stance from the tools that surround it: it does not predict prices, it *explains* events. For each abrupt move or relevant news item, the system shows what it detected, how, where the information came from, and which past events resemble the current one, together with what happened to prices afterwards (Figure 1.2).

That refusal is a design constraint rather than a limitation to apologise for. It rules out importing third-party forecasts, which would contradict the claim at the centre of the system, and it makes every output checkable, because a statement about what already happened can be verified against the record while a forecast cannot.

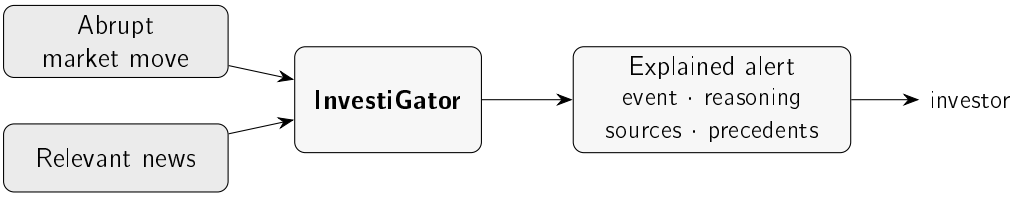

Figure 1.2: The two triggers, a sudden market move or a relevant news item, feed InvestiGator, which answers with an explained alert the investor can follow and check, not just a bare notification.

1.3 Research Questions and Objectives

The aim is to design, build and evaluate an explainable financial-alert system for US retail investors, driven by two independent triggers, a sudden market move and an incoming news item, and delivered over Telegram. Four research questions guide the work:

- **Research Question (RQ)1.** Can transparent statistical methods detect abrupt market movements consistently enough to be useful to a retail investor, while staying explainable?
- **RQ2.** Can a news–market correlation engine retrieve genuinely analogous historical news better than trivial baselines, and quantify the observed price impact of those precedents without lookahead?
- **RQ3.** Can the resulting explanations be made faithful to what the system actually computes, and useful to a retail investor?
- **RQ4.** Can a supervised model, trained on historical news and market context, usefully prioritise which news items deserve an alert, beyond simple volatility baselines, while never predicting price direction?

These questions map onto five concrete objectives:

- build an explainable alerting pipeline driven by the two triggers;
- evaluate the anomaly detector against transparent baselines;
- evaluate the retrieval of historical news against trivial baselines;
- assess whether the explanations are faithful and useful;
- train, calibrate and honestly evaluate a materiality-triage model that ranks news alerts.

The system is built incrementally, in its simplest defensible form first, a methodological choice explained in Chapter 3.

1.4 Scientific Contributions

This is a dissertation in Artificial Intelligence *Engineering*: the contribution is not a new algorithm but the disciplined assembly of existing ones into a system that works, explains itself, and can be reproduced. Four contributions stand out:

- A reproducible method for relating news to market impact, pairing sentence-embedding retrieval with event-study windows, with the similarity measure and the time horizons stated explicitly and no information about the future allowed to leak in.
- **InvestiGator**, an alerting system that shows its whole reasoning chain: the detected event, the supporting evidence, the sources, and the relevant historical precedents. Because each explanation is built directly from the quantities the system computes, the text cannot drift from the computation.
- A trained and calibrated *materiality-triage* model: supervised learning over historical news and market context that estimates how often news with a given profile was followed by an abnormal move, labelled by the system's own event-study code, evaluated against a volatility baseline, and integrated into the alerts as ranked, explained evidence rather than a forecast.
- A critical, honest evaluation of each core component on real data, against simple and lexical baselines, with the results, the ablations and the limitations all reported plainly.

1.5 Document Structure

The rest of the dissertation is written to be read in order, each chapter answering one question:

- **Chapter 2 (State of the Art):** what is already known, and what does this work choose?
- **Chapter 3 (Methods and Materials):** how is the system built, and how is it judged?
- **Chapter 4 (InvestiGator):** what is the system, and how do its data, parts, flow and decisions fit together?
- **Chapter 5 (Case Studies):** does it work on real data?
- **Chapter 6 (Conclusions):** what was learned, and what comes next?

Chapter 2

State of the Art

This chapter has one job: to justify every design choice in InvestiGator against the alternatives. Each section asks what a field offers and ends with what InvestiGator takes from it. The order is: how retail investors behave; anomaly detection; explainable AI; trust in automation; financial text representation; information retrieval; event studies; and case-based reasoning. The recurring finding is simple. The building blocks are mature, but an integrated, explainable system for the retail investor is still rare.

2.1 Retail Investing and Information Overload

How do retail investors behave, and what should an alert give them?

Behavioural finance shows that real investors depart systematically from the rational ideal (Barberis and Thaler 2003). One root cause is *loss aversion*: we feel losses more than equal gains (Kahneman and Tversky 1979), which makes investors especially sensitive to bad news, and a timely, well-explained alert valuable.

The strongest pattern is *attention*. Barber and Odean (2008) show that individuals are net buyers of attention-grabbing stocks (those in the news, with unusual volume, or extreme one-day returns): facing thousands of choices, they fall back on whatever caught their eye. Attention can even be measured through search volume (Da, Engelberg, and Gao 2011). This is decisive for the design, because a sharp price move and a salient headline are exactly InvestiGator's two triggers, so an alert that adds context lands where attention-driven decisions are made.

News also moves markets, and its text can be measured. Tetlock (2007) builds a daily pessimism index from a newspaper column and links it to price pressure and trading volume, early evidence that textual signals relate to market outcomes, which is the premise of InvestiGator's news engine. This audience is large and active too: even through the March 2020 crash, retail investors on one commission-free platform added to their holdings rather than panicking (Welch 2022), consistent with the broad ownership and market scale reported in Chapter 1 (Gallup 2025; Securities Industry and Financial Markets Association (SIFMA) 2025).

For InvestiGator: retail investors are attention-constrained and news-driven, so the system does not predict the market; it lowers the cost of *interpreting* the events that already grab attention, with a transparent explanation and real historical evidence.

2.2 Anomaly Detection in Financial Markets

How do you tell an abnormal price move from ordinary noise?

InvestiGator's first trigger is an anomaly detector, so it sits in the mature field of anomaly detection. The standard taxonomy (Chandola, Banerjee, and Kumar 2009) separates *point*, *contextual* and *collective* anomalies, and groups methods into statistical, distance/density, and machine-learning families (Figure 2.1). A sharp single-day return is a contextual point anomaly: abnormal relative to the asset's own recent behaviour.

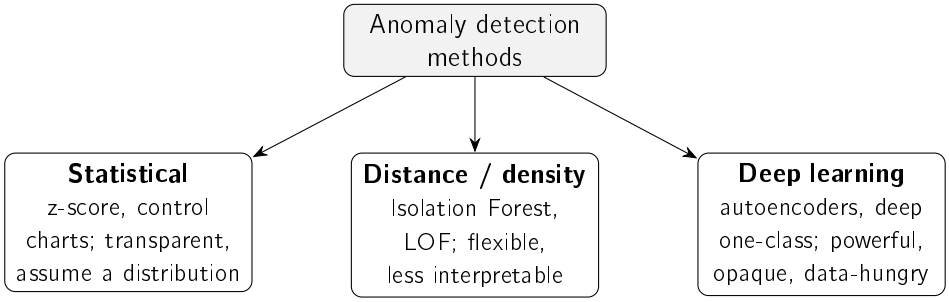

Figure 2.1: Families of anomaly-detection methods relevant to this work, following Chandola, Banerjee, and Kumar (2009) and Pang et al. (2021). Interpretability decreases, and modelling capacity increases, from left to right.

The simplest detector is a rolling z-score: flag a return that deviates far from its recent mean and standard deviation. It is attractive because it is transparent and easy to justify. Its one assumption, a locally stable distribution, is handled by the rolling window, which tracks *volatility clustering*, the well-known tendency of calm and turbulent periods to arrive in runs (Bollerslev 1986; Engle 1982). The rolling standard deviation is, in effect, a simpler non-parametric version of that idea: it adapts to the current regime without fitting a model the user could not scrutinise. Between the two sits the exponentially weighted moving average of squared returns (the RiskMetrics estimator), which weights the recent past more heavily than the distant past but still estimates no parameters per instrument. A full GARCH fit is declined here under defensible simplicity, but the claim is not left as opinion: the EWMA variant, the first step on the road to GARCH, is put through the same evaluation protocol in Chapter 5, so the trade-off is measured rather than asserted.

More expressive detectors trade interpretability for capacity. Isolation Forest isolates outliers by random partitioning and scales to high dimensions (F. T. Liu, Ting, and Zhou 2008); the Local Outlier Factor scores how isolated a point is within its neighbourhood (Breunig et al. 2000); deep detectors learn a model of normality (Pang et al. 2021). All are powerful but harder to read off, and their edge shows most in high-dimensional data, not a single return series. Rather than dismissing them on those grounds alone, Chapter 5 evaluates both Isolation Forest and the Local Outlier Factor under the same causal protocol as the statistical rule, so the comparison is empirical. In finance, detection is anyway dominated by *unsupervised* methods, because labelled anomalies are scarce (Ahmed, Mahmood, and Islam 2016), a constraint that also shapes how InvestiGator is evaluated (Chapter 5). Table 2.1 compares the families.

For InvestiGator: the signal is one low-dimensional return series that must be explained to a non-expert, so the transparent z-score wins; the more expressive families are noted as alternatives whose opacity the problem does not justify.

Table 2.1: Anomaly-detection approaches relevant to this work.

Approach	How it works	Strengths / limitations
Statistical z-score (rolling) (Chandola, Banerjee, and Kumar 2009)	Flags returns that deviate from a rolling mean and standard deviation	Transparent and easy to explain; assumes a locally stable distribution
Isolation Forest (F. T. Liu, Ting, and Zhou 2008)	Isolates points via random partitioning (ensemble)	Scalable, handles high dimensionality; score is not directly interpretable
Local Outlier Factor (Breunig et al. 2000)	Scores isolation within a local density neighbourhood	Captures local structure; sensitive to neighbourhood size, score hard to read
Deep detectors (Pang et al. 2021)	Learn a model of normality (e.g. autoencoders)	High capacity for complex data; opaque, data-hungry, hard to justify

2.3 Explainable Artificial Intelligence

What kind of explanation should the system give?

Explainable AI (XAI) exists because the strongest models rarely show their reasoning. Surveys map the field and its central split (Adadi and Berrada 2018; Barredo Arrieta et al. 2020): *transparent* models, interpretable in themselves, versus *post-hoc* methods that explain a trained black box, which can be organised by what they explain (Guidotti et al. 2018) (Figure 2.2). A useful caution: "interpretability" is not one thing (Lipton 2018), so a system should say which kind it offers. InvestiGator offers transparency of its decision logic and traceable evidence, not a claim of full interpretability.

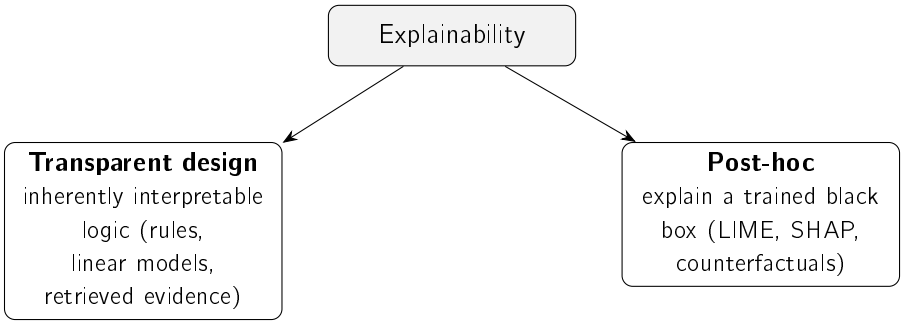

Figure 2.2: The two broad routes to explainability (Barredo Arrieta et al. 2020; Guidotti et al. 2018): building models that are transparent by design, or explaining a black box after the fact. This work follows the former.

Two post-hoc techniques are widely used: LIME fits a simple local surrogate around one prediction (Ribeiro, Singh, and Guestrin 2016), and SHAP attributes a prediction to its features using Shapley values (Lundberg and S.-I. Lee 2017). Both are valuable when one is committed to a black box. But for high-stakes decisions, Rudin (2019) argues for dropping that commitment and using models that are interpretable in the first place, since a post-hoc explanation is itself an approximation that can mislead. InvestiGator follows this: it explains

primarily by transparent design (every quantity exposed, real precedents shown as evidence), with feature attribution as an optional extra.

Good explanations are also a human matter. People want *contrastive*, selective reasons, not an exhaustive dump (Miller 2019), which is why each alert shows a few relevant precedents and the exact quantities behind the decision. Explanations must also be evaluated, not assumed: Doshi-Velez and Kim (2017) argue that interpretability claims need rigorous evaluation rather than assertion, and set out how such evaluation can be structured. The criterion this work adopts is *fidelity*, how well an explanation reflects the real computation, because it is the property InvestiGator can guarantee (Chapter 5). Table 2.2 summarises the options.

Table 2.2: Explainability approaches considered.

Method	**Type**	**Strengths / limitations**
LIME (Ribeiro, Singh, and Guestrin 2016)	Local surrogate, post-hoc	Intuitive, model-agnostic; explanations can be unstable, only locally faithful
SHAP (Lundberg and S.-I. Lee 2017)	Shapley-value attribution, post-hoc	Consistent, theoretically grounded; computationally costly
Transparent design (Barredo Arrieta et al. 2020; Rudin 2019)	Inherently interpretable logic	No approximation, faithful by construction; constrains model choice

For InvestiGator: pursue explainability by transparent design and judge it by fidelity; treat post-hoc attribution as a complement, not the foundation.

2.4 Trust and Appropriate Reliance in Decision Support

When should the investor actually trust an alert?

An explanation only helps if it produces *appropriate* reliance: not dismissing a sound warning, not acting blindly on a fallible one. The human-factors view (J. D. Lee and See 2004) calls the goal *calibrated* trust, reliance matched to the system's real competence, and names two failure modes, *disuse* (ignoring a good aid) and *misuse* (over-trusting a flawed one). With real money, both are costly.

Explanations are how trust gets calibrated, but they are not magic. Bansal et al. (2021) show that explanations can even cause *over-reliance*, with people accepting confident explanations of wrong answers. InvestiGator's response is concrete: explain with *verifiable evidence* (the actual z-score and its parameters; real precedents with their measured impact), and disclaim prediction on every alert.

For InvestiGator: transparency is not decoration but the mechanism for appropriate reliance, so the system shows checkable evidence rather than a persuasive story.

2.5 Financial NLP and Text Representation

How should the system represent the meaning of a headline?

To compare headlines, text must become numbers. The financial-NLP literature falls into three generations: lexicons, word embeddings, and contextual neural models. Surveys of textual analysis in finance up to the mid-2010s (Kearney and S. Liu 2014) cover the first two; the third arrived afterwards, and is where this work takes its representation.

Lexicons came first. Finance-specific word lists fix the fact that general sentiment dictionaries misread ordinary financial words such as "liability" or "tax" (Loughran and McDonald 2011); they are fully transparent but bounded by their vocabulary. Word embeddings then placed words in a vector space where proximity means similar use: word2vec from local context (Mikolov et al. 2013), GloVe from global co-occurrence (Pennington, Socher, and Manning 2014). Both give each word a single vector, blind to context.

The Transformer (Vaswani et al. 2017) removed that limit, and BERT (Devlin et al. 2019) produced deeply *contextual* representations. Comparing two texts with BERT directly is costly, though; Sentence-BERT (Reimers and Gurevych 2019) fixes this by giving each sentence one fixed-length vector comparable by a cheap cosine, exactly what precedent retrieval needs. At very large scale, exact search can be swapped for an approximate index (Johnson, Douze, and Jégou 2021) without changing the representation (future work). Two heavier options also exist: domain-tuned FinBERT models (Araci 2019; Y. Yang, Uy, and Huang 2020) and large financial LLMs such as BloombergGPT (Wu et al. 2023), both powerful but costlier, less transparent, and outside this work's free, reproducible constraint. Table 2.3 compares them.

Table 2.3: Text-representation approaches for financial news.

Approach	**Representation**	**Strengths / limitations**
Finance lexicons (Loughran and McDonald 2011)	Curated word lists / tone	Fully transparent; bounded by vocabulary
word2vec / GloVe (Mikolov et al. 2013; Pennington, Socher, and Manning 2014)	Static word vectors	Efficient; one vector per word, no context
BERT (Devlin et al. 2019; Vaswani et al. 2017)	Contextual token embeddings	Strong understanding; pairwise similarity is costly
Sentence-BERT (Reimers and Gurevych 2019)	Sentence embeddings (siamese)	Efficient cosine similarity; the choice in this work
FinBERT (Araci 2019; Y. Yang, Uy, and Huang 2020)	Domain-adapted language model	Finance-tuned; oriented to sentiment / communications
Financial LLMs (Wu et al. 2023)	Large generative model	Very capable; costly, opaque, outside free-tier scope

For InvestiGator: Sentence-BERT is the sweet spot (Figure 2.3), contextual meaning beyond lexicons and static vectors, yet cheap, comparable and free to run. One honesty note on the comparison: the evaluation in Chapter 5 measures Sentence-BERT against a *lexical* baseline, which brackets the question that matters (does meaning beat surface words?);

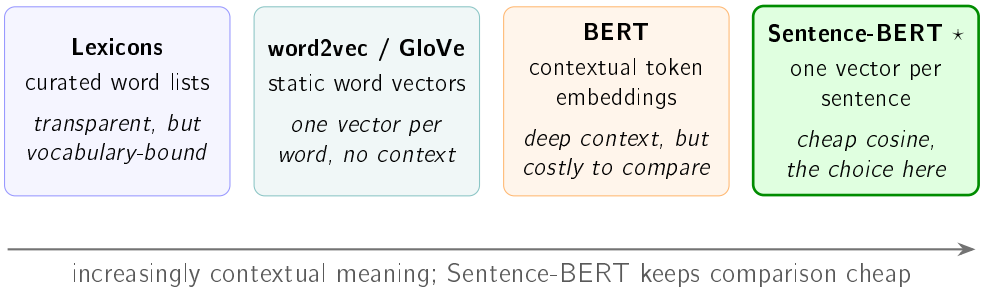

Figure 2.3: Three generations of turning financial text into numbers. Finance lexicons are transparent but bounded by their vocabulary; static word vectors add similarity of use but no context; contextual models (BERT) capture meaning yet are costly to compare pairwise; Sentence-BERT gives each headline a single vector comparable by a cheap cosine, exactly what precedent retrieval needs, and is the representation chosen here. Details in Table 2.3.

the embedder choice itself was also benchmarked on the same protocol. A domain-tuned FinBERT, mean-pooled into sentence embeddings, reaches only a precision@5 of 0.420, below MiniLM's 0.514 and consistent with its sentiment orientation, while current general encoders (E5-small, BGE-small) tie MiniLM (around 0.51) rather than beating it. The small, free, general encoder is therefore the sweet spot by measurement, not merely by argument; only static word vectors, strictly dominated for sentence comparison, are left to argument.

2.6 Information Retrieval and Ranking Evaluation

Once headlines are vectors, how do you rank past news, and how do you tell if the ranking is good?

Ranking a corpus by relevance to a query is the defining problem of information retrieval (IR). The vector-space model (Salton, Wong, and C. S. Yang 1975) represents documents and queries as vectors ranked by cosine similarity; InvestiGator's embedding retrieval is the same idea with richer vectors. Classical *lexical* ranking weights informative terms, from TF-IDF to BM25 (Robertson and Zaragoza 2009), still a strong baseline. Its known weakness is *vocabulary mismatch*: a relevant document that uses different words scores as dissimilar. Semantic embeddings overcome exactly that, which is why InvestiGator compares against a lexical baseline (Chapter 5).

Because retrieval returns a ranked list, it is judged with rank-aware measures (Manning, Raghavan, and Schütze 2008); the most direct is *precision@k*, the fraction of the top k results that are relevant. InvestiGator adopts it because an alert can show only a handful of precedents, so what matters is whether the top few are relevant, reported against random, recency and lexical baselines.

For InvestiGator: retrieval reuses a mature representation-then-rank pipeline and the standard precision@k metric; the contribution is the application and the pairing with measured impact, not a new algorithm.

2.7 Event Studies and News–Market Impact

Having found analogous news, how do you measure what it did to the price, honestly?

The tool is the *event study*, introduced by Fama et al. (1969) and standardised for daily returns by Brown and Warner (1985), who set the practice of measuring returns over short post-event windows, the basis for InvestiGator's +1, +3 and +5-day horizons. The impact is measured strictly *after* the event, so it describes an outcome, not a forecast.

Why not just predict the price? Because public news is absorbed into prices almost at once, the efficient-market hypothesis (Fama 1970), so forecasting returns from public news is very hard by construction. InvestiGator therefore does not compete with market efficiency; it measures and explains the reactions that *already* followed comparable news, which is both more honest and more defensible for a non-expert.

Relating news *text* to impact is an active area: Tetlock (2007) is an early example, and a more recent strand tries to *predict* moves from news (Ding et al. 2015; Xing, Cambria, and Welsch 2018). InvestiGator deliberately stops short of prediction. Doing this at scale needs news aligned to prices, which is the contribution of the FNSPID dataset (Dong, Fan, and Peng 2024), used to build the knowledge base; its coverage and period limits are noted in the data card of Chapter 3.

For InvestiGator: pairing semantic retrieval with event-study impact over FNSPID is the methodological core, evidence about the past, never a forecast.

2.8 Case-Based Reasoning and Precedent Retrieval

What is the engine, in one familiar idea?

It is *case-based reasoning* (CBR). In the classic account (Aamodt and Plaza 1994), a CBR system *retrieves* similar past cases, *reuses* their outcomes, and optionally *revises* and *retains* them. InvestiGator is the retrieve-and-reuse core: each past headline plus its measured impact is a *case*; a new headline is the query; cosine similarity retrieves; the precedents' impact is reused as evidence. It stops before *revise*, so it never turns evidence into a prediction.

For InvestiGator: naming the engine as CBR shows it is a recognised paradigm, not ad hoc, and its "this resembles these past cases" explanations are naturally intelligible to a non-expert.

2.9 Learned Alert Triage: Supervised Materiality Classification

When dozens of headlines arrive in a day, which ones deserve an alert?

Retrieval answers "what does this resemble?", and the event study answers "what followed similar news?". Neither ranks the incoming stream. That is a *triage* problem, and it can be posed as supervised classification without breaking the no-forecast rule. The model estimates the probability that an *abnormal move of unusual size, in either direction*, follows a news item; the label is produced by the system's own event-study measurement, the market-adjusted return of Brown and Warner (1985) against an index proxy. The target is *materiality*, whether the market reacted at all. Direction and magnitude of the price path are never predicted, so the task stays on the defensible side of the efficient-market line drawn earlier in this chapter.

Two model families cover the interpretability–capacity trade-off. Logistic regression is the transparent choice: with standardised inputs, its log-odds are an exact additive sum of per-feature contributions. That makes it the kind of inherently interpretable model Rudin (2019) argues should be preferred when explanations matter. Gradient-boosted decision ensembles (Friedman 2001) are the stronger nonlinear learner and serve as the ceiling the interpretable model is compared against. For an end user, the score is only honest if it is a *probability*: raw classifier scores are typically miscalibrated, and post-hoc calibration, such as fitting a sigmoid on held-out validation data (Platt scaling), corrects this (Niculescu-Mizil and Caruana 2005). Under class imbalance, evaluation uses the area under the precision–recall curve rather than accuracy. A budget-shaped precision completes the picture (how many of the N alerts a user can absorb per day were material), because that budget is exactly the alert-fatigue cost the triage exists to reduce.

For InvestiGator: triage adds a *trained* severity score on top of the transparent pipeline, ranked and gated per day, explained by its additive contributions, calibrated on validation data, and always compared against a volatility-only baseline before it is allowed to matter.

2.10 Uncertainty and Drift in a Deployed Model

A calibrated score is still not a guarantee, and a model does not stay still. What does the literature offer on both counts?

Calibration, as used for the triage score above, is an *aggregate* property: among items scored near a given probability, roughly that fraction should turn out positive. It says nothing about any individual item, and it offers no recourse once the model is applied outside the conditions in which it was fitted. *Conformal prediction* narrows this gap. It returns a set for each item rather than a point estimate, although its guarantee remains *marginal*, averaged over cases, rather than conditional on any one of them. The foundational treatment (Vovk, Gammerman, and Shafer 2005) shows that a predictor can be wrapped so that it returns a *set* of labels carrying a guarantee that is distribution-free and valid in finite samples: for a chosen error rate α, the set contains the true label in at least $1 - \alpha$ of cases. The guarantee assumes neither normality nor a correctly specified model, and requires only that calibration and test data be exchangeable. The split (or inductive) variant, which calibrates on a held-out block and so costs one extra pass rather than a retraining loop, is set out with its finite-sample quantile correction by Angelopoulos and Bates (2023).

The appeal for a system built on refusing to overclaim is direct. In a binary decision the returned set may contain *both* labels, which is an explicit and quantified "I do not know" rather than a probability near one half that still forces a choice. This is a different kind of honesty from calibration, and the two are complementary rather than alternatives.

The exchangeability assumption then raises the second question. Models are trained on a past and deployed into a future, and the relationship between inputs and target can change underneath them. The standard treatment of this in machine learning is the concept-drift literature, whose survey (Gama et al. 2014) sets out the taxonomy (sudden, gradual, incremental and recurring change), the detection strategies and the adaptation methods. It is a general treatment rather than a financial one, so it supplies the framing and the vocabulary while any statement about how much a particular corpus has drifted must come from measuring that corpus.

Both concerns belong to a wider point about systems rather than models. D. Sculley et al. (2015) argue, from experience with production systems, that machine-learning components accumulate a distinctive and largely hidden maintenance cost: entanglement, where changing anything changes everything; unstable and undeclared data dependencies; and configuration debt. Their account is a position paper drawn from practice rather than an empirical study, and it is cited here for that qualitative claim alone. Its relevance is that a dissertation prototype which runs continuously, as this one does, inherits the same category of problem in miniature.

For InvestiGator: these three strands convert two of the system's standing caveats into measurable quantities. The claim that a probability is not a promise becomes a coverage experiment (Section 5.7); the claim that the training corpus predates deployment becomes a drift measurement with conventional interpretation bands (Section 5.8). Neither changes what the deployed system does, and in both cases that restraint is itself the reported result.

2.11 Existing Tools for the Retail Investor

What can a retail investor already use, and what is missing?

Three kinds of tool are common today, and each leaves a gap. *Price alerts*, in every brokerage app, fire when a price or percentage threshold is crossed; they ride the same attention behaviour InvestiGator targets (Barber and Odean 2008), but a fixed threshold ignores each stock's own volatility (firing constantly on volatile names, rarely on calm ones) and says only *that* a level was crossed, never *why*. *News and sentiment apps* aggregate headlines and attach a sentiment score, building on the link between tone and markets (Tetlock 2007) and on finance lexicons (Loughran and McDonald 2011); but a single polarity is thin, often proprietary, and rarely connects a headline to concrete historical precedents and their outcomes. *Robo-advisors*, the most studied retail-fintech tool, genuinely help by improving diversification and curbing biases (Cardillo and Chiappini 2024; D'Acunto, Prabhala, and Rossi 2019), but they *act for* the investor with proprietary logic, the opposite of an explanation-first tool that leaves the decision to the user. Table 2.4 positions all three against InvestiGator.

Table 2.4: Existing retail tools versus the system proposed in this work.

Tool	What it does	Explains why?	Shows precedents?	Acts for user?
Brokerage price alert	Fires on a fixed price / % threshold	No	No	No
News / sentiment app (Tetlock 2007)	Aggregates news, scores sentiment	Partly (opaque score)	No	No
Robo-advisor (D'Acunto, Prabhala, and Rossi 2019)	Allocates and rebalances a portfolio	Rarely (proprietary)	No	Yes
This work (InvestiGator)	Explains events, shows evidence	Yes, by design	Yes (measured impact)	No

A fourth option has become common since large language models reached consumer products: asking a general assistant why a stock moved. These models are demonstrably capable on financial text, and domain-specific variants exist (Wu et al. 2023). The difficulty is not fluency but grounding. A general assistant has no access to the security's own recent volatility, so it cannot say whether a move was unusual *for that stock*; it does not estimate the security's sensitivity to its market and sector, so it cannot separate company from market; and its account of comparable past events is recalled rather than retrieved from an indexed corpus with measured outcomes. The answer reads well and cannot be checked, which is precisely the interpretability failure the explainability literature warns about (Lipton 2018; Rudin 2019).

Since 2025 that fourth option has stopped being a workaround and become a shipped feature, which makes it the closest competitor to this work's central claim and worth naming rather than describing as a category. Robinhood introduced *Cortex* in March 2025 with a feature whose stated purpose is to "answer the age-old question of, 'Why is this stock going up or down today?'" by generating summaries from news, analyst reports, technical signals and the broker's own platform data (Robinhood Markets 2025). Google rebuilt Google Finance around AI and shipped "key moments" that "explain why a stock moved", annotated onto the price chart (Google 2026). Both were observed on 2026-08-07; the wording quoted is each vendor's own.

The overlap with this work is real and should be stated plainly: these products answer the same question for a far larger audience than a dissertation ever will, and they answer it in plain language, which was one of the goals here. The difference is what stands behind the answer, and it is a difference of *kind*. A generated summary is a claim; this work's output is a claim with its evidence attached. Neither product states whether the move was unusual *for that security* against its own recent volatility, neither separates the company's contribution from its market and sector, and neither retrieves comparable past events with outcomes measured after the fact. Nor is either verifiable by the reader: Robinhood's own disclosure is that "there is no guarantee that AI will improve investing performance, mitigate risk, or reduce losses" (Robinhood Markets 2025), which is a statement about outcomes and not about whether any individual explanation is right. That is the gap this work occupies, and it is a narrow one honestly described: not a better answer, but an answer a reader can check.

Table 2.5 therefore scores the same tools against the three questions posed in Section 1.2, which is a stricter test than a feature list because each question has a checkable answer.

For InvestiGator: each existing free tool covers one corner, alerting, news, or allocation, and none combines volatility-aware detection, market-versus-company attribution and concrete historical evidence in a single tool that informs rather than decides. Professional terminals do combine them, at a price the target user does not pay. That intersection, all three answers at no cost and with the reasoning exposed, is the niche this work fills.

2.12 Comparative Analysis and Positioning

So what does InvestiGator choose, and why?

Table 2.6 gathers the choices against their alternatives. The rule throughout is *defensible simplicity* (Rudin 2019): when two options perform comparably, prefer the more transparent one, because the system must be understood and acted upon by a non-expert.

Table 2.5: Existing tools scored against the three questions a retail investor asks (Section 1.2).

Tool	**Q1 Unusual for this stock?**	**Q2 Company or market?**	**Q3 Happened before, with outcomes?**
Brokerage price alert	No (fixed threshold)	No	No
News / sentiment app	No	No	No (polarity only)
Robo-advisor (D'Acunto, Prabhala, and Rossi 2019)	Not surfaced	Not surfaced	No
General LLM assistant (Wu et al. 2023)	No (no volatility baseline)	No (no beta estimate)	Recalled, not retrieved
AI "why did it move?" feature (Robinhood Cortex (Robinhood Markets 2025); Google Finance key moments (Google 2026))	Not stated	Not stated	Generated, not retrieved
Professional terminal	Yes	Yes	Yes (paid)
This work (InvestiGator)	Yes (rolling z-score)	Yes (rolling beta)	Yes (retrieved, measured)

2.13 Chapter Conclusions

None of these building blocks is new. What is scarce, and what this work builds, is their disciplined integration into a single explainable system for the retail investor: a transparent detector, semantic retrieval paired with event-study impact, and explanations that are faithful by construction. The next chapter turns these choices into a method.

Table 2.6: Component choices in this work versus alternatives.

Component	**Chosen**	**Alternative**	**Why**
Anomaly detection	z-score (rolling)	Isolation Forest, deep detectors	Transparency; low-dimensional signal
News retrieval	Sentence-BERT + cosine	Lexicons, word2vec, LLMs	Contextual yet cheap and reproducible
Impact measurement	Event-study windows	—	Standard, reproducible, no forecasting
Explanation	Transparent design + precedents	LIME / SHAP only	Faithful by construction

Chapter 3

Methods and Materials

This chapter answers two questions: how is the system built, and how is it judged? It covers the approach, the data (in a reproducible data card), the techniques behind each component, the responsible-AI commitments, and the evaluation design, which was fixed before any experiment was run. One idea runs through all of it: keep things as simple as they can defensibly be, so that when two options perform about the same, the more transparent one wins. InvestiGator itself is described in Chapter 4; the experiments are reported in Chapter 5.

3.1 Approach and Methodology

How was the system built? Incrementally, end to end. Rather than finishing each component in isolation, a thin slice was built and proven first, from data acquisition to a delivered alert, so that integration risk surfaced early. Each component then started in its simplest defensible form and grew only where the extra complexity earned its place. This is what Artificial Intelligence *engineering* means here: the building blocks already exist in the literature (Chapter 2), so the contribution is their disciplined integration, application and critical evaluation, not a new algorithm.

Two habits follow. First, every component is defined by its *responsibility* and its *interface*, not by a particular implementation, which keeps it testable and swappable (one embedding model for another, say). Second, the project's non-negotiable constraints, only free and reproducible resources, the US market, transparency, and no prediction or trading, filter the design space before any performance question.

A third habit was adopted after it became necessary, and it deserves to be stated rather than implied, because it is a methodological commitment and not a matter of taste. The user-facing layer was rebuilt several times and rejected each time on aesthetic grounds, which is a criterion with no stopping condition: there is always another arrangement to try, and no version can ever be shown to be adequate. The remedy was to write the acceptance criteria *before* the code, in a versioned document, with each criterion expressed so that a test or a measurement decides it rather than an opinion. The discipline has a cost, and the cost is the evidence that it works: one criterion turned out to be wrong once the interface existed, and it was corrected in writing, with the reason recorded, instead of being quietly ignored. A criterion silently bypassed is indistinguishable from a criterion that was never written, so amending it in the open is the only version of the practice that means anything. The same logic governs the human evaluation instrument, whose analysis threshold is fixed before any response is collected, so that a threshold moved afterwards would appear in the version history rather than in the result.

3.2 Datasets and Data Sources

What data does the system use? Two clearly separated layers that share one schema, so they are directly comparable (Figure 3.1): a **historical** layer, built offline, that supplies the precedents, and a **live** layer that supplies real-time prices and news. The split is deliberate: the historical layer must be rich and stable, the live layer timely and cheap.

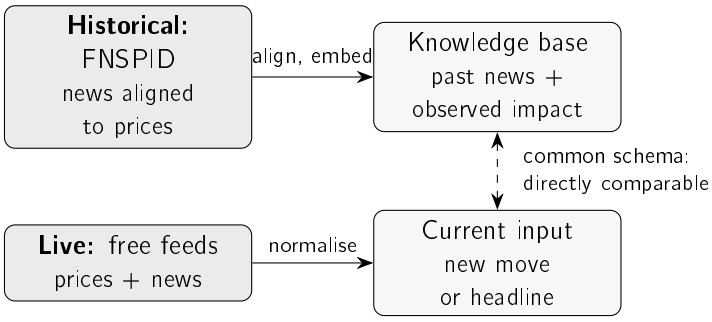

Figure 3.1: The two data layers. The historical layer turns the FNSPID corpus into a knowledge base of past news and its observed price impact; the live layer normalises real-time prices and news into the same fields, so a current event can be compared directly against the precedents.

3.2.1 Historical Layer: the FNSPID Corpus

The intended primary source for the historical layer is the FNSPID dataset (Dong, Fan, and Peng 2024), which provides financial news time-aligned with daily prices for thousands of US companies. That alignment is exactly what is needed to relate news to subsequent price impact without custom scraping. The dataset's news component is large (on the order of tens of gigabytes), which directly motivates the streaming, filtered ingestion described below. Table 3.1 records the data card: the choices that make the historical subset reproducible.

The fifteen large-cap tickers span five sectors, which makes the cross-sector evaluation of Chapter 5 meaningful while keeping the corpus tractable; the 2018–2023 window (the dataset's extent) is recent yet long enough to hold precedents. These are documented choices, not fixed ones: a different ticker set or window can be substituted without changing the method.

3.2.2 Preprocessing and Event Alignment

How is a news item turned into a historical case? In three steps. First, the news file is read in a stream and filtered to the chosen tickers and window, so only the relevant subset is materialised. Second, each item is normalised to the common schema (date, ticker, headline). Third, each item is aligned to the market and its impact measured. The alignment rule matters for honesty: the event day is the first trading day *on or after* the news date, and impact is measured from that day's *close* onward. Measuring from the close, not the open, avoids crediting a move already in the opening price, and measuring strictly forward enforces the no-lookahead guarantee of Section 3.6. Items with no aligned prices, or whose window runs past the data, are dropped rather than imputed. Algorithm 3.1 states the build.

To make the abstract concrete, Figure 3.2 follows *one real headline* through every stage of the system: the raw record as retrieved, the cleaning and no-lookahead alignment, the

Table 3.1: Data card for the *designed* historical layer (FNSPID).

Item	**Value**
Source	FNSPID (Dong, Fan, and Peng 2024) (financial news time-aligned with prices)
Licence	Creative Commons BY-SA 4.0 (attribution required)
Fields used	date, ticker, headline (mapped to a common schema)
Tickers (15)	AAPL, MSFT, AMZN, GOOGL, NVDA, TSLA, META, JPM, BAC, XOM, CVX, JNJ, PFE, WMT, KO
Sectors (5)	Technology, banking, energy, health, consumer staples
Time window	2018–2023 (daily granularity)
Prices	Daily closing prices for the same tickers
Governance	Large data not retained (regenerated on demand); only small samples kept

Note: this card documents the FNSPID-based historical layer. The multi-year subset *was* built by the streaming pipeline and drives the materiality-triage study of Chapter 5 (79,753 examples, 2018–2023; fourteen of the fifteen tickers, as the corpus indexes Meta under its former symbol). The retrieval study is run on the real recent-news evaluation corpus described in Section 3.2.3 (3,714 headlines for the same tickers, built through the identical process); the retrieval was later re-run at scale on the multi-year corpus (Chapter 6), leaving the market-adjusted impact-magnitude study as future work.

representation step where the "AI" actually happens (the headline becomes a 384-dimensional embedding, the event becomes a small vector of market-context features, and the outcome becomes a binary materiality label), and the two things those representations are finally used and measured for. Every value shown is genuine, taken from the dataset for Nvidia on 10 May 2018.

Figure 3.2 is deliberately schematic. Figure 3.3 shows the very same journey as the *actual objects*, verbatim, so there is no doubt about what the system stores and reads at each step.

3.2.3 Live Layer and Evaluation Dataset

The live layer provides real-time prices from a free market-data service (with a free fallback) and news from a free company-news service and RSS feeds. Free news is deliberately *not* used to build the historical layer, because free tiers cover only a short, recent, delayed window; but it is fine as the live trigger and as a source of *real* news for a first evaluation. For that, a set of 3,714 real headlines for the fifteen tickers was collected from the free company-news service (the most recent months available), and is used for the retrieval study in Chapter 5. This corpus is real but recent, which is why the multi-year FNSPID build remains the richer source; that build was carried out, and it supplies the triage, taxonomy, conformal, drift and convergence studies of Chapter 5.

3.2.4 Data Governance and Attribution

Two governance rules apply to both layers. First, large data and models are never versioned: they are recreated by the ingestion process, and only small illustrative samples are kept under version control. Second, third-party news text is not republished: the dataset is cited and

Algorithm 3.1 Knowledge-base construction with event alignment.

Require: news items $\{(d_i, c_i, h_i)\}$; daily close prices P_c per ticker; horizons $H = \{1, 3, 5\}$
Ensure: knowledge base $\mathcal{K}$ of cases $(d, c, h, \mathbf{e}, \text{impact})$

$\mathcal{K} \leftarrow \emptyset$
for each news item (d_i, c_i, h_i) **do**
  $e_i \leftarrow$ first trading day $\geq d_i$ in P_{c_i} ▷ event alignment
  if e_i exists **then**
    $\mathbf{e}_i \leftarrow \text{embed}(h_i)$ ▷ Sentence-BERT
    for $\tau \in H$ **do**
      $\text{impact}_i[\tau] \leftarrow$ return from close(e_i) to close$(e_i{+}\tau)$, or n/a if unavailable
    end for
    $\mathcal{K} \leftarrow \mathcal{K} \cup \{(d_i, c_i, h_i, \mathbf{e}_i, \text{impact}_i)\}$
  end if
end for
return $\mathcal{K}$

attributed as required by its Creative Commons BY-SA 4.0 licence (Dong, Fan, and Peng 2024), and only minimal headline excerpts appear in this document.

3.3 Models and Techniques

Three techniques do the work: a detector, a retrieval-and-impact engine, and an explanation step. Each is introduced by what it is *for*, then how it works.

3.3.1 Statistical Anomaly Detection

What it is for: to flag a daily move that is unusual *for that stock*, not just large in absolute terms. *How it works:* let r_t be the day's (log-)return and μ_t, σ_t the mean and standard deviation of returns over a trailing window of w days ending strictly before t. The day is flagged when the absolute z-score exceeds a threshold k:

$$z_t = \frac{r_t - \mu_t}{\sigma_t}, \qquad \text{anomaly} \iff |z_t| > k. \tag{3.1}$$

This is the statistical family of detectors (Chandola, Banerjee, and Kumar 2009), chosen for transparency: the investor can be shown exactly how many standard deviations the move was. More expressive detectors exist, Isolation Forest (F. T. Liu, Ting, and Zhou 2008) or deep models (Pang et al. 2021), but their opacity is not warranted for a single return series explained to a non-expert (Chapter 2). The window w and threshold k are explicit choices, examined by ablation in Chapter 5. Algorithm 3.2 states the rule; the window for day t uses only earlier days, so no future information enters.

Because the no-lookahead property is asserted repeatedly in this dissertation, it is worth showing where it is actually enforced rather than only describing it. Listing 3.1 is the detector as deployed. The guarantee is one slice: the window ends at -1, so the day being judged is excluded from the mean and standard deviation it is judged against. If that slice included the last element, every alert in this work would be circular, and no downstream evaluation would detect it.

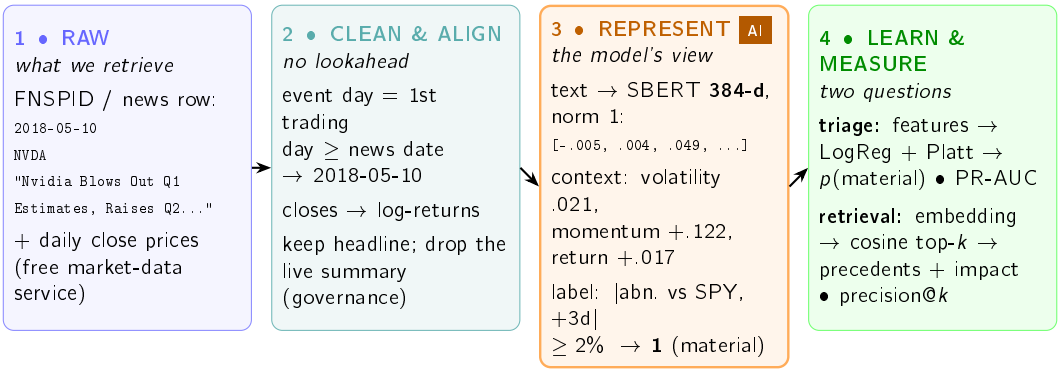

Figure 3.2: One real headline (Nvidia, 10 May 2018) travelling through every stage. It is retrieved as a plain row plus prices (1), cleaned and aligned to the first trading day with impact measured strictly forward (2), then *represented* for the models (3): the text becomes a 384-dimensional Sentence-BERT embedding, the market context becomes a handful of features, and the observed outcome becomes a binary materiality label. Those representations feed the two things the system measures (4): a calibrated triage probability and precedent retrieval. Every value is genuine, read from the dataset.

Algorithm 3.2 Rolling z-score anomaly detection (no lookahead).

Require: return series $r_1, \ldots, r_T$; window w; threshold k
Ensure: flag a_t for each day, with the quantities behind it

1: **for** $t \leftarrow w + 1$ **to** T **do**
2: $\quad W \leftarrow (r_{t-w}, \ldots, r_{t-1})$ ▷ strictly earlier days
3: $\quad \mu_t \leftarrow \text{mean}(W); \quad \sigma_t \leftarrow \text{std}(W)$
4: $\quad z_t \leftarrow (r_t - \mu_t) \,/\, \sigma_t$
5: $\quad a_t \leftarrow (|z_t| > k)$
6: **end for**
7: **return** for each flagged t, the tuple $(z_t, \mu_t, \sigma_t, w, k)$ ▷ exposed to the explanation engine

```python
r = pd.Series(list(returns), dtype="float64").dropna().reset_index(drop=True)
if len(r) < window + 1:
    raise ValueError(f"Need at least {window + 1} returns (got {len(r)}).")

recent = r.iloc[-window - 1 : -1]    # window BEFORE the last day (no lookahead)
last   = float(r.iloc[-1])
mu     = float(recent.mean())
sigma  = float(recent.std(ddof=1))
z      = (last - mu) / sigma if sigma > 0 else 0.0
return AnomalyResult(is_anomaly=abs(z) > threshold, z_score=z,
                     last_return=last, mean=mu, std=sigma, ...)
```

Listing 3.1: The rolling z-score detector. The window that defines "normal" stops one day before the day being judged.

Two details of that listing carry weight beyond the slice. The function returns the quantities behind the decision (z, μ, σ) and not only the verdict, which is what allows the explanation

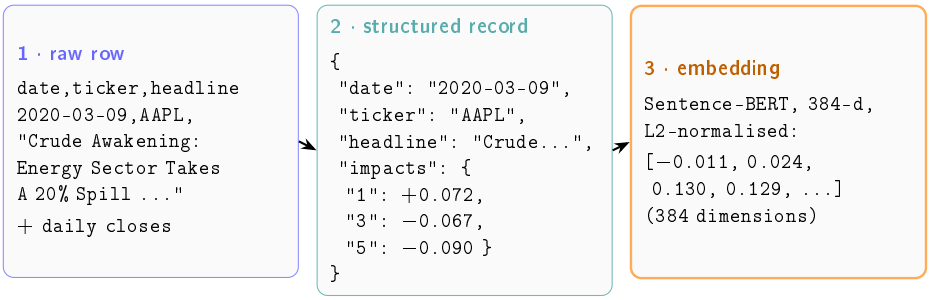

Figure 3.3: The same real record at three stages, shown verbatim. A plain news row (1) becomes a structured record once cleaned, aligned to its event day and annotated with the measured forward impact (2), and is finally represented as a 384-dimensional Sentence-BERT vector (3). The impacts make the theme-versus-direction point concrete in one object: the same oil-shock headline is followed by a +7.2% bounce at one day but a −9.0% slide by five days. Values are read directly from the dataset (impacts rounded to three decimals).

to be rendered from the computation rather than written alongside it. And a zero-variance window yields $z = 0$ rather than a division by infinity, so a stock that did not move at all is reported as unremarkable instead of crashing the scan.

A small example shows why this matters. Two stocks both fall 3.2% today. One has been calm ($\sigma \approx 0.40\%$), the other volatile ($\sigma \approx 1.50\%$). The same move is a clear anomaly for the calm stock ($z \approx -8.1$) but an ordinary day for the volatile one ($z \approx -2.2$), as Table 3.2 shows. A fixed percentage threshold would flag both; the z-score does not, which is exactly what we want.

Table 3.2: Worked example: the same −3.2% move on a calm and a volatile stock.

Quantity	**Calm stock**	**Volatile stock**
Recent mean μ	0.04%	0.10%
Recent standard deviation σ	0.40%	1.50%
Today's return r	−3.2%	−3.2%
z-score $(r - \mu)/\sigma$	**−8.1**	**−2.2**
Flagged at $k = 3$?	yes	no

The same numbers are easier to feel as a picture. Figure 3.4 draws both stocks' day-to-day return distributions with today's −3.2% move marked on each: the identical move lands far out in the tail of the calm stock but inside the ordinary range of the volatile one.

3.3.2 Semantic Retrieval and Impact Measurement

What it is for: given a new headline, find the most similar past headlines and show what happened to their prices. This news–market correlation engine is the heart of the work, and it combines two techniques. *Retrieval:* each headline becomes a sentence embedding with Sentence-BERT (Reimers and Gurevych 2019), chosen (Section 2.5) because it captures meaning yet yields directly comparable vectors, so the nearest past items are found cheaply by cosine similarity. *Impact:* for each precedent, the cumulative return is measured over

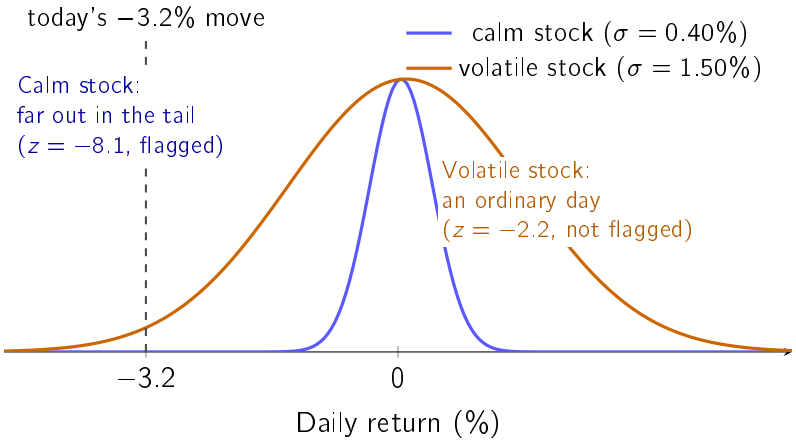

Figure 3.4: Why a fixed percentage threshold is rejected. Both stocks fall −3.2% today (dashed line). For the calm stock (narrow distribution) that move lies far out in the tail ($z = -8.1$) and is flagged; for the volatile stock (wide distribution) the identical move sits inside its ordinary day-to-day range ($z = -2.2$) and is not. The rolling z-score asks "unusual *for this stock*?", which a percentage cannot. Values from Table 3.2.

fixed post-event windows (+1, +3 and +5 trading days), in the manner of an event study (Brown and Warner 1985; Fama et al. 1969). Both the similarity metric and the windows are explicit, documented choices, and together they are the methodological contribution. Impact is measured strictly after the event, so it is an outcome, not a forecast. Algorithm 3.3 states the retrieve-and-reuse step (the case-based-reasoning core of Section 2.8).

Algorithm 3.3 Precedent retrieval and impact reporting (news trigger).

Require: headline h; knowledge base $\mathcal{K} = \{(d_j, c_j, h_j, \mathbf{e}_j, \text{impact}_j)\}$; neighbours k; horizon τ

Ensure: top-k precedents with similarity and observed impact, and their mean impact

1: $\mathbf{q} \leftarrow \text{embed}(h)$ ▷ same Sentence-BERT model as the KB
2: **for** each case $j \in \mathcal{K}$ **do**
3: $\quad s_j \leftarrow \cos(\mathbf{q}, \mathbf{e}_j)$ ▷ cosine over L2-normalised vectors
4: **end for**
5: $S \leftarrow$ indices of the k largest s_j
6: **return** $\{(d_j, c_j, h_j, s_j, \text{impact}_j[\tau]) : j \in S\}$ and $\text{mean}_{j \in S}\, \text{impact}_j[\tau]$

Two details of the embedding step deserve to be explicit, because each answers a question that naturally arises. The first is what embed(h) actually computes. Sentence-BERT is a *bi-encoder*: a pre-trained transformer reads the headline once and outputs one contextual vector per token, and the sentence embedding is the mean of those token vectors (mean pooling), scaled to unit length:

$$\mathbf{e} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{t}_i, \qquad \hat{\mathbf{e}} = \frac{\mathbf{e}}{\|\mathbf{e}\|_2}, \tag{3.2}$$

where $\mathbf{t}_1, \ldots, \mathbf{t}_n$ are the token vectors of the final transformer layer. The deployed encoder (MiniLM) is a distilled six-layer transformer producing 384-dimensional embeddings, small

enough for free infrastructure. The bi-encoder design is what makes a knowledge base practical: every stored case is embedded *once*, and a new headline costs a single forward pass plus cheap vector comparisons. The more accurate cross-encoder alternative, which re-reads query and candidate together, would need one full transformer pass per (query, case) pair, which at the scale of Chapter 5 means tens of thousands of passes for every incoming headline. The second detail is the similarity itself. For embeddings $\mathbf{q}$ and $\mathbf{e}$,

$$\cos(\mathbf{q}, \mathbf{e}) \;=\; \frac{\mathbf{q} \cdot \mathbf{e}}{\|\mathbf{q}\|_2 \, \|\mathbf{e}\|_2}, \qquad \|\hat{\mathbf{q}} - \hat{\mathbf{e}}\|_2^2 \;=\; 2 - 2\cos(\hat{\mathbf{q}}, \hat{\mathbf{e}}), \tag{3.3}$$

and the identity on the right settles the "why cosine?" question: on unit-length vectors the cosine equals the dot product, and Euclidean distance is a monotone function of it, so the three natural similarity measures induce exactly the same ranking. The choice of cosine is therefore canonical rather than arbitrary; what matters is the L2 normalisation, which stops longer texts from looking artificially "bigger" than short ones.

The impact itself is read off a short window after the event, as Figure 3.5 shows. The news may arrive on a non-trading day, so the *event day* is the first trading day on or after it; the return is then accumulated from that day's close to the close +1, +3 and +5 trading days later. Anchoring on the close, and looking only forward, is what keeps the measure free of lookahead: it records what happened *after* the event became actionable, never before.

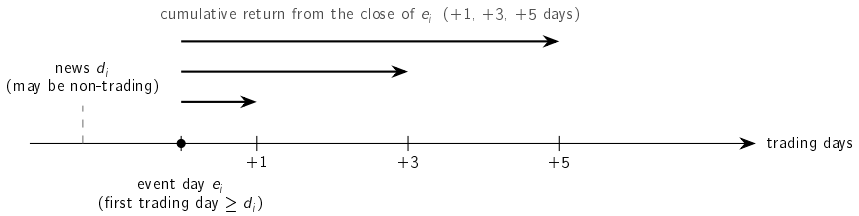

Figure 3.5: How impact is measured. The news date d_i may fall on a non-trading day; the event day e_i is the first trading day on or after it. The return is then accumulated forward from the close of e_i over +1, +3 and +5 trading days. Only post-event days are used, so no future information enters.

Three choices fix *how* impact is measured, each made for transparency as much as rigour:

- **Horizons.** Short windows (+1, +3, +5 days), standard for daily event studies (Brown and Warner 1985); longer windows let unrelated, market-wide moves creep in.
- **Baseline.** The *raw* cumulative return, not a market-adjusted abnormal return, because it is directly observable and needs no estimated market model a non-expert could not check. The cost, stated plainly, is that a raw return mixes the news reaction with any market-wide move on the same days (the confounding threat examined in Chapter 5); market adjustment is future work.
- **Aggregation.** The headline figure is the *mean* impact across the retrieved precedents, but the individual precedents are always shown too, so the investor sees the spread, not just one number.

One apparent inconsistency is worth resolving here, because two parts of the system measure the aftermath of an event differently, and both are right for their job. The *displayed* impact of a precedent uses the raw return, as chosen above: it is what the investor would actually have experienced, and it can be checked against any public price chart. The *label* of the

materiality-triage model (Section 3.3.4) instead uses the market-adjusted difference $|r^{\text{tkr}} - r^{\text{mkt}}|$, because a training target must isolate the ticker-specific move from market-wide swings; otherwise the model would learn to predict the index rather than the consequences of the news. It is the same event-study machinery answering two different questions: "what happened to this stock?" for evidence shown to a person, and "did this stock move for its own reasons?" for a label a model learns from.

A worked example makes it concrete. Figure 3.6 gives the intuition: the embedding places each headline in a space where similar themes fall close together, so a query lands near its matches and far from unrelated news.

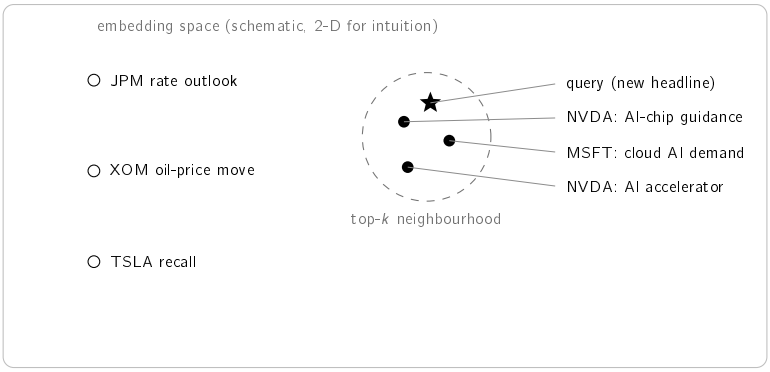

Figure 3.6: Conceptual illustration of semantic retrieval. Each headline becomes a point in a high-dimensional embedding space (drawn in two dimensions for intuition only). Texts on the same theme lie close together, so a query (star) falls near its semantically similar precedents (filled), which form the top-k neighbourhood, and far from unrelated news (hollow). The next table makes this concrete with real numbers.

Table 3.3 runs the step end to end on the small sample knowledge base shipped with the system. To keep it reproducible without downloading a model, it uses a transparent word-overlap encoder (the deployed system uses Sentence-BERT, evaluated in Chapter 5). For the query *"Nvidia demand surges on AI chip orders"*, the three nearest precedents are all AI-themed and not all from the same company: a Microsoft cloud-AI item is retrieved alongside two Nvidia items, showing the *cross-ticker* generalisation the evaluation rests on. Their mean +5-day impact is +6.5%, shown as evidence, with the explicit caveat that it records what followed comparable past news, not a forecast.

3.3.3 Attributing a Move: Market, Sector and Company

What it is for: answering the second of the three questions, *is this the company or the market?* The event study above measures what followed an event; this measures where today's move came from. They are different instruments and the thesis keeps them apart: the event study looks *forward* from the event with an implicit unit beta, and supplies the label for the triage task, whereas the decomposition explains a *contemporaneous* move with betas estimated only on prior data.

A single-factor market model would attribute everything not explained by the index to the company, which for a technology stock on a technology-led day is plainly wrong. The system therefore uses two factors, the market index and a sector ETF:

$$r_t = \alpha + \beta_m r_t^m + \beta_s \tilde{r}_t^s + \varepsilon_t, \tag{3.4}$$

Table 3.3: Worked retrieval example on the bundled sample knowledge base. Query: *"Nvidia demand surges on AI chip orders"*; similarities and impacts are real and reproducible.

Similarity	+5-day impact	Ticker / date	Retrieved headline (excerpt)
0.60	+3.5%	NVDA, 2023-05-25	Nvidia guidance surges on soaring demand for AI data-centre chips
0.38	+10.9%	MSFT, 2023-04-25	Microsoft cloud growth accelerates on AI demand
0.38	+4.9%	NVDA, 2023-06-13	Nvidia unveils new AI accelerator to extend data-centre lead
Mean +5-day impact across the retrieved precedents: +6.5%			

Note: the mean is computed from the unrounded impacts, so it need not equal the average of the rounded values shown.

where $\tilde{r}_t^s$ is the sector return *orthogonalised* against the market. The orthogonalisation is not cosmetic: a sector ETF and a broad index are strongly collinear, and without it the two coefficients absorb each other's variance and the split becomes arbitrary. The three reported contributions, $\beta_m r_t^m$, $\beta_s \tilde{r}_t^s$ and the residual ε_t, sum to the observed return by construction, which is what allows the alert to state them without the risk of a line that does not add up. Every beta is fitted on a window strictly *preceding* the day being explained, for the same anti-lookahead reason as the detector.

Shrinking the beta rather than clipping it. A beta estimated on twenty daily returns is noisy, and a noisy beta produces a confident and wrong attribution. The first implementation clipped the estimate at ± 4; measured on real data, AMD produced $\hat{\beta} = 4.43$ in a volatile window, fell through the clip into a silent fallback of $\beta = 1$, and attributed the entire -8.5% move to the company. A hard bound is exactly the kind of unjustified constant this work removes elsewhere: it does not improve the estimate, it replaces it. A fixed shrinkage weight, in the tradition of Blume (1971), is also unsatisfactory here, because it shrinks a well-estimated beta as hard as a poorly estimated one; on a synthetic case with a ticker moving at exactly twice the market, a fixed two-thirds weight returns 1.67 where the truth is 2.0. The system instead shrinks towards a prior in proportion to the *imprecision* of the estimate (Vasicek 1973),

$$\beta \;=\; w\,\hat{\beta} + (1 - w)\,\beta_{\text{prior}}, \qquad w \;=\; \frac{\sigma^2_{\text{prior}}}{\sigma^2_{\text{prior}} + \text{SE}^2(\hat{\beta})}, \tag{3.5}$$

so a clean fit is left alone ($\text{SE} \to 0 \Rightarrow w \to 1$) and a noisy one falls back towards the prior. The adjustment is continuous rather than all-or-nothing, and the prior dispersion is an economic quantity, the spread of betas across stocks, not a threshold chosen to make an example work.

Naming the driver. Reporting which factor drove a move looks trivial and is not. Taking the largest component in absolute value gives the wrong answer whenever a factor pulls *against* the observed move: on a measured day Nvidia rose $+0.25\%$ while its sector contributed -1.54%, so the largest component by magnitude was the sector, and reporting "the sector" would have been false. What explains a positive day are the positive contributions.

The driver is therefore the largest component *sharing the sign of the total*, and components that pulled the other way are reported separately rather than discarded, because a stock that rose while its sector fell is precisely the case a reader should be told about.

3.3.4 Materiality Triage: a Trained Severity Model

What it is for: to rank the incoming news stream, so that the user's limited attention goes to the headlines most likely to matter (Section 2.9), answering RQ4.

The unit is one (headline, ticker, event day) triple, where the event day d is the first trading day on or after the news date, the same alignment rule the knowledge base uses. The label asks a direction-free question by construction: did an abnormal move of unusual size follow? Formally, the example is *material* when $|r^{\text{tkr}}_{(d,\,d+h]} - r^{\text{mkt}}_{(d,\,d+h]}| \geq \tau$. The quantity inside the absolute value is the ticker's return over the h trading days after the event minus the market's (an index proxy): the market-adjusted outcome of Section 2.7. The primary setting is $\tau = 2\%$ at $h = 3$; a sensitivity grid over $\tau \in \{1.5\%, 2\%, 3\%\}$ and $h \in \{1, 3, 5\}$ is reported so the conclusion does not hinge on one threshold. Labels are produced by the system's own event-study code; no manual annotation is involved.

Features respect a strict timing convention, all computable at the close of day d, before the label window opens: the pre-event volatility (standard deviation of the twenty daily log-returns ending the day *before* the event), the five-day momentum to the same point, the event-day return itself (known at the close of d), the headline length, a sector indicator, and the sentence embedding of the headline.

The convention is enforced rather than trusted, and the mechanism is worth showing because it is the strongest single piece of evidence against leakage in this work. Listing 3.2 builds two price series that are identical up to the event day and then diverge absurdly, computes the features on both, and asserts they are equal. The complementary assertion is the one that gives the test its force: the same mutation must *change the label*. A test that only checked the features could be satisfied by a function that ignores prices altogether; requiring the label to move proves the future was genuinely reachable and genuinely excluded.

```
def test_anti_lookahead_mutating_the_future_does_not_change_features():
    base     = [100.0 + i * 0.1 for i in range(30)]
    future_a = base + [200.0, 300.0, 400.0]    # absurd future A
    future_b = base + [50.0, 10.0, 1.0]        # absurd future B
    d = len(base) - 1                          # event on the last day
    of 'base'
    assert event_features(_series(future_a), event_idx=d) == \
           event_features(_series(future_b), event_idx=d)

def test_mutating_the_future_changes_the_label_but_not_the_features():
    base   = [100.0] * 25
    up     = _series(base + [100, 110, 120, 130])   # large abnormal
    move
    flat   = _series(base + [100, 100, 100, 100])   # nothing happens
    market = _series([100.0] * 29)
    d = 25
    assert event_features(up, d) == event_features(flat, d)     #
    features identical
    assert abnormal_label(up,   market, d, 0.02, 3) == 1        #
    labels differ
    assert abnormal_label(flat, market, d, 0.02, 3) == 0
```

Listing 3.2: The anti-lookahead test. Mutating the future must leave every feature untouched and must change the label.

The temporal protocol is enforced in the same literal way. Listing 3.3 assigns blocks by *unique day* rather than by row, so every headline about a company on a given day lands in one block, and it discards a band of days after each boundary. That band is the part most often omitted in practice: with a label window of up to five trading days, a headline in the last days of the training block would otherwise be labelled by prices that belong to the validation block.

```
d = pd.to_datetime(pd.Series(dates).reset_index(drop=True))
unique_days = np.array(sorted(d.unique()))
n   = len(unique_days)
i1 = int(n * train_frac)                    # 0.70
i2 = int(n * (train_frac + val_frac))       # 0.85

train_days = set(unique_days[:i1])
emb1       = set(unique_days[i1 : i1 + embargo_days])          # discarded
val_days   = set(unique_days[i1 + embargo_days : i2])
emb2       = set(unique_days[i2 : i2 + embargo_days])          # discarded
test_days  = set(unique_days[i2 + embargo_days :])
```

Listing 3.3: Temporal split by unique day, with an embargo band after each boundary.

On the corpus of Chapter 5 the embargo removes 820 rows, which is the price paid for the guarantee and is reported rather than absorbed silently into the split.

Table 3.4 makes those columns concrete: it lists every feature, what it is, and its actual value for the worked example of Figure 3.2 (Nvidia, 10 May 2018), together with the target label. This is exactly the data on which the metrics are computed.

Table 3.4: The columns the triage model reads, with their real values for one example (Nvidia, 10 May 2018). Rows 1–6 are the features; the last row is the supervised target. The context columns (1–5) feed the logistic regressions and the gradient-boosted ensemble, scored by PR-AUC and precision@5/day; the embedding (6) also feeds precedent retrieval, scored by precision@k. Every feature is computable at the close of day d, before the label window opens.

Feature	**What it is**	**Example value**
Pre-event volatility	standard deviation of the 20 daily log-returns ending at $d-1$	0.021
Five-day momentum	cumulative log-return to $d-1$	+0.122
Event-day return	log-return of day d, known at its close	+0.017
Headline length	number of characters in the headline	55
Sector	one-hot sector indicator	tech
Headline embedding	384-dimensional Sentence-BERT vector	$[-0.005, 0.004, \ldots]$
Materiality *(target)*	$\lvert$abnormal return vs SPY over $(d, d+3]\rvert \geq 2\%$	1 (material)

Model families span the interpretability–capacity trade-off of Section 2.9: an always-alert floor, a volatility-only logistic regression (the strong naive rule any reviewer would ask about), logistic regressions on context features only, on text features only, and on both together (the main interpretable model), and a gradient-boosted ensemble (Friedman 2001) as the capacity ceiling.

The interpretable member of that family deserves its mathematics in full, because the deployed explanations decompose it term by term. With standardised features $\tilde{x}_1, \ldots, \tilde{x}_m$, the logistic regression scores an example as

$$u \;=\; b_0 + \sum_{j=1}^{m} w_j \tilde{x}_j, \qquad p_{\text{raw}} \;=\; \sigma(u) \;=\; \frac{1}{1+e^{-u}}, \tag{3.6}$$

so each product $w_j \tilde{x}_j$ is an *exact* additive contribution to the log-odds u: no surrogate model and no approximation, the property that makes the alert's "why" line faithful by construction.

That claim is easy to assert and worth showing, because "faithful by construction" is precisely the kind of phrase a reader should distrust on sight. Listing 3.4 is the whole computation behind the "why" line: the same scaler the model was fitted with, an element-wise product of coefficients and scaled inputs, and a grouping step for readability. There is no second model here, and nothing is estimated. The contributions sum to the log-odds exactly, which is what separates this from a post-hoc attribution that approximates a black box and can therefore disagree with it.

```
scaler = pipeline.named_steps["scale"]
lr     = pipeline.named_steps["lr"]
x_scaled = scaler.transform(np.asarray(x_row, dtype="float64").reshape
    (1, -1))[0]
contribs = lr.coef_[0] * x_scaled            # exact additive terms of
    the log-odds

grouped: dict[str, float] = {}
for name, c in zip(feature_names, contribs, strict=True):
    if   name.startswith("emb_"):    key = "headline content"   # 384
    dims -> one term
    elif name.startswith("sector_"): key = "sector"             # one-
    hot  -> one term
    else:                            key = _FRIENDLY.get(name, name)
    grouped[key] = grouped.get(key, 0.0) + float(c)
return sorted(grouped.items(), key=lambda kv: -abs(kv[1]))
```

Listing 3.4: The "why" line. Contributions are the model's own terms, not an approximation of them.

The grouping is a presentation decision with a cost that should be named. Summing the 384 embedding dimensions into a single "headline content" term makes the line readable, but it also means the user is told that the text mattered without being told which words did. That is an honest limit of the display, not of the model: the per-dimension terms remain exact and available, and no dimension of a sentence embedding has a meaning worth showing a non-expert anyway. Because a discriminative score is not automatically an honest probability, a Platt calibration is fitted on the validation block only,

$$p \;=\; \sigma\big(a\, p_{\text{raw}} + c\big), \tag{3.7}$$

with just two scalars (a, c). The parametric form is a deliberate choice over the non-parametric alternative, isotonic regression: a two-parameter sigmoid cannot overfit a modest validation block, whereas isotonic regression needs substantially more calibration data before its extra flexibility pays off (Niculescu-Mizil and Caruana 2005). That argument was later put to the test rather than left as an assumption. Both calibrators were fitted on the same validation block and scored on the same test set, for all five model families, under the frozen protocol: Platt wins or ties on the Brier score in every family, and the expected-calibration-error picture is mixed and by small margins. This holds even with a large validation block, which is the setting that should favour the non-parametric method, so the choice made on grounds of parsimony is also the choice the measurement supports. The comparison is additive and left the frozen artefacts untouched.

Table 3.5 makes the whole computation concrete by decomposing one *real* production decision: a Meta headline sent to the public alert channel on 12 July 2026, scored by the deployed context-only bundle. Reading the table top to bottom is reading Equations (3.6) and (3.7) with numbers in place: elevated recent volatility and the sector indicator push the log-odds up, the near-zero terms barely move it, the sum gives $u = +0.699$, the sigmoid turns it into $p_{\text{raw}} = 0.668$, and the Platt map (fitted values $a = 3.700$, $c = -2.313$) yields the calibrated $p = 0.539$, which clears the deployment gate of 0.5. The message actually delivered to the channel that day read *"Risk estimate (learned triage): 54% ... raised by recent volatility (20d) and sector"*: the decomposition reproduces the production number and its dominant factors exactly. For explanations, the embedding dimensions (when the text variant is used) are grouped into a single "headline content" term so the message stays readable, and every score is delivered with the fixed clause *"triage evidence, not a forecast"*.

Table 3.5: Worked triage example: the deployed context-only model scoring a real alert (META, 12 July 2026, *"Mark Zuckerberg Said Meta's AI Bets 'Haven't Come to Fruition Yet' as Shares Fell 5%"*). Contributions are the exact additive terms $w_j \tilde{x}_j$ of Equation (3.6); the decomposition is regenerated deterministically from the frozen inputs.

Term	**Contribution to log-odds**
Intercept b_0	-0.025
Recent volatility (20 d)	$+0.409$
Sector indicator	$+0.303$
Event-day return	$+0.010$
Headline length	$+0.002$
Recent momentum (5 d)	$+0.000$
Log-odds u (sum)	$+\mathbf{0.699}$
Raw probability $\sigma(u)$	0.668
Calibrated $p = \sigma(3.700\, p_{\text{raw}} - 2.313)$	$\mathbf{0.539}$
Deployment gate $p \geq 0.5$	passed (alert sent)

Deployment is deliberately conservative. The score sits behind a configuration gate that is disabled in the default configuration and enabled in the reference deployment; the deployed variant uses context features only, so it runs on the light software stack, and the alert text names the variant used.

What happens after deployment needs stating precisely, because the surrounding vocabulary invites an overclaim. Every triage decision is logged as it is made. A post-validation step,

run days later, labels each matured decision with what actually happened, using the same rule that produced the training label, and reports live precision and calibration. That is *monitoring and label collection*. It is not learning: no model is refitted, no weights change, and the artefact serving inference is the same file that was fitted once and frozen. The distinction matters because four things are routinely conflated. *Inference* is scoring a new item with fixed weights, which is what runs every cycle. *Training* is fitting those weights, which happened once, offline, on the historical corpus. *Retraining* is repeating that fit on newer data, which the collected labels make possible and which has not been performed. *Continual learning* is retraining automatically as data arrives, which this system does not do and is not claimed to do. What exists is the precondition for the third: a growing store of decisions labelled by outcome under the training rule, together with the command that would consume it. Section 6.5 reports what that store currently says, and it is not flattering.

3.3.5 Explanation Techniques

What it is for: to turn the computed objects into a message the investor can check. Explanation is pursued by transparent design (Section 2.3) (Barredo Arrieta et al. 2020; Rudin 2019). Each alert is assembled from three transparent ingredients: the rule that fired (the z-score with its window and threshold); the retrieved precedents with their observed impact; and, optionally, feature attribution over the detector with SHAP (Lundberg and S.-I. Lee 2017). Financial sentiment from a domain model (Araci 2019) is an optional enrichment, used only if it adds defensible value. Because every alert is rendered directly from these objects, the explanation is faithful to the computation by construction, a property tested in Chapter 5.

3.4 Tools and Frameworks

The system is built on the open-source scientific-Python ecosystem: mature, free libraries for numerical computing, market-data access, sentence embeddings and figures. This fits the free-resources constraint and reproducibility, since pinned dependencies and fixed seeds let the environment and results be recreated elsewhere. The reproduction environment and the evaluation outputs are collected in Appendix A, so this chapter can focus on the *methods* themselves.

3.5 Responsible AI and Ethics

The work is bound by several responsible-AI commitments that follow directly from its purpose and its audience. **No advice and no prediction:** the system performs neither price prediction nor trading recommendation; it surfaces observed past outcomes as evidence and states this explicitly in every alert, so that the investor is informed rather than instructed. **Transparency:** every alert exposes its full reasoning chain, in keeping with the explainability principles of Chapter 2, so that a user can verify rather than merely trust the output. **Honest evidence:** only results that can be reproduced and explained are reported, and no data, metric or citation is fabricated. **Data stewardship:** all sources are used within their free tiers and licences, the FNSPID dataset is attributed as its Creative Commons licence requires (Dong, Fan, and Peng 2024), third-party news text is not republished, and no personal data is processed. **Security:** credentials are never stored in versioned files. Finally,

the system is positioned as a decision-support aid for a non-professional investor, not as a substitute for professional advice, a limitation stated plainly.

3.6 Evaluation Methodology

How is the system judged? By a plan fixed before any experiment ran, so the metric cannot be chosen after the fact to flatter the result, a small-scale pre-registration. The results are in Chapter 5; here is the protocol they follow.

3.6.1 Retrieval metric and relevance

Retrieval is evaluated as an information-retrieval task (Section 2.6) with precision@k, the share of the top k precedents that are relevant. Writing Q for the set of query headlines and $R_k(q)$ for the top k items returned for query q,

$$\text{precision@}k = \frac{1}{|Q|}\sum_{q\in Q}\frac{1}{k}\sum_{d\in R_k(q)}\text{rel}(q,d), \qquad \text{rel}(q,d) = \mathbf{1}\big[\text{sector}(d) = \text{sector}(q)\big]. \tag{3.8}$$

Relevance is approximated by *sector agreement*: a precedent counts as relevant if its company is in the query's sector. This is an automatic stand-in for "analogous", used because no labelled set exists; its limits are stated with the results. Two choices make the metric demanding, not flattering. First, a strict *cross-ticker* constraint drops every precedent sharing the query's ticker, so the system cannot win by matching a company to its own past news; it must generalise *across* companies in a sector. Second, k is small (the headline figure uses $k = 5$), reflecting that an alert shows only a few precedents.

3.6.2 Baselines

A single number means little alone, so semantic retrieval is compared against three baselines, each controlling for a rival explanation. **Random** draws k precedents at random; its precision is the sector base rate, the no-skill floor. **Recency** returns the newest items, testing whether simply surfacing fresh news would do as well. **Lexical** ranks by word overlap under the same cosine, so the gap to the embedding model isolates the value of *meaning* over surface words. Each comparison is repeated over five random seeds (mean with standard deviation), and two Sentence-BERT variants are tried, so the result reflects semantic retrieval in general, not one lucky draw or one model.

3.6.3 Anomaly-detection protocol

The anomaly detector is judged primarily by an argument that needs *no label*: a good universal detector should fire at a comparable rate across instruments of very different volatility, whereas a volatility-blind rule cannot. Consistency is therefore measured as the range (maximum minus minimum) of per-ticker firing rates: the smaller the range, the more uniform the detector. Only secondarily is the detector scored against an explicit *extreme-move* proxy (a daily move at or beyond the 99th percentile of that ticker's own returns), via precision, recall and F_1; because this proxy is itself volatility-relative, it is treated as supporting rather than primary evidence. An ablation over the estimation-window length completes the picture by exposing the detector's sensitivity to its one main parameter. Finally, the transparent rule is challenged by a *learned* detector on equal terms: an Isolation

Forest (F. T. Liu, Ting, and Zhou 2008) trained causally on the same information (each day's return and the volatility of the twenty preceding days, fitted only on an initial past segment) is scored on the same evaluation region, so the choice of a statistical rule over a learned one is tested, not assumed. Two extensions harden that comparison further. The Local Outlier Factor (Breunig et al. 2000), cited in Chapter 2 as the density-based alternative, is evaluated under the identical causal protocol, so the statistical rule faces *two* learned detectors rather than one. And the rolling volatility estimate is challenged by an exponentially weighted variant (RiskMetrics-style, $\lambda = 0.94$), the first empirical step toward the GARCH family of Section 2.2, so the natural "why not GARCH?" question receives evidence rather than opinion.

3.6.4 Triage protocol

The materiality model of Section 3.3.4 is evaluated under a strict temporal protocol. Examples are split into training, validation and test blocks chronologically, by *unique event day* (70/15/15), so that all headlines from one day land in the same block, and an embargo of several trading days is removed after each boundary so that no label window can straddle two blocks. Calibration is fitted on the validation block only; the test block is touched once, to report. The primary metric is the area under the precision–recall curve, whose floor is the test prevalence (an always-alert strategy); the area under the ROC curve and the Brier score complete the picture, and a product-shaped metric, precision within a budget of N alerts per day, measures what the user actually experiences. The metrics themselves are standard but worth fixing precisely. Sweeping a threshold s over the scores traces $(\text{recall}(s), \text{precision}(s))$; PR-AUC is the area under that curve, and a scorer with no skill earns exactly the prevalence π, which anchors every comparison. ROC-AUC is the probability that a random positive outscores a random negative; it is reported for completeness but not used as primary, because it rewards ranking the abundant easy negatives and so flatters a triage model whose user only ever sees the top of the ranking. Calibration is measured by the Brier score,

$$\text{Brier} \;=\; \frac{1}{N}\sum_{i=1}^{N}\left(p_i - y_i\right)^2, \tag{3.9}$$

the mean squared gap between the stated probability p_i and the outcome $y_i \in \{0, 1\}$: it is low only when the probabilities can be taken at face value, which is exactly what a user reading "57%" deserves. The decisive comparison is against the volatility-only logistic regression: the learned model is claimed useful only where it beats that baseline, and the feature ablations (context only, text only, both) attribute any gain to its source. Sensitivity over the $\tau \times h$ label grid is reported alongside. Two further guarantees are pre-committed. Training is deterministic given the seed, and rerunning it reproduces the stored artefacts exactly. And the result is reported whichever way it falls: if the trained model fails to beat the volatility baseline on a given corpus, that finding stands as stated.

3.6.5 Uncertainty, drift and unsupervised structure

Three further protocols examine the system from the outside, asking what it does not know. All three are additive: they read the frozen artefacts and change nothing about them.

Conformal coverage. The frozen triage model is wrapped in split conformal prediction (Section 2.10). The nonconformity score is $1 - \hat{p}(y)$ for the true class y, and the threshold

$\hat{q}$ is the $\lceil (n+1)(1-\alpha) \rceil / n$ empirical quantile of the calibration scores; that finite-sample correction, rather than the plain empirical quantile, is what turns a tendency to cover into a guarantee. A label enters the prediction set when $\hat{p}(y) \geq 1 - \hat{q}$. Two quantities are reported at each α: empirical coverage, which tests whether the guarantee holds, and average set size, which measures what it costs. Because the guarantee depends on exchangeability, the protocol is run twice, once on a random split of the test block, where exchangeability holds by construction and the run therefore checks the implementation, and once on a temporal split, which is what deployment actually does. Before any of this, the model is required to reproduce its frozen test PR-AUC exactly; if it does not, the run stops, because nothing downstream would then describe the model this dissertation reports.

Distribution drift. Two complementary statistics compare the training block against a later one. The Population Stability Index sums $(p_{\text{cur}} - p_{\text{ref}}) \ln(p_{\text{cur}}/p_{\text{ref}})$ over quantile bins of the reference distribution, and carries conventional interpretation bands (< 0.10 stable, 0.10 to 0.25 moderate, above 0.25 significant) inherited from credit-risk practice. Quantile bins rather than equal-width bins are used because the financial features are strongly skewed, and equal-width bins would place almost all mass in the first one. The Kolmogorov–Smirnov statistic compares cumulative distributions. One reading rule is fixed in advance: with tens of thousands of points the associated p-value rejects almost any null hypothesis, so the *statistic* is reported as an effect size and the p-value is not used to argue severity. Bins that the current sample does not visit receive a small floor rather than producing an infinite index, since an unvisited bin is evidence of drift and not a division by zero.

Unsupervised event structure. Headline embeddings are clustered with k-means over L^2-normalised vectors, and k is chosen by the silhouette coefficient (Rousseeuw 1987), which contrasts each point's cohesion within its own cluster against its separation from the nearest other cluster. Because there is no ground truth for event type, agreement is measured against a keyword rubric that is written and committed *before* any clustering is run, so that it cannot be tuned into agreement afterwards. Two safeguards make the resulting numbers interpretable. Cluster purity is compared against a random assignment with identical cluster sizes, because purity computed with majority labelling inflates when the reference is imbalanced and k is large. And when the clusters are compared against different references, to ask whether they organise by event type or merely by subject, agreement is measured with adjusted mutual information (Vinh, Epps, and Bailey 2010) rather than purity: purity depends on the number of classes and their balance, so comparing it across references with different cardinalities compares incommensurable quantities, whereas the adjusted measure has a chance baseline close to zero that is not biased towards any particular number of classes. All such comparisons are computed on identical rows.

3.6.6 Explanation, scope, and rigour guarantees

The explanation engine is assessed for *fidelity*: whether the alert text reflects the system's actual computation. This holds by construction, because each alert is rendered directly from the computed objects. Its *usefulness* to a real investor is a separate matter, which would need a human study not yet conducted, and is reported as a limitation rather than a claim. Because the system performs neither price prediction nor trading, profit-based metrics are out of scope by design, not omitted by convenience. Underpinning every experiment are three guarantees. First, **no lookahead**: any feature computed at an instant uses only information available at or before that instant, and historical impact uses only post-event

windows. Second, **reproducibility**: random seeds are fixed, dependencies are pinned, and every data and results figure is produced by a versioned script. Third, **honesty**: results are reported with their caveats, and modest but genuine findings are preferred to inflated ones.

3.7 Chapter Conclusions

This chapter set the method: defensible simplicity and incremental delivery; a two-layer data architecture in a reproducible data card; the techniques for detection, retrieval, impact and explanation, each introduced by its purpose; and an evaluation fixed in advance. The next chapter assembles these into InvestiGator.

Chapter 4

InvestiGator: An Explainable Financial-Alert System for Retail Investors

This chapter answers five questions, in order: *What is the system, in one picture? What objects does it work with? What are its parts, and who does what? How does data flow through it? And how does it decide?* Chapter 3 argued for the individual methods; here they fit together into one working system, InvestiGator.

4.1 Overview

What is the system, in one picture?

InvestiGator watches the US equity market for a retail investor and raises an alert when one of two independent triggers fires: an abrupt price movement, or a relevant incoming news item. Every alert carries a full, traceable explanation, not a bare notification. The system is a pipeline of single-responsibility components linked by a common data schema, so the live and historical layers are directly comparable. Figure 4.1 is the one-picture view: two data sources and the knowledge base feed two engines, whose evidence converges on one explanation step before delivery.

Three engineering principles keep the system testable and defensible. First, **a thin slice was built end to end before breadth**: the market trigger worked from price to delivered alert before the heavier correlation engine was added, which controlled integration risk. Second, **pure logic is separated from input/output**, so the numerical core runs and is tested offline, without a network or the model stack. Third, **components are programmed to interfaces**: the embedding step has two interchangeable implementations (a dependency-free lexical baseline and Sentence-BERT), which is what makes the model comparison in Chapter 5 fair. Table 4.1 gathers the principal design decisions; in a work of this kind, that engineering judgement *is* the contribution.

4.2 Use Cases

What does someone actually do with this system?

The architecture above is easier to judge against the situations it was built to serve. Figure 4.2 places them in relation to the two user profiles of Section 1.2, and Table 4.2 states, for each, the question being asked and the components that answer it. The mapping is

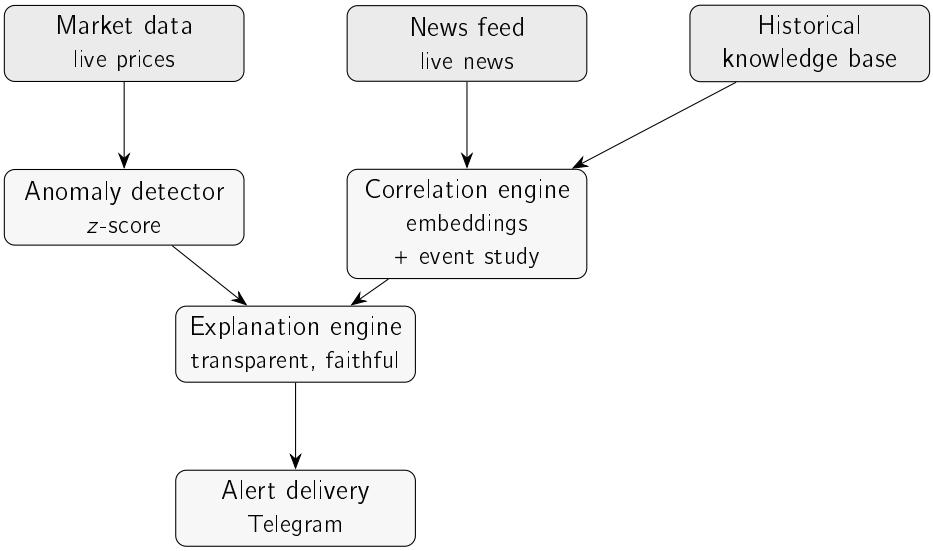

Figure 4.1: Architecture of InvestiGator. The live layer (market data, news feed) and the historical knowledge base feed two engines (the anomaly detector and the correlation engine) whose evidence converges on the explanation engine before an alert is delivered through Telegram. The market-movement trigger follows the left path; the news trigger the right.

deliberately strict: a component that no use case exercises would be a component without a justification.

Two of these deserve comment because they shaped the design rather than merely consuming it.

UC2 is the use case that motivated the market-versus-company decomposition. Without it the system could report that a holding fell sharply while leaving the reader unable to tell alarm from noise, which is precisely the failure mode identified for the long-term holder.

UC4 is the only use case that is legitimately *ahead* of events, and it is worth being precise about why that does not breach the no-prediction stance. A scheduled event is a matter of public record, not a forecast, and the historical reaction profile reports the distribution of past moves around such events rather than a claim about the next one. The system says what is known and what happened before; it does not say what will happen.

4.3 The Data Model

What objects does the system work with, and how do they relate?

The central object is a *case*: one past news item together with what happened to its price afterwards. Everything else exists to build cases, store them, and retrieve them. Figure 4.3 shows the objects and their relationships.

Two points make the model easy to hold in mind. First, the *shared schema*: a live news item and a historical case carry the same date, ticker and headline, so a fresh headline can be compared directly against the stored cases. A case is simply a news item that has been enriched with its headline *embedding* and its measured *impact* at +1, +3 and +5 trading days. Second, the *knowledge base* is just a collection of cases; asking it for precedents

Table 4.1: Principal design decisions in InvestiGator and their rationale.

Decision	Choice	Rationale
Detection method	Rolling z-score	Transparent; every alert is explainable to a non-expert
News representation	Sentence-BERT embeddings	Contextual meaning, yet cheap and reproducible to compare
Impact measurement	Event-study windows	Standard, reproducible; characterises an outcome, not a forecast
Explanation strategy	Transparent design + precedents	Faithful by construction, not a post-hoc approximation
Data architecture	Shared schema, two layers	Live and historical evidence are directly comparable
Component coupling	Programmed to interfaces	Testable offline; enables fair model substitution
Delivery channel	Telegram bot	Free, ubiquitous, no bespoke client needed

returns the top-k most similar cases with their similarity scores. The market side is simpler still: the detector emits one *detection result* holding the move and every quantity behind it (z, μ, σ, window, threshold). These few objects are all the explanation engine ever needs.

4.4 Components and Responsibilities

What are the parts, and who does what?

InvestiGator is one package per component, each with a single responsibility and a small interface, which is what lets the numerical core be tested offline and one implementation be swapped for another. Table 4.3 lists them with their one job and the data they turn into.

4.5 Data Flow

How does data move through the system, end to end?

Figure 4.4 traces the whole pipeline as three lanes that converge. The *offline* lane (top) is run once: FNSPID news is aligned to prices and embedded into the knowledge base (Dong, Fan, and Peng 2024). The two live triggers run online. The *news* lane (middle) embeds an incoming headline, finds its nearest cases in the knowledge base, and attaches their measured impact. The *market* lane (bottom) turns live prices into a z-score and fires only when it crosses the threshold. Both lanes hand their evidence to the explanation engine, which delivers one alert through Telegram.

One item, end to end

The diagram shows the shape of the pipeline; what it cannot show is what the data actually *is* at each step. Because a reader of this dissertation cannot inspect the running system,

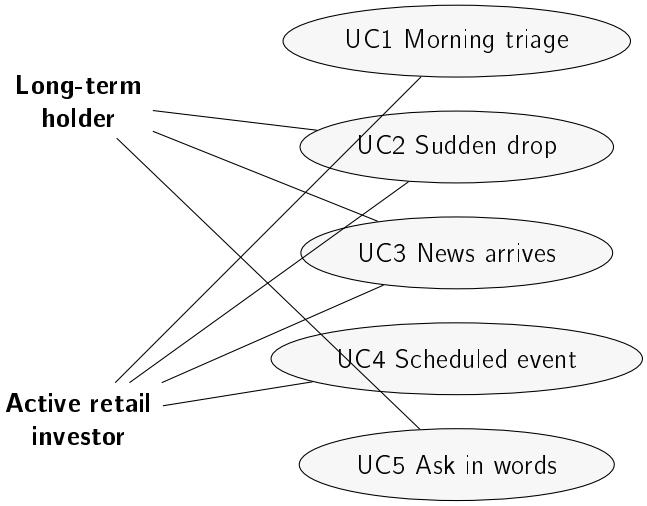

Figure 4.2: The five use cases and the profiles that exercise them. The long-term holder reaches the system when something worries them; the active investor uses it as a daily surface. Both converge on the same evidence, which is why a single explanation format serves both.

Table 4.2: Use cases, the question each answers, and the components involved.

ID	Question the user is asking	Components exercised
UC1	What in my watchlist deserves attention today?	Anomaly detector, learned triage, ranking
UC2	Why is this down, and is it the company or the market?	Decomposition, news linking, explanation
UC3	This headline arrived; has it mattered before?	Relevance filter, retrieval, event study
UC4	What is coming that could move this?	Scheduled-event lookup, historical reaction profile
UC5	Can I just ask, in plain words?	Grounded narrator over the components above

this subsection follows one real news item from the provider's response to the message on a phone, naming the form the data takes and where it rests at every stage. The item is the Microsoft headline the channel received on 9 August 2026; Table 4.4 is the same journey in one view.

1. Source and retrieval. A worker process wakes on a sixty-second cycle and requests company news for each watchlist ticker from a free news API over HTTPS, with the API key supplied by the environment and never stored in the codebase. The response is JSON: a list of objects carrying a headline, a summary, a source name, a URL and a publication timestamp in Unix epoch seconds.

2. Normalisation and what is deliberately discarded. Each object becomes an internal news item with four fields: date, ticker, headline and publication instant. The provider's *summary* is read for relevance filtering and then dropped rather than stored, because retaining third-party article text would republish licensed content; only the headline, which is the

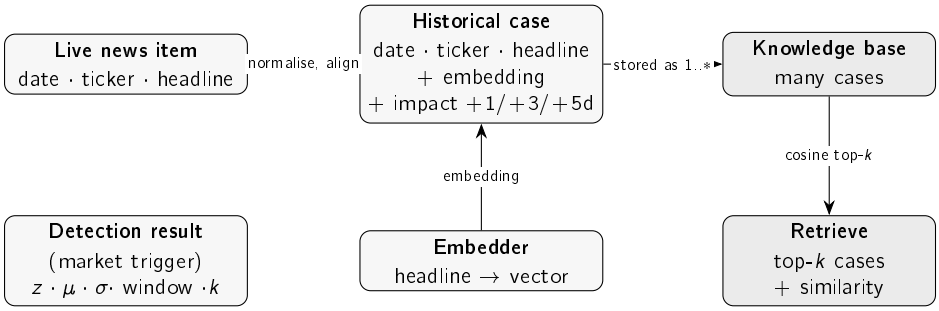

Figure 4.3: The objects InvestiGator works with. A **live news item** and a **historical case** share the same schema (date, ticker, headline); the case adds the headline's embedding and its measured post-event impact. The **embedder** produces those vectors; the **knowledge base** holds many cases; a query headline retrieves the top-k most similar cases with their similarity scores. The market trigger produces a separate **detection result**.

minimum needed to compute a similarity, is kept. The epoch timestamp is preserved to the second, which is what later makes the latency decomposition of Chapter 6 possible at all.

3. Relevance. The headline must name the company, matched against an alias list ("Microsoft", "MSFT", and the names of its principals), and must not match market-roundup boilerplate. This gate exists because the provider tags loosely: a law-firm notice and a "top S&P 500 movers" list both arrive labelled with the ticker.

4. Representation. The surviving headline is encoded into a 384-dimensional unit-length vector by the quantised sentence encoder, on CPU, in a few milliseconds. This is the one point in the pipeline where a neural network runs in production.

5. Storage. Two things are written. The vector plus its metadata is appended to a *pending* case file, where it waits until its own five-day forward impact becomes observable and it can mature into a precedent. Separately, the decision about to be taken is appended to a decision log. Both files live on a version-controlled data branch rather than on the dyno's own disk, because the dyno's filesystem is ephemeral and is not shared with the process that serves the dashboard.

6. Retrieval. The vector is compared by cosine against every stored case, and the top three are taken, ranked with an age-decay factor while the similarity displayed remains the true cosine. For this item the nearest neighbours were three Microsoft headlines from 2023 at similarities 0.70, 0.57 and 0.55.

7. Measured outcome. Each retrieved precedent already carries the return that followed it at +1, +3 and +5 trading days, measured when the case matured and never recomputed at query time. Here they ranged from −1.67% to +4.79% at five days.

8. Features and score. In parallel, the ticker's recent prices yield the context features of Section 3.3.4, and the frozen model turns them into a calibrated probability with its exact additive contributions.

9. Gates. The item passes a freshness check, an evidence floor on the best similarity, the materiality gate, the per-company daily cap with its escalating floor, and a cross-producer duplicate check. Any one of these can end the journey silently, which is why the screener surface exists to report which one did.

Table 4.3: InvestiGator's components, each with a single responsibility.

Component	Responsibility	Input → output
Market data	Fetch prices, compute log-returns	ticker → daily returns
News feed	Fetch and normalise live news	ticker → news items
Embedder	Turn a headline into a vector	headline → embedding
Knowledge base	Hold past cases; retrieve the nearest	headline → top-k cases
Anomaly detector	Flag volatility-relative moves	returns → detection result
Correlation engine	Cosine similarity + event-study impact	headline, KB → precedents + impact
Triage model	Learned severity: calibrated P(abnormal move follows)	headline + context → probability + contributions
Explanation engine	Render a faithful alert from the objects	objects → alert text
Alert delivery	Send the alert to the investor	alert text → Telegram
Orchestration	Run each trigger end to end	trigger → delivered alert

10. Rendering and delivery. The alert text is assembled directly from the objects above and sent to the channel; the same record is appended to the shared history, from which the dashboard later draws it. Delivery took under a second from detection. The message the user received is reproduced in Section 4.8.

Table 4.4: One real news item (Microsoft, 9 August 2026) at every stage, with the form the data takes and where it rests. Read top to bottom, this is the whole system.

#	Stage	Form of the data	Where it rests
1	Provider request	JSON over HTTPS	in memory
2	Normalisation	date, ticker, headline, instant	in memory (summary discarded)
3	Relevance filter	same, or dropped	—
4	Encoding	384-d unit vector	in memory
5	Capture	vector + metadata; decision record	data branch (two files)
6	Retrieval	top-3 cases + cosine	in memory
7	Outcome lookup	returns at +1/+3/+5 days	read from the stored case
8	Features + score	9 numbers → probability + terms	frozen model file
9	Gates	keep or drop, with the reason	gate log (data branch)
10	Delivery	message text	Telegram + shared history

Two properties of that path are worth naming because they are design decisions rather than consequences. Nothing between steps 4 and 8 requires the network, so a provider outage degrades the system to silence rather than to error. And every step that discards an item records why it did, which is what makes a quiet day distinguishable from a broken one.

One detail in the offline build matters for honesty: each news item is aligned to the first trading day on or after its date, and impact is measured only from that day's close onward,

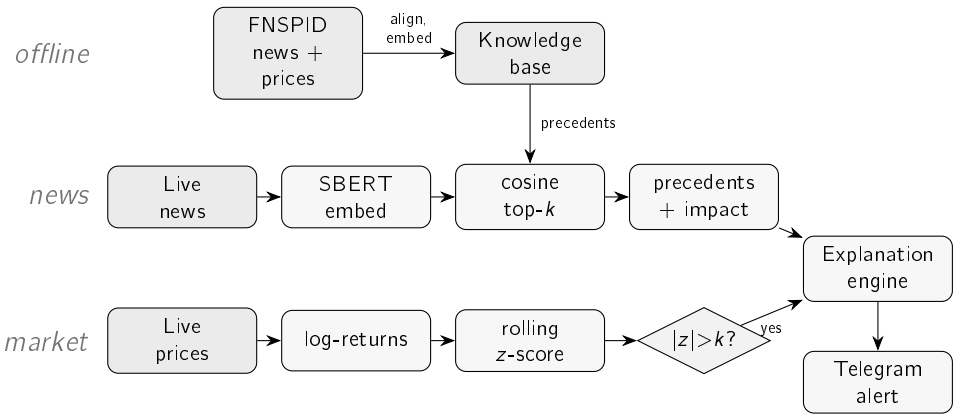

Figure 4.4: The end-to-end flow of InvestiGator as one connected pipeline. *Offline* (top), FNSPID news is aligned to prices and embedded into the knowledge base. The *news trigger* (middle) embeds an incoming headline, finds its nearest precedents in the knowledge base and attaches their observed impact; the *market trigger* (bottom) turns live prices into a z-score and fires only when it exceeds the threshold. Both paths converge on the explanation engine, which delivers one explained alert through Telegram. This is the simplified view; the complete pipeline, with every decision gate and its live configuration value, is Figure A.1 in Appendix A.

so a jump already in the opening price is never credited to the system. Building the full multi-year corpus is a long one-off job; it was run, and it supplies the 79,753 examples behind the triage study of Chapter 5 and the scale re-run of the retrieval result. The retrieval case study itself uses a knowledge base built from real recent news through the identical process, and what remains outstanding is narrower than the build: the market-adjusted impact-magnitude study over those multi-year precedents.

4.6 Decision Logic

How does the system actually decide? Each trigger applies one simple, transparent rule.

Market trigger. Flag the day when the absolute z-score crosses the threshold, $|z_t| > k$, with z_t computed from the trailing window as in Equation 3.1. The detector returns not just the yes/no decision but every quantity behind it (z, μ, σ, window, threshold), which is exactly what makes the explanation faithful. The same rule is applied across whole price series in the evaluation, so the experiments use the production logic.

News trigger. Embed the incoming headline, rank the knowledge base by cosine similarity, and take the top-k cases. For each, report the observed impact at +1, +3 and +5 days, measured from the event-day close, and show their mean as evidence. Nothing here forecasts: the figures are what *did* happen after comparable past news, and the knowledge base is consulted but never decides.

Learned severity (triage). On top of the two transparent rules sits the optional trained component of Section 3.3.4: each incoming news item is scored with the calibrated probability that an abnormal move follows, and the alert can be *gated*, sent only when the score

clears a configured threshold, and *annotated* with the score and its top additive contributions. The gate fails open: with no model file, or too little price history to compute the context features, the pipeline behaves exactly as before. It ships disabled in the default configuration and is enabled in the reference deployment, where it acts as the volume control examined in Section 4.9. The deployed variant uses context features only, so it runs without the heavy embedding stack.

The wording of that line deserves its own paragraph, because an earlier version of it was *dishonest* and the correction is the clearest statement of where this system's no-prediction rule actually falls. The alert used to close with "triage evidence, not a forecast". That was false. A probability that an unusually large move will occur over the next few days is, by definition, a statement about the future, and the project's own interface criteria bar this number from every dashboard view for exactly that reason. Claiming in the alert that it is not a prediction, while banning it from the screen because it is one, is a contradiction an examiner would find. The distinction that *is* true, and the one the training label supports, is between *materiality* and *direction*: the label asks whether an abnormal move of unusual size followed, in either direction, and never which way it went. The line therefore now states the probability, names the two features driving it, says explicitly that it is about whether the market reacts and never about which way, and disclaims price forecasting and advice. The system does predict one thing, narrowly and by design, and the honest course is to say which thing rather than to deny predicting at all.

Budgeting by materiality rather than by arrival. A daily cap of two news alerts per company protects the reader from fatigue, but a cap is only as good as the rule deciding which two get through. The runner originally walked each day's headlines in the order the feed returned them, so two immaterial stories in the morning could exhaust a company's budget and a genuinely material story that afternoon was discarded without trace. The fault was found in operation rather than in testing, on a day when Nvidia rose sharply on a supply announcement: the channel carried two lesser stories about that company and never carried the one that moved it. The uncomfortable part is that the system already held the right instrument. The triage component of Section 3.3.4 exists precisely to rank news by materiality, and Chapter 5 reports that within a five-alert budget it raises precision from 0.163 to 0.632 on held-out data; the cap simply never consulted it, so the score was measured and then ignored.

The first repair was to serve the cap in descending order of that score, and it did not work. The failure is recorded rather than quietly replaced, because its cause is a property of the deployment that no component test could expose: sorting can only reorder candidates present together, and the scan emits at most one headline per company per cycle, so two stories competing for the *same* company's quota never coexist in one batch. Across a day the order therefore remained the order of arrival, which was the original defect. The test written alongside that change compared three headlines for one company in a single call, a situation production cannot produce, and a test that passes over an impossible scenario is indistinguishable from a fix that works.

What governs the cap instead is an *escalating floor*: the k-th alert for a company on a given day must clear a higher calibrated probability than the one before it. The floors are derived rather than chosen. Sweeping the decision threshold under an explicit cost ratio R between a missed move and a false alarm (Chapter 6) gives $\tau^\star = 0.49$ at $R = 1$, which is the first alert of the day, where the two errors cost about the same, and $\tau^\star = 0.64$ at $R = 0.5$,

which is the second, where interrupting an already-interrupted reader makes fatigue the dominant cost. No third and stricter floor was added: the highest score the model produces on the frozen test set lies between 0.65 and 0.66, so a floor above that could never fire, and dead code with the appearance of rigour is worse than no rule. The mechanism is visible in live operation, where the runner logs refusals of the form *"second alert of the day requires* $p \geq 0.64$ *and this one has* 0.58*"*.

One limit is structural rather than an implementation gap, and stating it is more useful than claiming the problem is solved. The first slot is still spent on the first headline that clears the ordinary gate, because at the moment of that decision the afternoon's story does not yet exist: quota cannot be reserved for news not yet seen, and a delivered alert cannot be recalled. What an online rule can do is make each additional slot more expensive, which is what this one does. Ordering by materiality is retained for the smaller benefit it genuinely delivers, deciding which company is announced first when several qualify within one cycle.

The same story told twice. Deduplication keyed on the exact alert text, so one story filed by two outlets under different headlines produced two alerts. Alerts are now also compared on the content words of the *headline*, above a similarity threshold. That the comparison runs on the headline and not on the rendered alert is deliberate and was itself a corrected mistake: the alert is largely fixed template, so two genuinely different stories about one company share most of their words, and comparing rendered alerts would have suppressed a legitimate second alert in silence. The check also declines to judge headlines with very few content words, where any two strings agree; there it fails open, because a repeated alert costs the reader far less than a missing one.

4.7 Explanation and Delivery

How is the alert explained, and how does it reach the investor?

The explanation engine renders every alert directly from the objects above, so the text cannot diverge from the computation. For the market trigger it states the move, the z-score, the threshold and window, and the recent mean and standard deviation, and it puts the z-score in plain language (how many times a typical day's move it represents) so a non-expert can read it. For the news trigger it lists the retrieved precedents (date, ticker, similarity, measured impact, headline) and their average impact, and always ends with an explicit disclaimer that these are observed past outcomes, not a prediction.

Delivery is over Telegram, chosen for its free, ubiquitous bot interface. A real alert was delivered during testing; Figure 4.5 shows the format the investor receives, with the whole reasoning chain in one message. In the deployed system the message layout was later compacted for readability (the strongest fact first, the method in a short footnote, and the range of precedent outcomes shown alongside their average); the fields carried are exactly the ones above, so the faithfulness argument is unchanged.

The same reasoning is also published as a live dashboard (Figure 4.6), and its organising decision is worth stating first because it is the clearest link between this chapter and Chapter 1. The three questions a retail investor asks (Section 1.2) become three *named sections* on every card, in the same order every time: is this unusual for this stock, is it the company or the market, and has this happened before. They appear even when the answer is that nothing happened, because a question that is present only sometimes teaches the reader

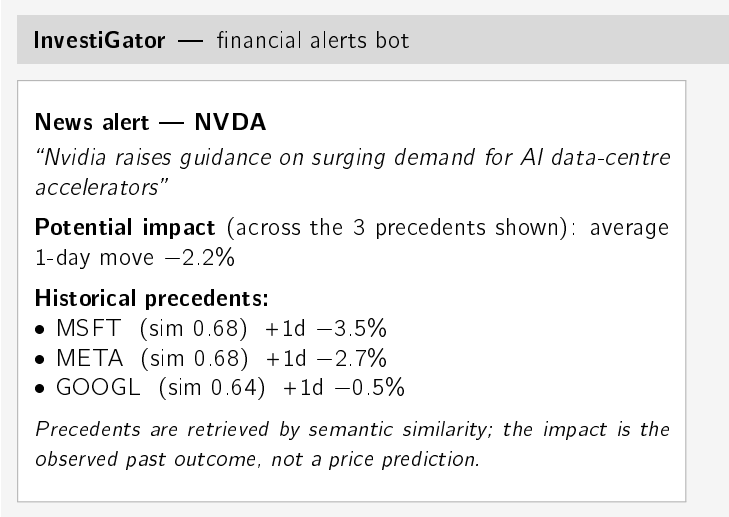

Figure 4.5: An InvestiGator news alert as delivered through Telegram. A single message carries the full, traceable reasoning chain: the detected event, the average observed impact of analogous past events, the individual precedents with their similarity scores and measured impact, and an explicit no-prediction disclaimer. The figures shown are a representative, rounded illustration; the full real end-to-end output appears in Chapter 5 (Case Study 3).

not to look for it. The opening surface is a grid with one card per watched company, none privileged, ordered by how rare the day was rather than alphabetically.

Three further surfaces complete it. A per-company detail view holds the price chart with the detector *replayed* over the past year so the marks are the same rule the system runs live rather than annotations added by hand, the market/sector/company split as bars that sum to the observed move, and the alerts exactly as the Telegram channel received them, read from the one shared versioned record rather than recomputed. A screener answers the opposite question, why each company was *quiet*: it lists every name the scan examined, the gate that stopped it, and the margin by which it fell short. This surface exists because silence is a decision the system makes, and a decision that cannot be inspected is indistinguishable from a system that is not running. A method page carries the frozen evaluation numbers, including the negative result of Chapter 5, each shown beside the file it comes from, for a reader who wants to check the method before trusting the output.

Three choices within these surfaces were made against earlier drafts rather than inherited. First, rarity is expressed as an *empirical count* – "5 of the last 249 trading days moved this much or more" – and not as a probability. Converting a z-score into a probability would require assuming normality, and daily equity returns have heavier tails than that, so the resulting figure would be least trustworthy on precisely the days the system exists to explain. A count assumes nothing and is checkable by anyone holding the same price series. Second, the card names the *driver* of a move in words while the signed three-way split sits one click away; an earlier design placed all three components on every row, and with a watchlist of twelve that is thirty-six signed numbers competing for attention on first contact, which defeats the purpose of ranking at all. Third, the promise and the no-prediction commitment appear exactly once per page, as its footer, rather than beside every card: repetition had made an earlier interface read as defensive without adding information, while dropping the statement altogether, which a later revision briefly did, removes the one ethical commitment

the system makes out loud.

The two measures of unusualness can disagree, and the interface says so rather than choosing the more comfortable reading. The detector judges a day against the previous twenty; the count judges it against the year. A company in a calm stretch can therefore fall below the alert threshold while still sitting near the top of its annual range, and the card reports both facts — normal for recent weeks, rare for the year — and leaves the reader to weigh them.

One capability is deliberately absent from the page, and saying so is more useful than implying otherwise. Precedent retrieval does not run in the browser: loading the sentence encoder was measured to add about seven seconds to a cold page load, which is most of the budget the interface has. The detail view states this in place rather than leaving a silent gap, and points to where the answer does live, namely the news alerts themselves, each of which carries its own retrieved precedents and their measured outcome at +1, +3 and +5 days.

That budget is also why the page performs no computation of its own. The worker writes a precomputed snapshot at the end of each cycle, where the cost of fetching prices has already been paid, and the page reads it; measured, building the grid from the network costs about five seconds against eleven milliseconds to read the file. Because a snapshot that silently stopped being written would look exactly like a calm market, the page displays the snapshot's age, so staleness is visible on screen rather than inferred.

4.8 The Life of One Alert

What does the whole pipeline do, concretely, from raw feed to phone? The cleanest answer is to follow one *real* alert through every stage. The case is the same production decision decomposed in Table 3.5: on 12 July 2026 the public channel received an alert for Meta on the headline *"Mark Zuckerberg Said Meta's AI Bets 'Haven't Come to Fruition Yet' as Shares Fell 5%"*. Table 4.5 walks the stages with the values the system actually computed; every row is verifiable in the shared history record the dashboard reads.

The same gates, seen in aggregate, are the system's answer to alert fatigue. Figure 4.7 shows the first five days of live operation: the scan captured 944 distinct relevant headlines across the ten names the watchlist then held, and the channel received 42 alerts, a 22:1 selectivity concentrated on the three names where the evidence cleared every gate. News flowed for every company; alerts happened only where a strong precedent *and* a material triage score coincided. (One honest note: the per-ticker daily cap entered production on 11 July, so the two earliest days show more alerts per name than the steady state.)

An alert that contradicted itself. Composing an explanation from several independent components creates a failure mode that no individual component can detect: each line can be correct while the message as a whole is incoherent. Reading the first thirty alerts the channel had actually sent found this in nine of them. A peer-comparison line reported that other names in the sector had moved the same way, and two lines below it the two-factor decomposition reported that most of the move belonged to the company. The clearest instance had AMD down 13.23% while its sector peers were down 2.0%, 1.9% and 1.4%. Both lines were right. The peer comparison tested only the *direction* of the neighbouring moves and never their *size*, so a shared direction was being reported as a shared explanation. The comparison now takes the median peer move into account and, when the subject moved disproportionately, says exactly that: the direction is shared, the size is not. The defect is

Table 4.5: The life of one real alert (META, 12 July 2026), stage by stage. Every value is real: the message is preserved verbatim in the shared alert history, and the triage score is decomposed term by term in Table 3.5.

Stage (gate)	What happened for this alert
1. Fetch	The scheduled scan pulls the week's company news for each watchlist ticker from the free news API.
2. Relevance filter	The headline names the company ("Meta", "Mark Zuckerberg" are in the ticker's alias list) and matches no market-roundup boilerplate pattern: kept.
3. Capture (living KB)	The headline is embedded (MiniLM, 384-d) and stored as a *pending* case; it matures into a precedent once its own +5-day impact becomes observable.
4. Freshness	Dated within the two-day window: eligible (the same story cannot re-alert for days).
5. Retrieval	Knowledge bases are ranked by cosine with age decay; top three: MSFT 2023-11-07 ($s = 0.46$, +2.70% in 5 d), NVDA 2023-08-02 ($s = 0.51$, −3.87%), NVDA 2023-09-21 ($s = 0.46$, +5.05%).
6. Evidence floor	Best cosine $0.51 \geq 0.45$: strong enough to show. The precedents disagree in direction, so the message carries the explicit *"moved in BOTH directions"* warning.
7. Learned triage	The deployed context-only model scores $p = 0.54 \geq 0.5$ (decomposed in Table 3.5): not gated out.
8. Caps and dedup	Under the two-alerts-per-ticker-per-day cap; not previously sent by any producer (shared-history dedup): kept.
9. Delivery and record	Sent to the public Telegram channel with the fixed no-prediction clause; appended verbatim to the shared history, from which the dashboard marks it on the price chart.

recorded here because it is invisible to component tests by construction, and because an alert that contradicts itself two lines later destroys precisely the trust the rest of the system is built to earn.

4.9 Practical Use and Responsible Design

What does it take to run, and what does responsible use require?

InvestiGator runs as two entry points, one per trigger, each performing a complete cycle. The reference deployment runs them continuously on a hosted platform: a worker repeats the scan on a sixty-second cycle and delivers any alerts to a Telegram channel, while a second process serves the dashboard. An earlier arrangement used a free continuous-integration timer instead, and the move away from it is examined in Chapter 6, where measuring the delivery latency showed that the scheduler was never the binding constraint. Running the system needs only free data resources: a market-data source, a company-news or RSS source (an API key kept out of version control), and the knowledge base built once from the corpus. Appendix A gives the exact steps. Real alerts delivered during testing and by the scheduled scan show the path from raw data to the investor's phone works end to end. Once live, the triage decisions are also *post-validated*: each logged decision is labelled days

later with the outcome that actually followed, giving live precision and calibration monitoring and fresh training data (Section 3.3.4).

Two practical limits are worth stating plainly. The knowledge base the deployed system queries is far smaller than the multi-year corpus that was built for the evaluation: the full base does not fit the free hosting tier, so production runs on a curated subset together with the cases the system captures and matures itself. This is a deployment limit rather than a method one, and it is the reason the live precedents are shallower than the evaluated ones. And InvestiGator is a decision-support prototype, not a production service: no user accounts, no persistence beyond the knowledge-base file. These are ordinary steps from prototype to product.

Shipping a transformer under free-tier constraints

One deployment problem deserves its own account, because it is where the free-tier constraint bites hardest and where the solution is a genuine engineering result rather than a workaround.

The retrieval component evaluated in Chapter 5 is a sentence transformer, and the usual way to run one is its reference implementation on top of a deep-learning framework. That stack is several hundred megabytes and assumes an environment that free hosting does not reliably provide. The naive resolutions are both bad: keep the heavy stack and accept that the public deployment cannot run, or substitute a lighter but different model in production and quietly evaluate one thing while shipping another. The second is worse than the first, because it makes the evaluation describe a system that no user ever touches.

The resolution taken was to export the *same* model to ONNX with int8 quantisation, roughly 23 MB, running on CPU with no deep-learning framework present, and then to verify the substitution is faithful before trusting it. Two checks were run against the library implementation, over 503 queries against the curated knowledge base. Embedding agreement gave a mean cosine similarity of 0.992 with a minimum of 0.983, the residual being quantisation noise rather than a different semantic space. Retrieval agreement matters more, because the product returns neighbours rather than vectors, and it must be measured in the configuration the product actually runs: the knowledge base stores embeddings computed once by the library implementation, and only the incoming query passes through the exported model. On that footing the two engines return an identical top-3 for 76% of queries and share 95% of retrieved neighbours overall. Where they disagree, they disagree by a hair: the median similarity gap between the neighbour one engine chose and the one the other chose in its place is 0.006, which is a tie rather than a different theme.

Two methodological points are worth recording, because each was a mistake caught by measuring. The first is that an earlier version of this check re-embedded the whole knowledge base with each engine. That doubles the quantisation error and answers a question deployment never asks, and it depressed the agreement figure accordingly. The second is that the same check was originally run on 23 queries and reported 20 of them as identical. That statistic does not reproduce: a different sample of the same size gives 12. The instability was in the sample size, not the engine, which is why the figures above are computed over an order of magnitude more queries and why the robust quantity to quote is the share of shared neighbours rather than the count of exactly identical lists.

Two properties follow, and both matter beyond convenience. First, the evaluated model and the deployed model are the same model, so the numbers in Chapter 5 describe what a

user actually receives. Second, the model artefact is fetched on demand and verified against a pinned SHA256 digest before use, which makes the dependency auditable: a substituted or corrupted download fails closed rather than silently changing what the system means by "similar". For a system whose central claim is that its reasoning can be checked, the provenance of the weights is part of that claim.

A smaller model still has to fit the machine it runs on, and the way that lesson arrived is worth recording because it could not have arrived any other way. Moved to a container with 512 MB of memory, the worker entered a crash loop at roughly 1.4 GB. Two plausible explanations were wrong: neither the thread pool of the inference runtime nor the size of the data branch accounted for it. A probe running inside the container itself showed startup consuming only 281 MB, which located the peak in the scan rather than in the load, and the cause was then obvious in hindsight: the runner embedded every new headline in a single batch, and on a machine with no prior state *every* headline is new. On the author's own machine the pending file already existed, so the peak never occurred and no amount of local testing would have produced it. Embedding now runs in fixed batches.

The fix invited an assumption worth reporting because measuring it refuted it. The natural sentence to write is that batching cannot change results, since the same texts are encoded either way. That is false for this model: with int8 quantisation the padding within a batch perturbs the arithmetic, and measured batch-boundary differences reached 0.022 in cosine similarity. The property that actually matters was then tested directly rather than assumed, and it holds: the top-3 retrieved neighbours were identical in 8 of 8 probes. The distinction is the same one the ONNX validation above rests on. What the product owes its user is the same *neighbours*, not bit-identical vectors, and only one of those two claims survives measurement.

Live operation also taught one engineering lesson the honest way. During the first days of continuous deployment the market path went silent: the free price source blocks the shared network addresses of hosted runners, and with a single source and no fallback, the scan simply received no data. The failure was diagnosed from the system's own records (news alerts kept flowing while not one market summary appeared, which isolated the price feed as the cause) and answered with engineering rather than with a parameter change: daily prices now come through a chain of free providers tried in order, and the real-time quote check, which uses an authenticated and therefore reliable source, was promoted from the optional watcher into every scheduled scan. The episode is reported because it is instructive: a single free data source is a single point of failure, and a system that logs its own behaviour is what makes such a failure diagnosable at all.

A tool aimed at non-experts also raises two responsible-design questions. The first is *alert fatigue*: a tool that fires too often trains the user to ignore it. The market trigger addresses it by firing only on volatility-relative extremes. The news side is addressed by the triage gate, and what that gate can honestly be said to do requires care, because measurement narrowed the claim after deployment.

It was introduced to *rank* the stream and drop the least material items, and on held-out data from the training corpus it does exactly that (Chapter 5). On the population it meets in deployment it does not: its ranking is indistinguishable from chance (Section 6.5). What survives measurement is narrower and still real. The gate suppresses roughly three fifths of news decisions, so it is an effective *volume* control, and the volume it removes is bounded by a rule the reader can inspect rather than by a quota; and every item it passes carries the

exact additive reason it passed, which a simpler cut on arrival order or a fixed daily count could not provide. The claim that the items it keeps are the material ones is withdrawn for the deployed setting and retained only where it was measured.

The component is kept rather than removed for a reason worth stating, since removing it would also be defensible. The gate is the instrumented part of the pipeline: every decision it takes is logged and later labelled by outcome, which is what made the negative finding above observable at all. Switching it off would end the measurement that revealed the problem and would remove the only channel through which a change in the upstream filters, or in the news mix, could be detected as a change in the score's usefulness. A component that is not earning its keep but is telling you so is more valuable than silence. The evaluation of Chapter 5 accordingly reports a precision-within-daily-budget metric, and Chapter 6 reports what the same mechanism achieves live, which is not the same number. The second is *over-reliance*: a confident explanation can be trusted more than its evidence warrants (Section 2.4). The present design mitigates this by showing the individual precedents and their spread rather than a lone number, and by disclaiming prediction on every alert. Two further mitigations were added after reading the alerts the channel had actually sent, and both were prompted by defects rather than foresight. Precedents are now de-duplicated on headline text: the previous key combined date, ticker and headline, which looks like the more thorough choice and is exactly why it failed, because one macroeconomic event retrieved under three different tickers produced three different keys and was displayed as three independent cases. In the live history this affected 18 of the 165 news alerts carrying two or more precedents, or 11%, and the alert that exposed it read "3 of 3 shown cases moved down" when it was one event seen three times. That is not a presentation blemish but a false statement about the weight of the evidence, and the reader has no way to tell the two apart from the text. A retrieved cluster that disagrees in direction is now flagged explicitly as well, which matters because thematic similarity carries almost no directional information (Chapter 5).

4.10 Chapter Conclusions

InvestiGator is a two-trigger, two-layer pipeline built from a few clear objects (a *case*, the knowledge base, an anomaly result) and single-responsibility components, with explainability built in by rendering every alert directly from the system's own computation, and an optional trained triage score, off by default and honestly labelled, layered on top. The next chapter puts the core components to the test on real data.

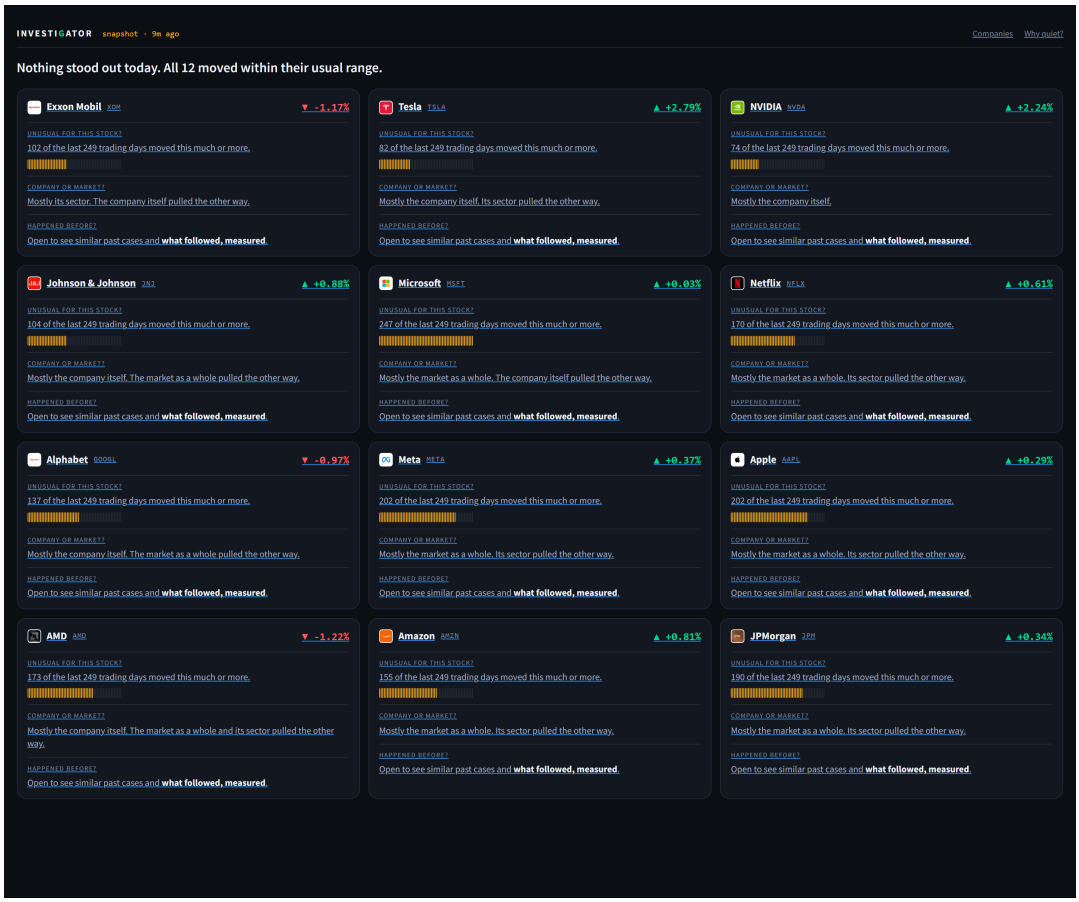

Figure 4.6: The public dashboard (a genuine screenshot, not a mockup), opening surface, captured from the deployed codebase during live operation. One card per company, ordered by how rare the day was. Every card answers the same three questions from Section 1.2 under the same headings and in the same order, including on a day when the answer to all of them is that nothing much happened: the headline sentence states exactly that, and each card evidences its own quietness with a count rather than asserting it. Rarity is given twice, as a sentence anyone can check ("247 of the last 249 trading days moved this much or more" for Microsoft, at +0.03%) and as a strip of marks, so a reader can compare companies without reading a number. The attribution line names the driver in words and, where a component pulled against the move, says so rather than hiding it: AMD fell 1.22% "mostly the company itself", with both the market and its sector pulling the other way.

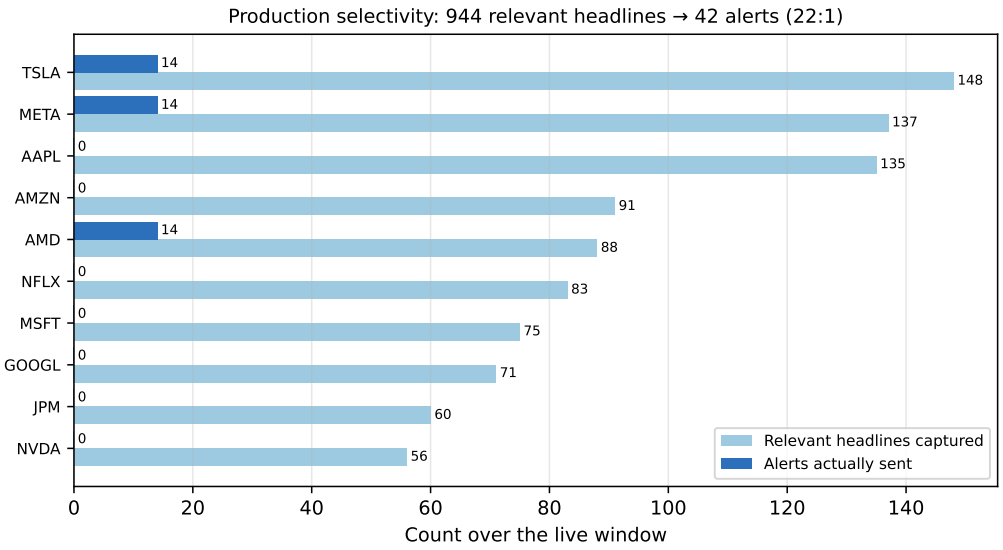

Figure 4.7: Production selectivity over the first five live days: relevant headlines captured (light) versus alerts actually sent (dark), per watchlist ticker. All counts are real, read from the shared data branch; the gap between the two bars is the work of the quality gates of Table 4.5.

Chapter 5

Case Studies

This chapter evaluates InvestiGator through eight case studies on real data, in two groups. The first four test the core design: the anomaly trigger across stocks of very different volatility; the correlation engine and the precedents it retrieves; a complete alert, and whether its explanation is faithful; and the materiality-triage model trained and tested on the multi-year corpus. The final four examine the same system from the outside, asking what it does *not* know: whether the embedding space encodes what kind of event a headline reports, what a distribution-free guarantee on the triage score would cost, how far the deployed present has drifted from the training corpus, and whether several signals agreeing beat the best one alone. Each of those four ends in a decision *not* to change production, taken on the evidence. Every number below comes from a versioned script with a fixed seed, the evaluation followed the plan set out in Chapter 3, and each study states its own limitations.

5.1 Common Experimental Setup

What do the four studies share? Three real datasets are used. For the anomaly trigger, daily closing prices for the fifteen large-capitalisation tickers listed in the data card were retrieved from a free market-data service over a fixed three-year window (June 2023 to June 2026), yielding 750 daily log-returns per ticker. For the correlation engine, 3,714 real financial-news headlines for the same tickers were collected through a free company-news service (the most recent months available on the free tier), spanning five sectors (technology, banking, energy, health and consumer staples). This is the retrieval evaluation corpus, distinct from the designed FNSPID layer documented in the data card of Chapter 3. For the materiality-triage study (Case Study 4), the multi-year FNSPID subset of the data card *was* built, by the streaming pipeline, and aligned to prices: 79,753 (headline, ticker, event-day) examples over 2018–2023. News headlines and prices are normalised to a common schema so that the live and historical layers are directly comparable. The recent corpus is enough for the retrieval experiment; a multi-year impact analysis over the retrieval knowledge base remains future work. The no-lookahead, reproducibility and honesty guarantees of Section 3.6 apply to every study below.

5.2 Case Study 1: Transparent Anomaly Detection on US Equities

Is a rolling z*-score a better universal detector of abrupt moves than a simple fixed-percentage threshold?* **Yes, and clearly so**: the z-score (Chandola, Banerjee, and Kumar 2009) fires

at a near-constant rate across stocks of very different volatility, where the fixed rule does not.

The setup is as follows. A "true" anomaly is proxied by a daily move in the upper percentile of *that ticker*'s own returns ($|r| \geq$ the 99th percentile); the baseline is one global threshold ($|r| \geq 3\%$) applied to every ticker. Because that label is itself volatility-relative, it is only supporting evidence; the primary argument, made first below, needs no label at all.

Firing-rate consistency (main argument). The strongest evidence is independent of any label. A good universal detector should fire at a comparable rate across instruments; a volatility-blind threshold cannot. Table 5.1 and Figure 5.1 show that the fixed threshold fires on roughly 1% of days for a calm stock (KO) but on about 35% of days for a volatile one (TSLA, NVDA), whereas the z-score fires at a near-constant rate ($\approx$ 2–3%) for all tickers. The range of firing rates across tickers is more than twenty times tighter for the z-score (0.015 versus 0.344), confirming that normalising by recent volatility makes detection comparable across the market.

Table 5.1: Firing rate across the fifteen tickers (three years of daily data).

Method	Min rate	Max rate	Range
z-score (window 20, ± 3)	0.016	0.031	**0.015**
Fixed threshold ($\lvert r\rvert \geq 3\%$)	0.009	0.353	**0.344**

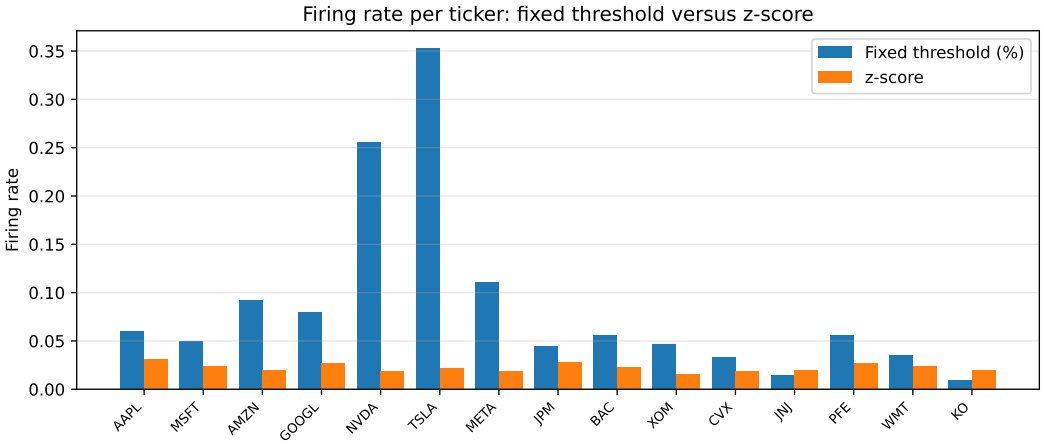

Figure 5.1: Firing rate per ticker. The fixed percentage threshold varies dramatically with each stock's volatility, while the z-score is stable across all tickers.

A worked example. Figure 5.2 shows the detector at work on a single volatile stock (Tesla) over the three-year window. The flagged days are not simply the largest raw moves: they are the moves that are large *relative to the asset's own recent volatility*, which is exactly the behaviour a fixed percentage threshold cannot reproduce. Over this period the detector flags 16 of the 750 trading days (about 2%), consistent with the near-constant firing rate reported above.

To make this concrete, consider the single largest surprise in the window. On 24 October 2024, following its quarterly earnings, Tesla's daily log-return was +0.198, a price move

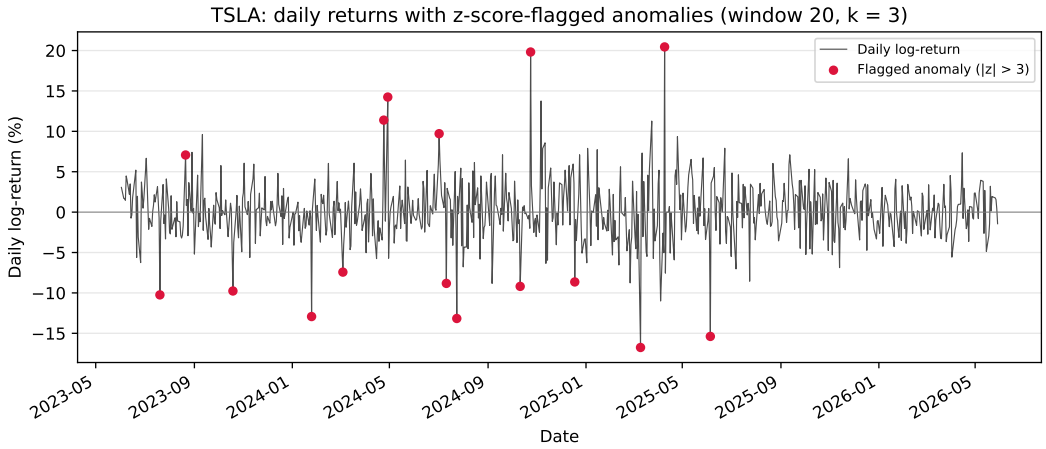

Figure 5.2: Daily log-returns of a volatile stock (Tesla) over the evaluation window, with the days flagged by the z-score detector (window 20, $k = 3$) marked. The detector responds to volatility-relative surprise rather than to absolute move size.

of about +22%. Measured against the preceding twenty days, whose mean log-return was −0.92% and standard deviation 2.73%, this is a z-score of +7.6, far beyond the threshold $k = 3$ (Table 5.2). The same rule that leaves an ordinary two-percent wobble unflagged identifies this earnings reaction at once, and exposes the quantities that justify the alert: the move, the recent norm it is judged against, and the resulting z-score, which is what the explanation engine then shows the investor.

Table 5.2: Worked anomaly: Tesla's post-earnings move on 24 October 2024 (window 20, $k = 3$); real, reproducible figures.

Quantity	**Value**
Trailing-20-day mean of log-returns, μ	−0.92%
Trailing-20-day standard deviation, σ	2.73%
Day's log-return, r (a price move of $\approx$ +22%)	+19.82%
z-score, $(r - \mu)/\sigma$	**+7.61**
Flagged at $k = 3$?	yes

Agreement with the proxy label (supporting). Against the per-ticker extreme-move label, the z-score attains an F_1 of 0.516 (precision 0.381, recall 0.800) versus 0.218 for the fixed threshold (precision 0.122, recall 0.992): the fixed rule catches almost every large raw move but at very low precision. A window ablation (Figure 5.3) shows F_1 rising with the estimation window (0.385, 0.516 and 0.678 for 10, 20 and 60 days), indicating that a longer volatility estimate yields a steadier baseline before it begins to saturate. This label is itself volatility-relative, so some circularity is unavoidable; that is why the firing-rate consistency above, which depends on no label, is treated as the primary evidence.

Statistical rule versus learned detector. The protocol of Section 3.6.3 closes with a challenge: would a *learned* detector, given exactly the same information, beat the transparent rule? An Isolation Forest (F. T. Liu, Ting, and Zhou 2008) was trained causally per ticker on

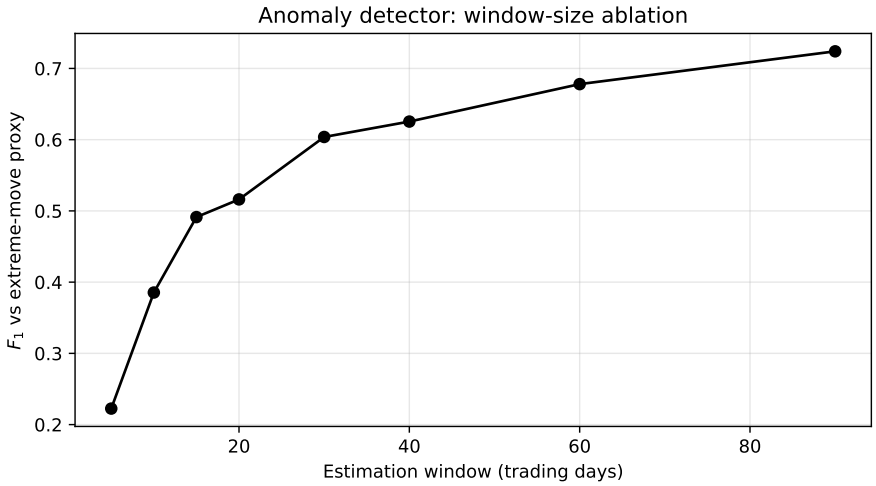

Figure 5.3: Effect of the estimation-window length on the detector's F_1 against the extreme-move proxy. A longer window yields a steadier volatility baseline and higher agreement, with diminishing returns at the longest windows.

each day's return and the volatility of the twenty preceding days, fitted on an initial 250-day segment only, and both detectors were scored on the identical remaining region. The learned model loses clearly: against the extreme-move proxy it reaches an F_1 of 0.271 (precision 0.159, recall 0.913), while the z-score on the same region reaches 0.530 (precision 0.407, recall 0.761); the forest also fires at rates that vary across tickers by 0.135, two orders of magnitude less uniform than the z-score's 0.015, failing the consistency requirement that motivates the design. The comparison, rather than any assumption, is what justifies keeping the statistical rule in production: given identical causal information, the interpretable detector was not merely simpler but better.

Extension: a second learned detector, and a second volatility estimate. Two natural objections remain: perhaps a *different* learned detector would win, and perhaps a more adaptive volatility estimate would. Both were tested under the identical causal protocol (Section 3.6.3), in a fresh rerun whose z-score row reproduces the frozen values above exactly (the forest shifts in the third decimal because the free price source re-adjusts historical closes retroactively). Table 5.3 and Figure 5.4 report both answers. The Local Outlier Factor (Breunig et al. 2000), the density-based alternative of Chapter 2, behaves like the forest: it recalls almost everything (0.989) at very low precision (0.163), lands at an F_1 of 0.280, and fires at rates that vary across tickers by 0.183, the least consistent of the three. The transparent rule beats both learned detectors on the same information.

The volatility question produced the more interesting answer, and it is reported exactly as it fell. Replacing the rolling standard deviation with an exponentially weighted estimate (RiskMetrics, $\lambda = 0.94$), the first step toward the GARCH family, *improves* agreement with the extreme-move proxy substantially: at the same recall (0.800), precision rises from 0.381 to 0.568, lifting F_1 from 0.516 to 0.664, with per-ticker firing rates slightly *more* uniform (0.012 versus 0.015). The mechanism is volatility clustering itself: after a shock, a rolling window dilutes the new regime across twenty equal weights, whereas the exponential estimate absorbs it at once and stops re-flagging ordinary days that follow. The experiment

was designed to license the phrase "GARCH would add little", and it does not license it; what it shows instead is that the first adaptive step already helps *on this proxy*. The deployed detector keeps the rolling rule, because "the average and spread of the last twenty days" is explainable to a non-expert in one sentence while exponential weights are not, but the gain is recorded as validated rather than speculative future work, one line of configuration away. The proxy's own volatility-relative circularity, assumed throughout this case study, applies here equally.

Table 5.3: Extended comparison under the frozen causal protocol. Top: detectors on the identical scored region and features. Bottom: the same z-score rule with two volatility estimates, over the full evaluation region. Regenerated deterministically.

Detector (same region)	**Precision**	**Recall**	F_1	**Rate range**
z-score (deployed rule)	0.407	0.761	**0.530**	**0.017**
Isolation Forest	0.158	0.913	0.269	0.135
Local Outlier Factor	0.163	0.989	0.280	0.183
Volatility estimate (z-score)				
Rolling σ, 20 d (deployed)	0.381	0.800	0.516	0.015
EWMA σ ($\lambda = 0.94$)	0.568	0.800	**0.664**	**0.012**

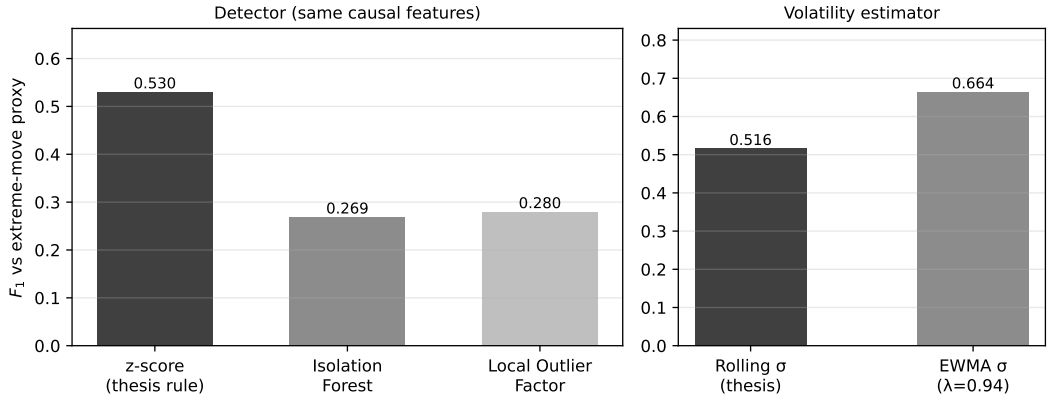

Figure 5.4: The extended comparison at a glance. Left: on identical causal features, the transparent z-score rule beats both learned detectors. Right: the exponentially weighted volatility estimate outperforms the rolling one on the proxy, a finding reported as it fell; the rolling rule stays deployed for explainability, with the EWMA gain recorded as validated future work.

5.3 Case Study 2: News–Market Precedent Retrieval

Does semantic retrieval find genuinely analogous precedents, better than trivial baselines? **Yes**: on real data the embeddings beat every baseline by a wide margin.

The metric is precision@k with a sector proxy, in a *cross-ticker* setting: for each query the k most similar headlines *from other companies* are retrieved, and precision is the share that match the query's sector. Excluding the query's own company rules out the trivial win of matching a company to its own news, and tests thematic analogy instead. Same-sector

is an automatic stand-in for "analogous", so the metric shows whether retrieved news is on-theme, not whether a person would call it useful; that gap is real, and a human study is future work. To show the result is not tied to one model, two Sentence-BERT models (Reimers and Gurevych 2019) (the lightweight MiniLM and the larger MPNet) are compared against a lexical (hashing) baseline and two trivial ones, random (the sector base rate) and recency. Five hundred queries were sampled in each of five runs (seeds 42–46); Table 5.4 reports the mean and standard deviation.

Table 5.4: Cross-ticker precision@k by sector, mean $\pm$ std over five runs (3,714 real headlines, five sectors).

Method	**P@5**	**P@10**
SBERT (MiniLM, default)	0.514 ± 0.015	0.478 ± 0.011
SBERT (MPNet)	$\mathbf{0.538 \pm 0.011}$	$\mathbf{0.506 \pm 0.010}$
Lexical baseline (hashing)	0.346 ± 0.011	0.314 ± 0.008
Recency	0.126 ± 0.006	0.106 ± 0.005
Random (base rate)	0.240 ± 0.004	0.240 ± 0.004

As Table 5.4 and Figure 5.5 show, the default MiniLM model reaches a precision@5 of 0.514, about 2.1 times the random base rate (0.240) and well above the lexical baseline (0.346). The stronger MPNet model does slightly better (0.538), which suggests the advantage belongs to semantic embeddings in general rather than to one particular model; the small standard deviations (around 0.01) show the gap is robust rather than a sampling artefact. Recency performs worst, below even the random base rate, so simply surfacing the latest news is no substitute for semantic matching. The lift over both the random and lexical baselines is the honest measure of the value added by the embeddings.

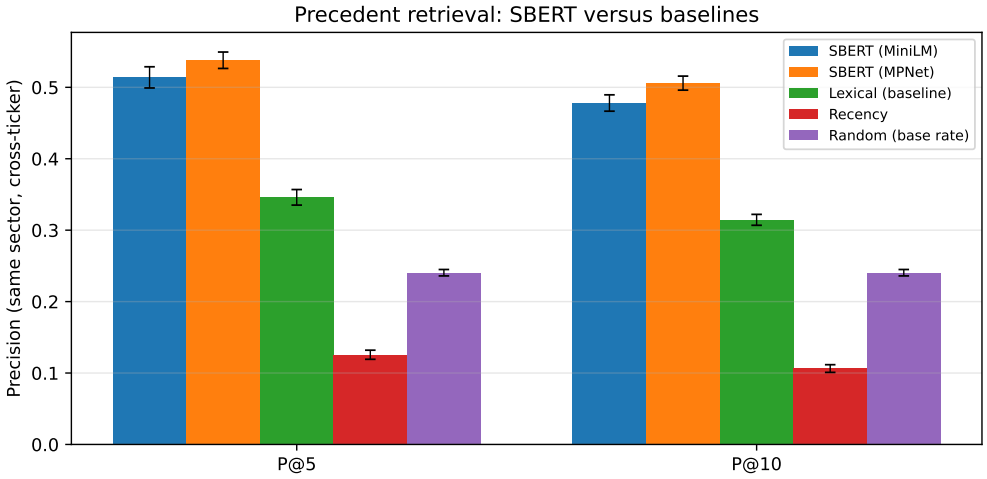

Figure 5.5: Cross-ticker precision@k by sector. SBERT clearly exceeds the lexical, recency and random baselines; error bars show ± 1 standard deviation over the five runs.

Seeing the space. The embedding space itself, so far only sketched conceptually (Figure 3.6), can be shown for real. Figure 5.6 is a principal-component projection of the production knowledge base, all 2,016 cases with their genuine 384-dimensional MiniLM embeddings, coloured by sector, with a live query (the worked example of Chapter 3) marked

as a star and its three retrieved neighbours circled. Two honest readings follow. Even this two-dimensional shadow, which preserves only 8.6% of the variance, places the query inside its topical neighbourhood, and the three retrieved cases (all semiconductor-demand stories, at cosines 0.58–0.61 in the full space) sit visibly adjacent to it. And the sectors do *not* separate cleanly in two dimensions, which is exactly why the system ranks by cosine in the full 384-dimensional space rather than in any projection: the picture is for intuition, the ranking is not done there.

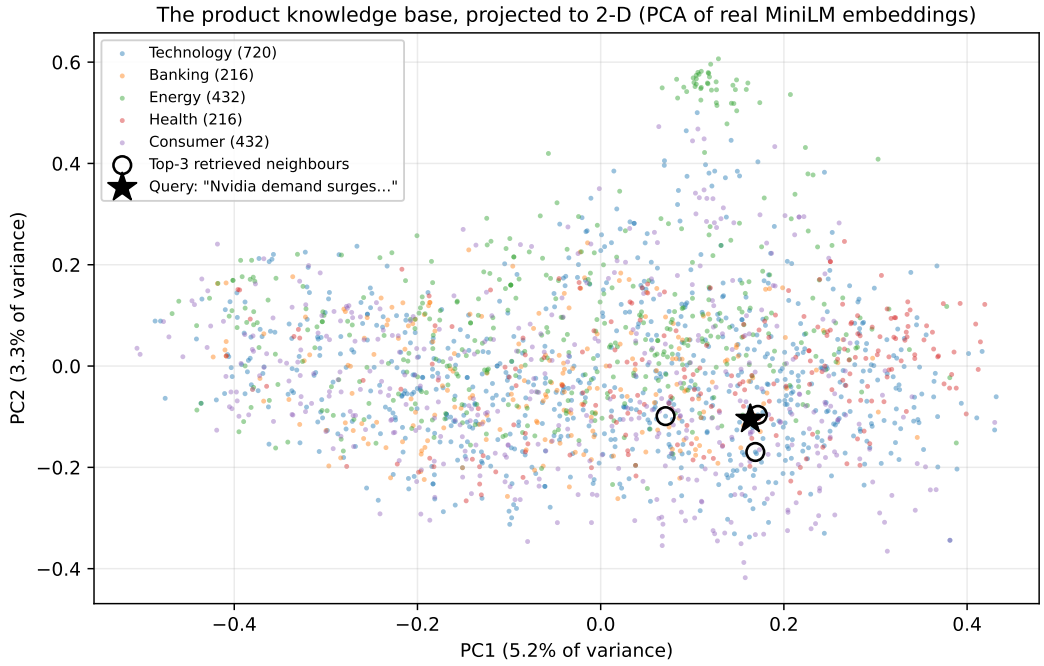

Figure 5.6: The production knowledge base projected to two dimensions (PCA over the real MiniLM embeddings; axes show the variance each component preserves). Colours are sectors; the star is the query *"Nvidia demand surges on AI chip orders"*; circles mark its top-3 retrieved neighbours. The projection is a shadow for intuition: retrieval ranks by cosine in the full 384-dimensional space. Regenerated deterministically.

Impact measurement. For each retrieved precedent the engine reports the observed post-event return at +1, +3 and +5 trading days, measured from the event-day close in the style of an event study (Brown and Warner 1985). These figures are observed outcomes presented as evidence, never a prediction. On the worked knowledge base the measured impacts are directionally coherent (for example, the competitive-threat news cluster in the end-to-end case study below is followed by uniformly negative one-day returns), which provides face validity for the measurement; a rigorous multi-year impact study on the full FNSPID corpus is identified as future work.

Retrieval by sector. The aggregate figure hides useful variation across sectors. Table 5.5 and Figure 5.7 break the retrieval down by the query's sector, using the whole population of each sector as queries (so the figures are exact rather than sampled). Two readings matter. The raw precision@5 is highest for technology (0.712), but largely because technology dominates the corpus (47% of headlines) and therefore has a high random base rate (0.429);

the fair cross-sector measure is the *lift* over that base rate. By that measure the engine adds the most value in **energy** (+0.377) and **health** (+0.348), sectors whose vocabulary is distinctive (production, output, barrels; trial, drug, approval), and the least in **consumer** (+0.100), whose headlines are more generic and thematically overlap with other sectors. Retrieval is positive in every sector, but its benefit is greatest where the domain language is most specific.

Table 5.5: Cross-ticker precision@k per sector (whole-population queries, MiniLM).

Sector	N	P@5	P@10	Random	Lift P@5
Technology	1736	0.712	0.675	0.429	+0.283
Energy	497	0.448	0.408	0.072	**+0.377**
Health	494	0.419	0.379	0.071	+0.348
Banking	496	0.272	0.228	0.072	+0.200
Consumer	491	0.171	0.150	0.071	+0.100

Note: the lift is precision@5 minus the random base rate, computed from unrounded values; it may differ from the rounded columns by ±0.001.

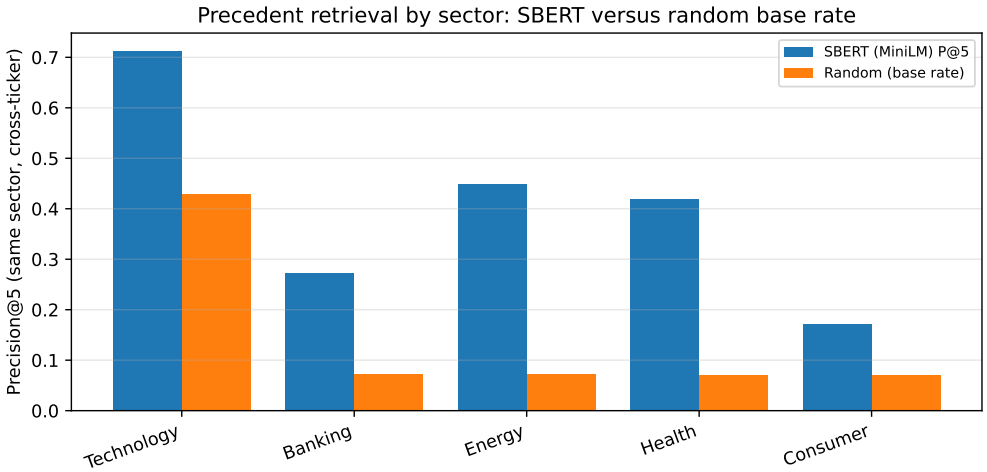

Figure 5.7: Precision@5 per sector against the random base rate. Retrieval exceeds the base rate in every sector; the gap (lift) is largest for energy and health, whose vocabulary is most distinctive, and smallest for the broad consumer sector.

Why some sectors retrieve better: a qualitative look. The per-sector gap has a concrete explanation, visible in individual queries. In technology, different companies genuinely share themes: a headline about NVIDIA's data-centre accelerators retrieves, from the Microsoft, Meta and Alphabet feeds, articles about the same wave of AI-chip competition (Case Study 3), so cross-ticker precision is high. The consumer sector behaves differently. Looking at the raw neighbourhood (before the cross-ticker constraint is applied), a query about Walmart's grocery guidance retrieves mostly Walmart's own items plus a macro "US retail sales" story carried on bank feeds: adjacent in theme, but not a consumer analogue from another consumer company. Once the query's own ticker is excluded, as the metric requires, little genuinely relevant material is left, which is why the cross-ticker lift for consumer is low.

A query about Coca-Cola retrieves almost only other Coca-Cola items, about its pricing power and beverage volumes, which simply do not resemble Walmart's store-traffic news. The shared label "consumer" is too broad: its members' news is idiosyncratic, so cross-ticker analogues are scarce and the lift over the base rate is small. This is the pattern the numbers show, and it points to a clear refinement for future work: grouping by finer-grained themes rather than by sector.

5.4 Case Study 3: End-to-End Alert and Explanation Faithfulness

Does a complete alert hold together, and is its explanation faithful? **Yes, faithful by construction.** To see the news trigger end to end, take a new NVIDIA headline, *"Nvidia raises guidance on surging demand for AI data-centre accelerators"*. The system embeds it, retrieves the most similar precedents from the knowledge base built on the real news corpus, and renders the alert below (top five precedents, one-day horizon):

```
News alert for NVDA
"Nvidia raises guidance on surging demand for AI data-centre
accelerators"
Potential impact (from 5 similar past events):  average 1-day move:
−1.97%
Historical precedents:
- 2026-06-24 MSFT (sim 0.68) +1d −3.46%:  "Qualcomm debuts line of AI
data center chips..."
- 2026-06-24 NVDA (sim 0.68) +1d −1.64%:  "Qualcomm debuts line of AI
data center chips..."
- 2026-06-24 META (sim 0.68) +1d −2.65%:  "Qualcomm debuts line of AI
data center chips..."
- 2026-06-24 GOOGL (sim 0.64) +1d −0.46%:  "This AI Infrastructure
Stock Could Be Bigger Than Nvidia..."
- 2026-06-24 NVDA (sim 0.58) +1d −1.64%:  "Tecan Accelerates
Data-Driven Lab Journey...  NVIDIA"
Note:  precedents are retrieved by semantic similarity; the impact is
the observed past outcome, not a price prediction.
```

Every retrieved precedent concerns AI data-centre chips and competition for NVIDIA, even though the articles were filed under different company feeds (Microsoft, Meta, Alphabet and NVIDIA itself). Unlike the retrieval metric of Section 5.3, the live alert does not exclude the query's own ticker: that exclusion exists only to keep the evaluation non-trivial, whereas a real user is well served by the most similar precedents, whichever company they concern. Strikingly, the same story (a rival entering the AI data-centre chip market) surfaces across three different company feeds, which the engine correctly groups as one theme. This is direct evidence that retrieval is driven by *meaning* rather than the company name or exact keywords, the very behaviour the correlation engine is designed for. In this instance the precedents happen to be uniformly negative (a competitive-threat cluster), and their average is reported as an observed past outcome, not a prediction. The alert ends with an explicit disclaimer, keeping the system within its no-forecasting scope.

What the retrieval does and does not capture. This example also exposes an honest limitation. The query headline reads as positive (guidance raised), yet the precedents it

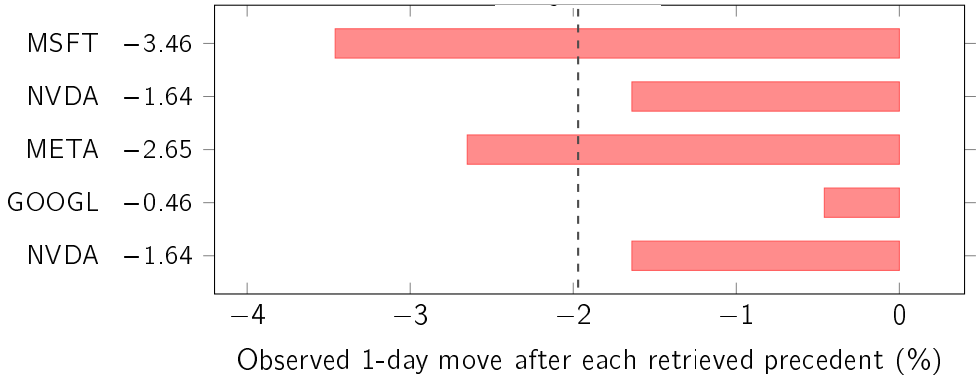

Figure 5.8: The theme-versus-direction lesson on one real alert. The query headline *"Nvidia raises guidance on surging demand for AI data-centre accelerators"* reads clearly positive, yet the five most similar past headlines all belong to an AI-chip competitive-threat theme and every one fell the next day (average −1.97%, dashed). Retrieval matches *meaning*, not sentiment or direction, so the reported average is evidence about how a theme has moved, never a forecast for this item. Values from the worked alert above.

retrieves form a competitive-threat cluster whose one-day outcomes are uniformly negative (Figure 5.8). There is no contradiction once the mechanism is clear: sentence-embedding similarity matches on *theme*, here "AI data-centre chips", and not on sentiment or expected direction. The mean impact the alert reports is therefore evidence about how a theme has tended to move, not a directional forecast for this particular item, which is exactly why the alert always lists the individual precedents and closes with a no-prediction disclaimer rather than leaning on a single averaged number. Treating that average as a prediction would be the over-reliance that the trust literature warns against (Bansal et al. 2021; J. D. Lee and See 2004), and the design guards against it by showing the evidence rather than issuing a verdict.

Two cosmetic artefacts of the shallow recent corpus are also visible and worth naming plainly: the precedents share a single retrieval date, because the free news tier returns only a recent window and there is little genuine history to draw on, and the same ticker can appear twice with an identical impact, because same-ticker precedents aligned to the same event day necessarily share their forward return. Both follow from corpus depth, not from the method, and both disappear once the full multi-year knowledge base is built. For the same reason, the positive mean impact obtained for a similar Nvidia query on the bundled sample base in Section 3.3.2 (+6.5% at five days) and the negative mean here (−1.97% at one day) are not in tension: they use different knowledge bases, horizons and encoders, and neither is a forecast.

Explanation faithfulness. Faithfulness is guaranteed by construction: the alert text is rendered directly from the same objects the system computes (the z-score, window and threshold for the anomaly path, and the exact retrieved precedents with their similarity scores and measured impacts for the news path), so the explanation cannot diverge from the underlying computation. This is checked programmatically by an automated test that asserts the rendered alert reproduces exactly each retrieved precedent's date, ticker and similarity score and introduces none that were not retrieved; it is the kind of transparent, design-time explainability advocated in the XAI literature (Barredo Arrieta et al. 2020). Usefulness to

a retail investor is inherently subjective; a small rubric-based human assessment (clarity, completeness, actionability) is planned but has not yet been conducted, and is therefore reported as a limitation rather than a result.

5.5 Case Study 4: Learned Materiality Triage

Question: can the trained triage model of Section 3.3.4 prioritise news better than simple volatility evidence (RQ4)? Answer: every trained ranker clears the always-alert floor by a wide margin, and within a realistic alert budget triage nearly quadruples precision; but none of the models that read the headline text beats the volatility-only baseline. On this evidence, the triage signal lives in market context, not in headline wording, and the dissertation reports that finding as it stands.

Corpus and setup. The FNSPID subset was streamed and aligned to prices exactly as designed: 79,753 examples over 1,501 unique trading days (January 2018 to December 2023), with zero discarded rows. Fourteen of the fifteen tickers are represented; Meta is absent because the corpus indexes it under its former symbol (FB) for most of the window, a coverage note rather than a method change. The temporal split by unique days (70/15/15, five-day embargo) yields 28,574 training, 17,710 validation and 32,649 test rows, with material examples at 38.5%, 47.0% and 37.8% respectively. Two corpus properties matter for reading the results. News density grows over time (2023 alone holds 44% of the examples), so the later blocks hold more rows than their share of days. And headlines from the same (ticker, day) pair share one label, so rows come in clusters; the split by unique days keeps each cluster inside a single block.

Results. Table 5.6 reports the test block; Figure 5.9 shows the precision–recall curves and Figure 5.10 the calibration of the main interpretable model.

Table 5.6: Materiality triage on the FNSPID corpus (test block; prevalence 0.378). PR-AUC is the primary metric; precision@budget admits at most five alerts per day.

Model	PR-AUC	ROC-AUC	Brier	Prec.@budget
Always alert (floor)	0.378	0.500	0.622	0.163
Volatility-only LR (baseline)	**0.542**	**0.665**	**0.218**	**0.632**
Context-only LR	0.538	0.658	0.224	**0.632**
Text-only LR	0.439	0.564	0.240	0.572
Context+text LR (main)	0.496	0.622	0.229	0.585
Gradient boosting (context+text)	0.469	0.624	0.228	0.551

Reading the result. Three observations, in decreasing order of practical weight. First, *triage works as a product mechanism*: within a budget of five alerts per day, ranking by any trained score raises the share of material alerts from 0.163 (picking blindly) to 0.632, nearly four times better, with calibrated probabilities (Brier 0.218 against 0.622 for the floor). This is the alert-fatigue payoff the component was built for. Second, *the signal is market context, not text*: the volatility-only baseline attains the best PR-AUC (0.542), the context-only model matches it in budgeted precision, and adding the headline embedding

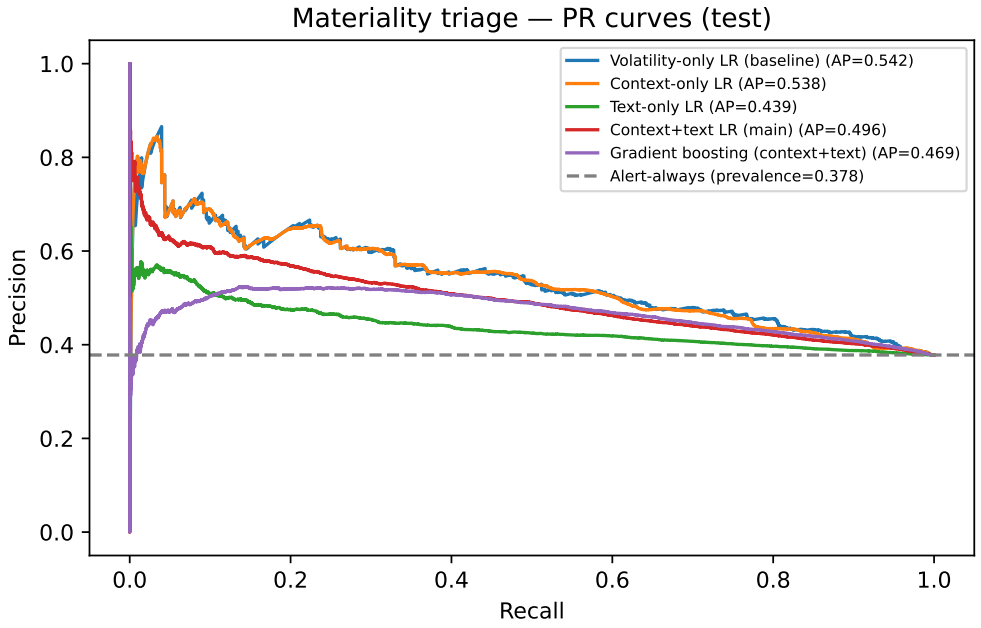

Figure 5.9: Precision–recall curves on the test block. All trained models sit well above the always-alert floor (dashed line at the prevalence); the volatility-only baseline is not overtaken by any model that uses the headline text.

lowers the linear model to 0.496; the gradient-boosted ensemble does not recover the gap (0.469). Headlines alone are far above the floor (0.439) but well below volatility. Third, the answer to RQ4 as posed is therefore *negative on the text hypothesis and positive on the mechanism*: a supervised, calibrated ranker adds real product value over naive alerting, but on this corpus and with this text representation it does not beat the simple volatility baseline it was required to beat. This is the second time in this chapter that a learned method was given a fair, causal comparison against the transparent choice and lost (Case Study 1); both comparisons were pre-committed in Section 3.6, and both validate the dissertation's simplicity-first design with evidence rather than assumption. The deployed variant (context-only, Section 4.6) is consistent with this finding: it uses exactly the features that carry the signal.

Caveats. The label is a proxy: market-adjusted move size, not human judgement of materiality. Same-day headlines share one label, so the effective sample size is smaller than the row count, and the corpus's news density grows over the window. Headlines are short, so a richer text representation (article bodies, financial sentiment) might yet close the text gap; that possibility is recorded as future work rather than assumed.

The deployed model on one real decision. The mechanism claim becomes tangible on a single live case. Figure 5.11 decomposes a real production decision, the Meta alert of 12 July 2026 whose full arithmetic is Table 3.5: elevated recent volatility and the sector indicator carry essentially the whole log-odds, the near-zero remaining terms are what the aggregate finding predicts a single case should look like, and the calibrated 54% that reached the public channel is reproduced exactly by the decomposition. The evaluation's aggregate

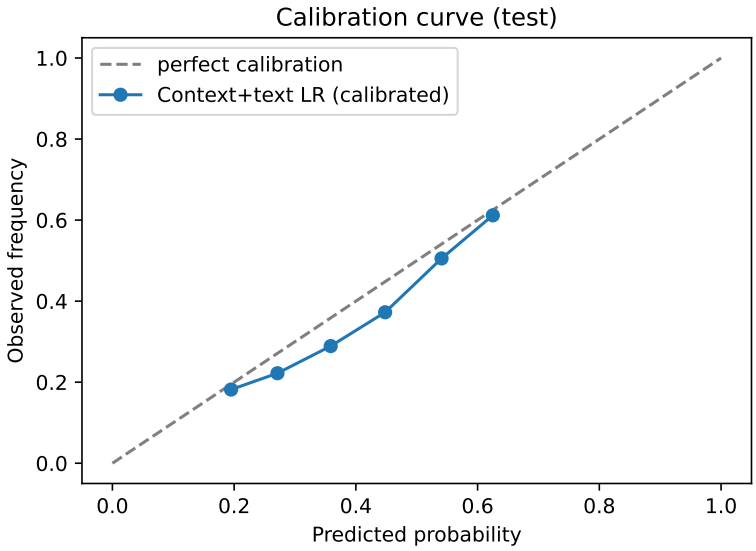

Figure 5.10: Calibration of the main interpretable model after Platt scaling on the validation block: predicted probabilities track observed frequencies, which is what makes the score honest to show to an end user.

finding (context, not text) and the deployed product's individual decisions are one and the same computation, which is the faithfulness property the design promises.

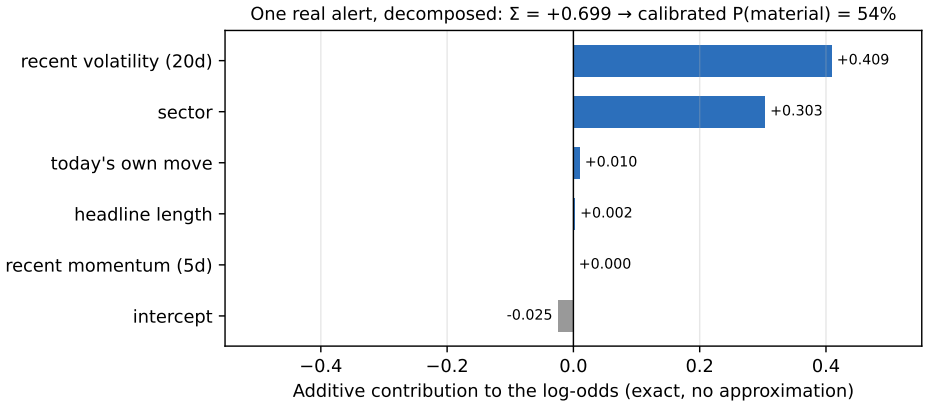

Figure 5.11: One real production triage decision (META, 12 July 2026), decomposed into its exact additive log-odds contributions. Recent volatility and sector dominate; the sum +0.699 maps through the sigmoid and the Platt calibration to the 54% actually sent to the channel (Table 3.5). Regenerated deterministically.

Do more market features help? A feature ablation. The result above says the signal lives in market context rather than headline text, which invites the natural follow-up: would *more* context help? Five additional signals were engineered, all cheap (no new data source) and all under the same causal convention as the originals, so that nothing after the event day can leak into them: the broad market's own volatility regime (on the market index), the stock's twenty-day momentum, the ratio of short to long volatility, the standardised immediate reaction (the detector's own z), and a downside-only volatility. The context-only

model was retrained on these under the frozen protocol (temporal split, Platt calibration on validation, seed 42). The volatility anchor reproduces the frozen number (0.537 against the frozen 0.538, the tiny gap coming only from the stricter sixty-day history the extended features require). Adding all five signals moves PR-AUC to 0.535, no improvement at all. A leave-one-in / leave-one-out sweep (Figure 5.12) explains why: the standardised reaction is the only signal with a positive sign, and only by 0.001; the others are flat or slightly negative. Rolling volatility already absorbs almost everything these cheap features carry, which is the same lesson as the text result, reached from the opposite direction: on this corpus, near-term materiality is remarkably well summarised by one number. The finding is reported exactly as it fell and changes nothing in production; the full extended-feature table is reproduced deterministically from the frozen inputs.

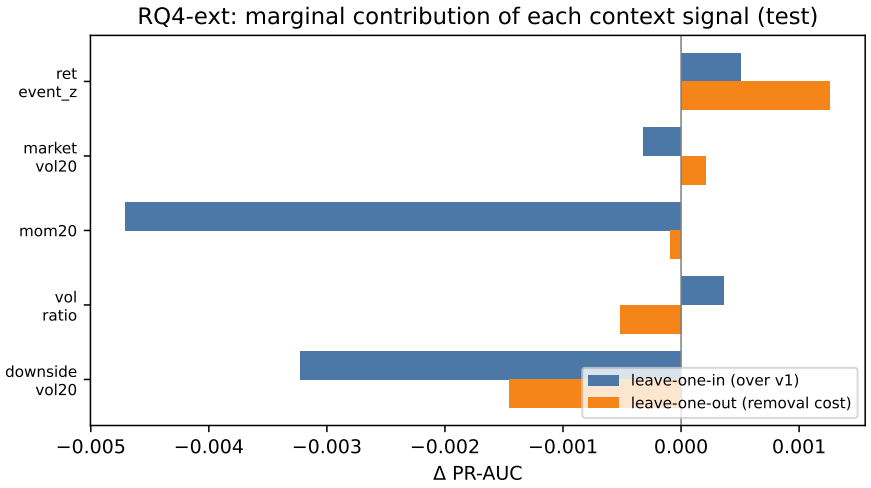

Figure 5.12: Marginal PR-AUC contribution of each extended context feature on the test block, measured two ways: added on its own to the volatility model (leave-one-in) and removed from the full set (leave-one-out). All effects are within ±0.005 of zero: the cheap engineered signals are redundant given rolling volatility. Regenerated deterministically.

5.6 Case Study 5: Event-Type Structure in the Embedding Space

Question: does the sentence-embedding space that drives retrieval carry any notion of what kind of event *a headline reports, or only of what the headline is about? Answer: it carries both, and event type is the stronger of the two once the comparison is made fairly; but the separation is weak in absolute terms, and that is why the resulting taxonomy is kept descriptive rather than wired into retrieval.*

Why the question matters. Case Study 3 showed a positive headline retrieving a cluster of precedents whose measured outcomes were negative, and attributed it to theme rather than direction. That diagnosis leaves something unexplained: if the space encoded the *kind* of event, precedents could be restricted to the same kind, and the mismatch would narrow. The knowledge base, however, is a flat collection of vectors with no notion of event type at all. This case study asks whether one can be recovered without supervision.

Setup. The cached MiniLM embeddings of the 78,933 corpus headlines were clustered with k-means over L^2-normalised vectors, sweeping k from 6 to 20. Because there is no ground truth for event type, a reference was built separately: a keyword rubric covering eight types (earnings, guidance, analyst action, product, legal or regulatory, mergers and acquisitions, personnel, and macro or market). The rubric is deliberately high-precision and low-coverage. It abstains when no pattern fires and also when *more than one* fires, because a headline that is simultaneously an earnings report and an analyst action is genuinely ambiguous, and a reference that resolves ambiguity by list order would be inventing precision it does not have. It labels 11,889 headlines, 15.1% of the corpus, and all agreement figures are computed on that subset with the count reported alongside.

Two decisions guard the measurement. The rubric was written and committed *before* any clustering was run, so it could not be tuned until it agreed with the clusters. And k was chosen by silhouette rather than by agreement with the rubric, since selecting k to maximise the evaluation metric would tune the unsupervised method against its own assessment.

Results. Table 5.7 reports the outcome at $k^* = 18$; Figure 5.13 shows the silhouette sweep and the agreement comparison.

Table 5.7: Unsupervised event-type structure at $k^* = 18$. Purity and agreement are measured on the 11,889 headlines the reference rubric covers.

Quantity	Value	Comparison
Silhouette (cosine)	+0.084	weak separation in absolute terms
Purity against the rubric	0.712	random, same cluster sizes: 0.444
Agreement with event type (AMI)	**0.358**	the reference of interest
Agreement with ticker (AMI)	0.188	subject, not event
Agreement with sector (AMI)	0.130	subject, not event
Stability across seeds (ARI)	0.786	mean over three reseeded runs

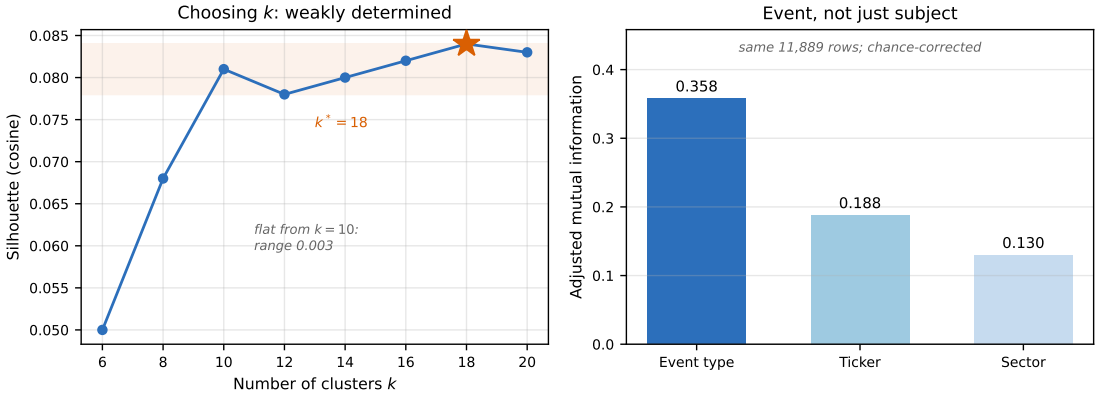

Figure 5.13: Left: the silhouette sweep. The curve is flat from $k = 10$ onwards (a range of 0.003), so $k^* = 18$ is only weakly preferred over its neighbours, and is reported as such. Right: chance-corrected agreement between the clusters and three different references, all measured on the same 11,889 rows. Event type outscores both subject references.

Two controls, without which the headline number means little. A purity of 0.712 looks convincing and is largely an artefact. Purity computed with majority labelling inflates when the reference is imbalanced and k is large: one type accounts for 44.4% of the labels and there are eighteen clusters, so many acquire that label almost by default. A random assignment with exactly the same cluster sizes already reaches 0.444, so the genuine gain is +0.269 rather than 0.712.

The second control decides the question actually asked. Purity cannot be compared across references with different numbers of classes, because its value depends on cardinality and imbalance; eight event types, fourteen tickers and five sectors are not on a common scale. Adjusted mutual information corrects for both chance and cardinality, and all three references are evaluated on identical rows. On that footing, agreement with event type (0.358) exceeds agreement with ticker (0.188) and sector (0.130).

Reading the result. A purely qualitative reading of the clusters suggests the opposite conclusion, and the tension is worth resolving rather than hiding. Several clusters have company and sector names among their highest-weighted terms, which invites the conclusion that the space merely rediscovers subject. The two observations reconcile once the measurement is put on a fair footing: the space encodes subject *and* event type simultaneously, and in a corpus of fifteen tickers subject is the more visible axis to the eye, because company names dominate the term lists. Measured with a chance-corrected statistic, the event axis explains more of the partition.

One confound must be stated, because it is the first objection an examiner would raise. The rubric assigns labels from words present in the headline, and the embeddings encode those same words, so part of the event-type agreement is guaranteed by construction. What this does not damage is the *comparison*: company names are also present in the headlines, often more than once and at the front, so all three references share the bias and their ordering remains informative even if the absolute values are inflated.

Why it is not wired into retrieval. The evidence supports a descriptive conclusion and not a production change, and the distinction is deliberate. The separation is weak in absolute terms, the choice of k is only weakly determined, and cluster labelling depends entirely on a rubric that covers 15.1% of the corpus. Filtering precedents by a mistaken event type would remove valid precedents silently, which is worse than not filtering at all. The taxonomy is therefore retained as a characterisation of the corpus, and the route the evidence does support, the transparent rubric, is recorded as future work in Chapter 6.

5.7 Case Study 6: Distribution-Free Guarantees for the Triage Score

Question: the triage score is a calibrated probability, but calibration promises nothing about any individual item. Can the score be given a guarantee instead? Answer: yes, split conformal prediction supplies a distribution-free coverage guarantee, which holds under random splitting and holds at the two looser confidence levels under temporal splitting but fails at the strictest one. The price is stark: to promise 90% *coverage, the model can make a definite call on only* 39.5% *of headlines.*

What conformal prediction adds. Platt scaling makes the triage score an honest *aggregate* statement: among items scored near 0.6, roughly 60% proved material. It describes a historical record and promises nothing about the next item, and it offers no recourse if the model is used outside the regime in which it was fitted. Split conformal prediction trades the point estimate for a set and obtains a guarantee that is distribution-free and finite-sample: for a chosen α, the set contains the true class in at least $1 - \alpha$ of cases. It assumes neither normality, nor a well-specified model, nor even a good one.

In a binary problem the returned set has a direct reading. A single label is a definite call; the empty set means neither class is plausible at the requested level; and a set containing *both* labels is an explicit *"I do not know"*, produced by a procedure whose coverage guarantee holds on average over cases. That third case is the one that matters here: a system that refuses to forecast prices should also be able to decline to adjudicate its own triage, rather than pushing forward a 0.51 that only appears to decide.

Setup. The frozen context-only model was loaded exactly as deployed and its test PR-AUC reproduced to 0.538479, identical to the frozen value, before anything else was computed. Without that gate, nothing downstream would describe the model the dissertation reports. Nonconformity is $1 - \hat{p}(\text{true class})$, and the threshold is the $\lceil (n+1)(1-\alpha) \rceil / n$ empirical quantile of the calibration scores. The finite-sample correction is what converts "usually covers" into a guarantee; without it, coverage sits systematically below nominal.

The experiment is run twice, and the contrast is the finding. A *random* split of the test block satisfies exchangeability by construction, so the guarantee must hold and the run serves as a check on the implementation. A *temporal* split, calibrating on the earlier half and predicting the later, is what the deployed system actually does, and exchangeability is not guaranteed.

Results. Table 5.8 and Figure 5.14 report both.

Table 5.8: Split-conformal coverage on the frozen triage model (test block, 32,649 rows, split in half for calibration and evaluation). "Don't know" is the share of items whose set contains both labels.

Split	α	Nominal	Coverage	Set size	"Don't know"
Random	0.05	0.95	0.951	1.784	78.4%
Random	0.10	0.90	0.902	1.605	60.5%
Random	0.20	0.80	0.803	1.323	32.3%
Temporal	0.05	0.95	**0.937**	1.741	74.1%
Temporal	0.10	0.90	0.900	1.584	58.4%
Temporal	0.20	0.80	0.822	1.368	36.8%

Reading the result. The pattern, rather than a single verdict, is the finding. Under random splitting coverage is attained at all three levels, confirming the implementation is correct; had it failed there, the fault would have been in the method and not in the data. Under temporal splitting it holds at 90% and 80% and falls short at 95% (0.937).

The direction is what theory predicts, and stating it plainly is worthwhile: *the tighter the required coverage, the more fragile it is to drift*. Demanding 95% forces the threshold onto the tail of the calibration distribution, and the tail is exactly what moves first when the

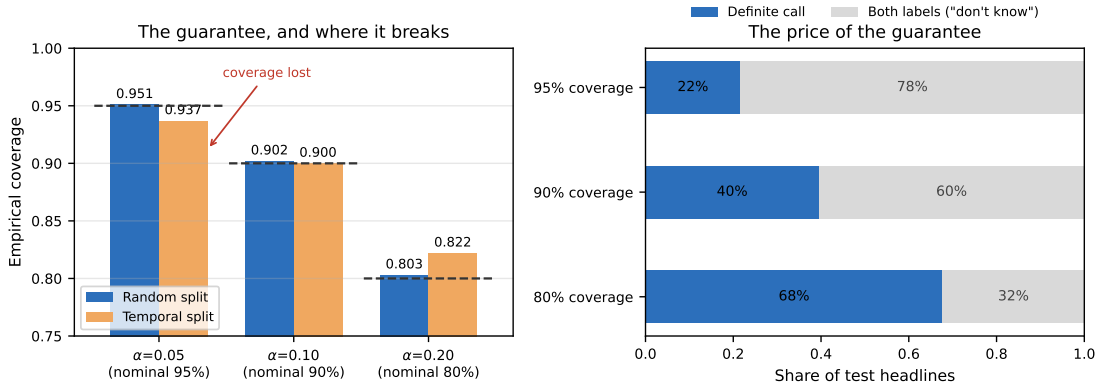

Figure 5.14: Left: empirical coverage against the nominal level (dashed) for both splits. Coverage is attained everywhere except at $\alpha = 0.05$ under temporal splitting. Right: how the guarantee is paid for. At 90% coverage the model issues a definite call on only 40% of headlines and declares "don't know" on the rest.

regime changes. At 80% and 90% there is enough slack to absorb the shift in this window. This is not a shortcoming of conformal prediction; it is the method detecting a breach of exchangeability and identifying the level at which it begins to bite. A guarantee that fails measurably is worth more than a probability that never promised anything and therefore can never be contradicted.

The price, and what it says about the model. The hardest number here is not the coverage but its cost. To promise 90% coverage, the triage model can issue a definite call on only 39.5% of headlines; on the remaining 60.5%, the honest set contains both classes. This does not contradict Case Study 4, it explains it from an independent direction and without training anything new: the available signal simply does not separate most items at the confidence levels usually demanded. It is also the missing instrument for the deployed threshold, which by construction forces a decision on every item; this measurement quantifies how often that forced decision rests on very little.

The layer is therefore kept as measurement and is not wired into production. The reason is design rather than time: the product commits to a legible cadence, and an alert stream that declared "don't know" on 60% of items would break that commitment without anyone having decided to break it.

5.8 Case Study 7: Measuring the Deployment Gap

Question: the model was trained on a 2018–2023 corpus and runs in 2026, a gap this dissertation has repeatedly named as a limitation. How large is it actually? Answer: measurable and concentrated in one input. Pre-event volatility drifts significantly; the other inputs stay stable; and the label prevalence does not trend but oscillates, which explains both why the frozen numbers survive and why the tightest conformal guarantee does not.

Why it needs doing. Asserting a limitation is cheap. An asserted limitation is an opinion; a measured one is a result, and one that can be discussed at a viva with numbers on the table.

Setup. Two measures are used because they see different things. The Population Stability Index compares binned probability mass and carries conventional interpretation bands (< 0.10 stable, 0.10 to 0.25 moderate, > 0.25 significant), inherited from credit-risk practice. The Kolmogorov–Smirnov statistic compares cumulative distributions and catches systematic shifts that binning dilutes.

One methodological point changes how the results should be read. With tens of thousands of points a Kolmogorov–Smirnov test rejects almost any null hypothesis, so its p-value is close to useless here: a statistically significant difference can be trivially small. What is read instead is the statistic D, an effect size in $[0, 1]$ that does not grow merely because the sample is large. Reporting the p-value alone would convert "the sample is large" into "the drift is severe".

Results. Table 5.9 and Figure 5.15 compare the training block (January 2018 to March 2022) against the test block (February to December 2023), which is the drift the frozen results have already passed through.

Table 5.9: Input drift between the training and test blocks, by feature, ordered by severity.

Feature	PSI	Band	KS D
Pre-event volatility (20 d)	**0.281**	significant	0.170
Headline length	0.111	moderate	0.103
Event-day return	0.020	stable	0.036
Five-day momentum	0.014	stable	0.039

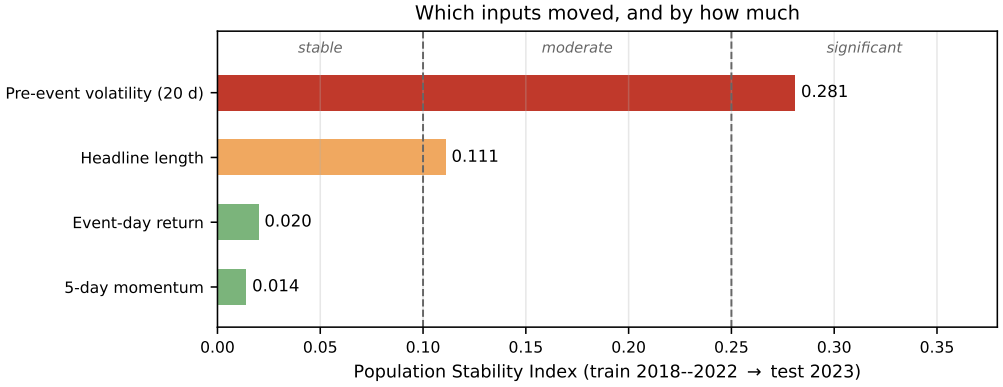

Figure 5.15: Distribution drift for each context feature between the training and test blocks, against the conventional interpretation bands. Only pre-event volatility crosses into the significant band; the two return-based features barely move.

Reading the result. The feature that drifts is the volatility estimate, and the reason is not mysterious: the training window contains the 2020 shock, and realised volatility in that

period does not resemble 2023. The model learned in a more turbulent world than the one it was assessed in.

The label deserves separate treatment, because it is easy to conflate with the inputs. Prevalence moves from 0.3854 in training to 0.3781 in test, a relative change of only 1.9%. Taken at the endpoints alone, the target looks stable. It is not: the validation block, which lies between the two in time, reaches 0.4704. The sequence rises and returns, so comparing only the endpoints would hide an excursion of roughly a fifth. The honest formulation is that *the drift is mostly in volatility, and it is cyclical rather than directional*. That single statement accounts for two things at once: why the frozen numbers survive, since the protocol already traverses one such oscillation, and why the tightest conformal guarantee fails under temporal splitting (Section 5.7), since an oscillating tail is precisely what a 95% guarantee has least slack to absorb.

A live snapshot, and the caveat that makes it usable. The same price-based features were recomputed from current market data for the watchlist. Pre-event volatility returns a PSI of 2.866, far beyond the significant band. That number overstates its case, and the report itself gives the reason away: the mean shifts by only +0.18 standard deviations. A PSI approaching three alongside so small a shift in mean does not describe an unrecognisable market; it describes a sample with few *independent* observations. The cause is mechanical. Roughly a thousand rows come from the ten tickers then watched, over about a hundred trading days, and twenty-day volatility is a rolling statistic, so two consecutive days share nineteen of their twenty returns. The observations are near-repetitions, the resulting distribution is lumpy, and the index penalises the empty bins heavily. The two indices are therefore *not comparable in magnitude*, and placing them side by side in one table would mislead.

What the snapshot does support is still worth having: volatility is again the feature that moves most, obtained independently on present-day data; the shift in mean is modest; and the remaining features stay in the stable and moderate bands, consistent with the within-corpus measurement. The honest answer to *"your model was trained on old data"* is therefore that the distance was measured two independent ways, is concentrated in volatility, is significant but not extreme, and means the 2023 result should not be treated as a promise about 2026. The measurement also supplies a verifiable retraining trigger, the conventional 0.25 threshold, in place of an intuition.

5.9 Case Study 8: Multi-Signal Convergence

Question: the system computes four independent things about a stock on a given day. Does their agreement *rank material days better than the best single signal alone? Answer: not convincingly. Fusion wins at one alert budget out of three, and a gain that depends on the budget chosen is a gain that could have been chosen, so the fused score is not put into production. The volume detector built along the way is kept, and an unexpected negative weight turned out to be the most informative result of the study.*

Where the idea comes from. The design was adapted from World Monitor (*World Monitor* 2026), a free real-time intelligence dashboard recommended by the co-supervisor of this work. What that tool does at scale, fusing dozens of live layers from many providers onto one surface, is out of reach for a project restricted to free APIs, and it also includes scenario tooling that this system deliberately does not attempt. What transfers is the *principle*: an

event on which several independent sources agree deserves more attention than one where a single source fires.

The four signals, and the one that was being thrown away. The system already computes four things about a (ticker, day) pair and treats them separately: whether the price moved unusually *for that stock*, how much news arrived, and how material the triage model judged it. The fourth, *volume*, was not computed at all, even though it costs nothing in data: it arrives in every price bar the system already downloads and was being discarded. It answers the question any trader asks immediately after seeing a move, namely how many people were trading. A move of three per cent on ordinary volume and the same move on triple the usual volume are not the same event.

The volume detector reuses the price detector's causal convention, with three deliberate differences. It standardises the *logarithm* of volume, because volume is strongly right-skewed and strictly positive, so a raw z-score fires almost exclusively upward. It flags only the *upper* tail, because unusually low volume is nearly always a holiday or a half session rather than an event. And it reports a plain ratio ("3.2× the usual volume") alongside the z-score, because the ratio communicates and the z-score decides.

Setup. The unit of analysis is the (ticker, day) pair, which is the unit of the label; ranking individual headlines would fill the top of each day with copies of the same name. Fusion weights are *derived*, by logistic regression on the four standardised signals fitted on the validation block and evaluated on test, never chosen by hand. The fusion model is linear so that each signal's contribution is exact rather than approximate.

One property of the corpus must be stated because it bounds what this study can claim. FNSPID's ticker coverage varies over time: the training block holds thirteen tickers, validation eight and test nine. Collapsing the later blocks to (ticker, day) pairs therefore leaves 1,951 test pairs, a much smaller sample than the retrieval and triage studies rest on. The volume data aligns on every one of them, so this is a property of the corpus and not a defect of the join.

Results. Table 5.10 reports the test block.

Table 5.10: Multi-signal convergence against each signal alone (test block, 1,951 (ticker, day) pairs, prevalence 0.351). Precision@k ranks each day and admits the top k.

Signal	PR-AUC	P@1/day	P@3/day	P@5/day
Price z-score	0.339	0.339	0.347	0.340
Volume z-score	0.363	0.385	0.368	0.340
News intensity	0.386	0.430	0.392	0.389
Triage probability	**0.515**	**0.634**	**0.504**	0.434
Convergence (fused)	0.475	0.584	0.472	**0.439**

Reading the result. Fusion beats the best single signal at one budget out of three (+0.0045 at five alerts per day) and loses at the other two (−0.0498 at one, −0.0317 at three). A mixed result of that shape does not support wiring the score into production.

A gain that depends on which budget is quoted is a gain that could have been selected, and the standard applied throughout this work is that a new capability enters only when the measurement supports it without choosing the angle.

The reason is not mysterious once stated. The four signals are not independent in practice: days with heavy news tend also to be days of high volume and large moves, so combining them adds less than the argument suggests. This is the same lesson the text features and the five extended context features already delivered by different routes. On this corpus, short-horizon materiality is remarkably well summarised by very few numbers.

The negative weight, which is the most useful finding here. The derived weight on news intensity is *negative* (-0.283). More headlines on a day makes that day *less* likely to be material. This is a finding and not a sign error. Days with many headlines tend to be days of automatically generated content, market round-ups, suggestion lists and order-imbalance updates, rather than days on which something happened. It is the same input-quality problem that motivated the relevance filter described in Chapter 4, arriving here from the quantitative side.

It also supplies the empirical justification for a rule this work has followed throughout. A convergence score with hand-chosen weights would almost certainly have given news intensity a positive weight, because that is what intuition says, and it would have been wrong. Deriving the weights turned an intuition into a measurement that contradicted it.

What is kept. The volume detector is retained: it is a genuine new capability at zero data cost, transparent by the same construction as the price detector. The fused score is retained as a measurement only. What transfers to the product is the *human* reading of convergence, reporting how many of the four signals crossed their own thresholds. Saying that three of four signals fired communicates immediately and can be checked against the components; a fused score of 0.73 cannot be checked anywhere.

5.10 Discussion and Threats to Validity

The first four case studies support the core design: volatility-normalised detection is consistent across the market where a fixed threshold is not, semantic retrieval surfaces markedly more sector-analogous precedents than lexical or trivial baselines, the end-to-end alert is faithful to its own computation, and the learned triage adds a large budgeted-precision gain while confirming, against its own text features, that the transparent volatility evidence carries the signal.

The final four examine the same system from the outside, asking what it does *not* know. The embedding space turns out to carry event-type structure, though too weakly to filter precedents by (Section 5.6); the triage score can be given a distribution-free guarantee, at the cost of admitting uncertainty on most items (Section 5.7); the gap between the training corpus and the deployed present is concentrated in one input and is cyclical rather than directional (Section 5.8); and fusing four independent signals does not reliably beat the best of them alone (Section 5.9). Each of the four ends in a decision *not* to change production, taken on the evidence rather than for want of time, and each is recorded as such. All results rest on real data and are reproducible from versioned scripts.

The results also have clear limits, which are set out below under the four standard categories of validity so that nothing is glossed over.

Construct validity: do the metrics measure what is claimed? Two proxies carry weight here. Retrieval relevance is judged by a *sector* label, an automatic stand-in for the human judgement of "analogous", so a same-sector precedent on an unrelated topic counts as a hit; the qualitative inspection in Section 5.3 shows this proxy is reasonable but imperfect. Likewise, the anomaly "extreme move" label is volatility-relative, so it is not an independent ground truth, which is why the primary anomaly argument is the label-free *firing-rate consistency*, not agreement with that label. Finally, the event-study figure measures the price change that *followed* a news item, which is an outcome, not a causal effect of the news; it is reported as evidence, never as a forecast.

Internal validity: could something else explain the effect? The retrieval advantage might, in principle, be inflated by trivially matching a company's news to its own past news; this is removed by construction, because the evaluation forbids same-ticker precedents and measures *cross-ticker* precision only. Lookahead is excluded throughout: every quantity uses information available at the time, and impacts are accumulated strictly forward from the event-day close. The main residual threat is confounding (a price move in a post-event window may be driven by market-wide factors rather than the news), which the short windows (+1, +3, +5 days) limit but do not eliminate, and which the no-causation framing acknowledges openly.

External validity: how far do the findings generalise? The study covers fifteen large-capitalisation US tickers across five sectors, so the numbers need not transfer to small-capitalisation stocks, to other markets, or to other periods, and news coverage is thinner outside large caps. The retrieval study runs on a recent window from a free service; the triage, taxonomy, conformal, drift and convergence studies run on the multi-year FNSPID history, which broadens the period but not the ticker set, since that is fixed by the data card in both cases.

Statistical-conclusion validity: are the comparisons sound? The retrieval results are averaged over five random samples with standard deviations reported (the error bars in Figure 5.5), and the gap between semantic retrieval and every baseline is large relative to that spread; still, five runs on one corpus is a modest sample, and no formal significance test is claimed. The anomaly comparison is reported descriptively, as a difference in firing rates across instruments, rather than as a hypothesis test.

One limit is deliberate rather than a weakness: the system makes no price prediction and evaluates no trading profit, so those questions are out of scope by design. The full FNSPID corpus, a human relevance and usefulness study, and a multi-year impact analysis are the natural next steps, and are carried forward to the conclusions.

5.11 Chapter Conclusions

Across eight case studies on real data, InvestiGator's anomaly trigger fired far more consistently than a fixed threshold, its correlation engine retrieved sector-analogous precedents well above every baseline and independently of the embedding model, its explanations were

faithful to the underlying computation by construction, and its trained triage model nearly quadrupled within-budget alert precision while honestly failing to beat the volatility-only baseline with text. The four studies that followed measured what the system does not know, and each of them ended by leaving production unchanged because the evidence did not support a change. The next chapter draws these findings together against the research questions.

Chapter 6

Conclusions

This dissertation set out to design, implement and evaluate an explainable, reproducible financial-alert system for US retail investors, driven by two independent triggers and delivered through Telegram. This chapter summarises what was achieved against the research questions, revisits the contributions, and states the limitations and the directions they open.

6.1 Main Conclusions

The work is best summarised by returning to the four research questions of Chapter 1, gathered at a glance in Figure 6.1 and then taken one at a time.

Research question	Verdict	One number
RQ1: detect abrupt moves transparently	**Yes**	firing rate 0.015 vs 0.344; F_1 0.516
RQ2: analogous precedents, no lookahead	**Yes** (retrieval)	P@5 0.514 vs 0.346 lexical, 0.240 random
RQ3: faithful and useful explanations	Faithful; useful open	faithful by construction; no human study yet
RQ4: triage beyond volatility	Offline yes; text no	share 0.163 $\rightarrow$ 0.632; PR-AUC 0.542 vs 0.496

Figure 6.1: The four research questions and their verdicts, reported exactly as they fell. Two are an unqualified yes; two are honest splits. Explanations are faithful by construction but their usefulness awaits a human study. Learned triage helps as a product mechanism *on held-out data* and no headline-text model beat the volatility baseline on this corpus; measured in deployment the same gate ranks at chance (Section 6.5), which is why its verdict is qualified rather than green. Numbers are from Chapter 5.

RQ1: transparent detection of abrupt movements. Yes. A rolling z-score detector flags abnormal returns and, more importantly, does so at a steady rate *across stocks of very different volatility*, where a fixed-percentage threshold cannot. As reported in Chapter 5, the firing rate varied by only 0.015 across the fifteen tickers for the z-score versus 0.344 for the fixed baseline, and the z-score reached an F_1 of 0.516 against an extreme-move proxy, more than double the baseline. The method is simple enough that every alert can be explained to the investor.

RQ2: analogous precedents without lookahead. Yes, for retrieval. Semantic retrieval with Sentence-BERT recovered markedly more sector-analogous precedents than every trivial alternative: a cross-ticker precision@5 of 0.514 ± 0.015 (over five runs), against 0.346 for a lexical baseline, 0.240 for random and 0.126 for recency. A per-sector breakdown showed the lift is greatest where domain vocabulary is most distinctive (energy, health). The end-to-end case study showed it surfacing genuinely topical precedents across different company feeds. Impact is measured strictly after the event, in the style of an event study, so no future information leaks into the evidence. This was then validated at scale: on the multi-year FNSPID corpus (about eighty thousand headlines) the same cross-ticker protocol reached a precision@5 of 0.595, above the preliminary figure, and the theme-not-direction property was quantified directly, since the retrieved clusters' direction consistency (0.71) sits barely above its 0.69 chance floor.

RQ3: faithful and useful explanations. Faithful, yes; useful, still open. Because each alert is rendered directly from the same objects the system computes (the z-score and its parameters, or the retrieved precedents with their scores and measured impacts), the explanation cannot diverge from the underlying logic, and every alert closes with an explicit no-prediction disclaimer. Usefulness to a retail investor is plausible by design but has not yet been measured with a human study, and is therefore reported as an open point rather than a settled result.

The faithfulness half was subsequently extended from templated text to *generated* language. A language model was introduced strictly as a narrator, permitted to phrase the evidence but never to supply it, and a runtime guard validates its output against the evidence before delivery: any number absent from the computed evidence, any figure whose sign has been inverted, any predictive or advisory wording, and any attribution that contradicts the computed one causes the whole response to be discarded in favour of the deterministic text. The guarantee is therefore a property of the system rather than of the model's good behaviour.

That distinction is what makes the claim measurable, and it produces two figures rather than one. Across a fixed set of cases against two providers, the raw model output violated the evidence in a small number of cases, which is the number that justifies the guard existing; the text actually delivered violated it in none, which is the number that tests the guard. Both are reported, together with the honest observation that the second is partly circular, since the same checker both decides and grades. The counterweight is an adversarial corpus: an independent red-team exercise against an earlier version of the guard produced a set of confirmed bypasses, each reproduced mechanically, and those attacks are now permanent regression tests. The first design, a blocklist of forbidden phrasings, failed against them decisively, and the lesson generalises: the space of paraphrases is unbounded while any list of them is finite, so the guard was rebuilt as a closed vocabulary in which anything not explicitly permitted is refused.

RQ4: learned triage beyond volatility baselines. No on the text hypothesis; yes on the mechanism. A supervised materiality model was trained, calibrated and tested on 79,753 FNSPID examples (2018–2023) under a strict temporal protocol. As a product mechanism, learned triage is worthwhile *on held-out data from the corpus it was fitted on*: within a five-alerts-per-day budget it raised the share of material alerts from 0.163 to 0.632, with calibrated probabilities. That qualification is not decorative. Measured in deployment, on a population enriched by the upstream filters, the same gate shows no significant benefit (Section 6.5), so the mechanism claim is an evaluation-set claim and is stated as one. But

the pre-committed decisive comparison went the other way: no model reading the headline text beat the volatility-only baseline (PR-AUC 0.542 against 0.496 for the main context+text model), so on this corpus the triage signal lives in market context, not headline wording. Together with the Isolation Forest comparison of the anomaly study, this is the second fair, causal test in which the transparent choice won, and both results are reported exactly as they fell. The negative-on-text result was further stress-tested: a cluster bootstrap over (ticker, day) confirms that adding headline text lowers precision robustly rather than by chance, and a deliberately fair re-test, with tuned regularisation, a dimensionally reduced text block and a finance-domain encoder, recovers the text only to the level of market context, never above it. The finding is robust, not an artefact of an under-fit text pipeline.

A final observation concerns how the triage model is *used* rather than how it scores. The deployed system gated alerts at a hand-set probability threshold, which sat on top of a calibrated model without ever exploiting that calibration. Sweeping the threshold across the frozen test set under an explicit cost ratio between a missed move and a false alarm turns the constant into a derived operating point, and it also exposes what the original choice had silently assumed: the deployed value corresponds to treating a missed move as costing roughly the same as a false alarm. Under equal costs the optimum sits essentially where the constant already was, so the choice is vindicated, but it is now argued rather than asserted. The same analysis re-tested the negative result on operational terms, comparing policies at matched daily alert budgets rather than by PR-AUC, and the learned score again failed to beat volatility consistently. The negative therefore survives a change of metric, which is a stronger statement than the original result alone.

6.2 Positions Taken by Exclusion

Some of the clearest design decisions in this work are things the system deliberately does not do. Stating them as positions rather than omissions matters, because each was reachable with the data and effort available and was declined on principle.

Third-party forecasts. Analyst price targets and recommendation trends are freely available and would have improved coverage of the question "why did this move". They were excluded because importing another party's prediction into a system defined by not predicting is a contradiction that no amount of framing removes. Coherence was preferred to coverage.

Personal holdings. Portfolio-level analytics, concentration and correlation exposure, are computable from prices alone and were among the most requested capabilities. They were excluded for two compounding reasons: holdings are personal financial data, which turns a research prototype into a data-protection obligation rather than a data-protection discussion, and any statement of the form "your largest positions are effectively one bet" is advice however it is phrased, which crosses the boundary that the system's ethical position depends on not crossing.

Conversational agency. The narrator is a pure function from evidence to a paragraph, not a multi-turn assistant and not an agent. A single model calling several tools is not a multi-agent system, and describing it as one would have been the least defensible claim in the dissertation.

Insider filings and scheduled-disclosure feeds. These are genuinely useful and genuinely free. They were excluded on scope rather than principle: each adds an ingestion path and a failure mode without an evaluation attached, and an unevaluated component contributes nothing to the questions this work asks. They appear in future work instead.

A fused convergence score. Ranking alerts by the agreement of several independent signals is intuitive, and the machinery for it was built and measured (Section 5.9). It was declined because the measurement did not support it: fusion beat the best single signal at one alert budget out of three, and a gain that appears only at the budget one chooses to quote is not a result. The component built along the way, a volume anomaly detector at zero data cost, was kept, and so was the readable part of the idea, reporting how many signals agree rather than a fused number a user cannot check. This is the pattern several of these positions share: build it, measure it, and let the measurement decide, including when the measurement says no.

Streaming rather than polling. Delivery latency is a real property of an alerting product, and the obvious engineering answer is a streaming connection to the price feed. It was declined for reasons that are worth separating, because only one of them is about effort. The estimated work was substantial, but the decisive argument is that no research question in this thesis becomes answerable by shortening delivery from minutes to seconds: the detector is defined on daily bars, and the retrieved precedents are days old by construction. Reasoning about the two user profiles of Chapter 1 points the same way, and it should be labelled for what it is: neither profile, as characterised there, would plausibly distinguish five seconds from five minutes for an alert meant to prompt reflection rather than a trade. That is an assumption about two constructed profiles, not a finding about people, and it would be settled by the same human study that RQ3 still awaits. The system therefore polls on a fixed cycle, which is enough for the claims it makes, and the resulting latency is *measured* rather than promised.

An earlier version of this paragraph stated that latency is bounded by the polling cycle. Measuring it showed that this is false, and the correction is worth keeping visible because it changes what the decision above is a decision about. Decomposing the recorded timestamps of every delivered alert into publication $\rightarrow$ detection and detection $\rightarrow$ delivery gives a median of roughly one second for the second component and around two and a half hours for the first. Shortening the cycle from the best-effort scheduler used earlier, measured at one and a half to two hours, to a sixty-second worker moved the end-to-end median by well under an hour, because the cycle was never the binding constraint. The delay lives in discovery: the free news endpoint is not a real-time channel, and the most recent headline that passes the relevance filter is typically older than the most recent headline in the feed. A streaming price connection would not have touched either. The defensible claim is therefore narrower and, unlike the original one, verifiable: the system delivers in seconds what its source gives it. A product aimed at intraday trading would need both the streaming path and a paid news feed; this one is not that product, and saying so is cheaper and more truthful than building an unused capability.

6.3 Research Questions and Objectives Achieved

The general objective, to design, implement and evaluate an explainable, reproducible alerting system driven by two triggers, was met: InvestiGator is a working system whose two

triggers were proven end to end and whose core components, including the trained triage model, were evaluated on real data. Taking the five objectives of Section 1.3 one at a time, four are met in full and one in half:

- *Build an explainable alerting pipeline driven by the two triggers.* Met. Both triggers run end to end in continuous deployment, and every alert is rendered from the computed objects.
- *Evaluate the anomaly detector against transparent baselines.* Met, and extended beyond what was promised: the detector was compared not only against a fixed threshold but against two learned detectors and a second volatility estimator under the same causal protocol.
- *Evaluate the retrieval of historical news against trivial baselines.* Met, against random, recency and lexical baselines, and subsequently repeated at scale on the multi-year corpus.
- *Assess whether the explanations are faithful and useful.* Half met, and the half that is missing is named rather than absorbed into the half that succeeded. Faithfulness was established, first by construction and then, for generated language, by a runtime guard with an adversarial regression corpus. Usefulness was not measured: the instrument exists and the analysis threshold was fixed in advance, but no human study was run, so no claim is made.
- *Train, calibrate and honestly evaluate a materiality-triage model.* Met, including the part that was least comfortable to report, since the pre-committed decisive comparison went against the trained model.

The methodological commitments of Chapter 3 also held: a thin end-to-end slice was delivered first, each component was kept in its simplest defensible form, the evaluation followed a design fixed in advance, and the whole system is reproducible from versioned scripts.

6.4 Contributions Revisited

As an Artificial Intelligence *Engineering* dissertation, the contribution is the integration, application and critical evaluation of existing components into a working, explainable and reproducible whole, rather than a new algorithm. Four concrete contributions stand out. First, a documented and reproducible methodology for **news–market impact correlation** that combines sentence-embedding retrieval with event-study impact windows, with explicit choices of similarity metric and horizon and explicit avoidance of lookahead. Second, **InvestiGator**, an **XAI-first alerting system** whose entire reasoning chain (detection, evidence, sources and precedents) is exposed to a non-expert user and is faithful by construction. Third, a trained and calibrated **materiality-triage model**, labelled by the system's own event-study code over the multi-year corpus, integrated into the alerts as explained evidence behind a configuration gate, and monitored in deployment against the outcomes that actually followed — monitoring that then showed the gate ranking at chance on the deployed population, which is itself part of the contribution. Fourth, a **critical, honest evaluation** of each core component against trivial, lexical and volatility baselines on real data, including two pre-committed statistical-versus-learned comparisons that the transparent methods won; results, ablations and limitations are all reported plainly and regenerated from versioned scripts.

6.5 Limitations

Several limitations remain, stated plainly:

- **Corpus.** The primary retrieval evaluation used a recent, few-month news corpus from a free service, which makes those figures preliminary; a follow-up on the multi-year FNSPID corpus validated the retrieval at scale (precision@5 of 0.595). The triage study did use the multi-year FNSPID subset, but with fourteen of the fifteen tickers (Meta is indexed under its former symbol in the corpus) and headline text only.
- **Proxies.** Relevance is proxied by sector agreement, and the anomaly label is volatility-relative, which is why the label-free firing-rate argument is given primacy.
- **News coverage, now measured rather than asserted.** Earlier statements that the free news tier is limited never said by how much. Over a year of captured headlines across the twelve names, at least one relevant headline was present on 88.5% of the days the detector flagged as unusual ($|z| \geq 1.5$; 90.4% at $|z| \geq 3.0$). The complement is the share of unusual days on which *no code change helps*: if the provider does not attach a story to the ticker, it never enters the funnel and none of the five gates is ever consulted. This separates two limitations that were previously conflated, what the *design* discards and what the *source* never delivered, and only the first is a decision of this work. The figure is an upper bound, because it asks whether *a* story was present and not whether the *right* one was; establishing the latter needs per-day human judgement. Coverage is also markedly uneven: the worst-covered name reaches only 50% while five names exceed 97%, and the worst-covered name is simultaneously the one with the highest headline density, so its coverage arrives in bursts. The obvious explanation, truncation against the provider's per-request ceiling, was tested and rejected: no request window in the corpus came within a third of that ceiling.
- **Short text.** News is represented by headlines only, which limits the semantics captured.
- **Theme, not direction.** Embedding similarity captures theme, not direction, so a retrieved cluster can mix up and down moves; the mean impact is theme-level evidence, not a directional signal, which is why the precedents are always shown individually. The natural remedy, filtering precedents by event type, was tested rather than assumed (Section 5.6): the embedding space does carry event-type structure, and carries it more strongly than it carries subject, but too weakly to filter on. Applying a mistaken event type would drop valid precedents silently, so the taxonomy stays descriptive.
- **Confidence is thin, and now quantified.** Wrapping the frozen triage model in split conformal prediction (Section 5.7) shows that guaranteeing 90% coverage leaves a definite call on only 39.5% of headlines. This is an independent confirmation of the Chapter 5 finding that the available signal is weak, obtained without training anything new, and it is a limitation of the evidence rather than of the implementation.
- **The deployment gap is real, and its size is known.** The model was trained on a 2018–2023 corpus and runs later. That gap is no longer only asserted: it is concentrated in pre-event volatility, which drifts into the significant band, while the return-based features stay stable, and the label prevalence oscillates rather than trends (Section 5.8). The reported figures should therefore be read as measurements under one such oscillation, not as promises about a later period.

- **The deployed gate shows no measurable benefit on live data, and the reason is instructive.** The offline result of Chapter 5 (budgeted precision 0.163 → 0.632) is an evaluation-set result. In deployment the system logs every triage decision and labels it days later with the outcome that actually followed, using the same rule as training. Over 530 matured decisions the alerts the gate *kept* were material 0.592 of the time against 0.647 for the ones it *suppressed*: no benefit, with the point estimate in the wrong direction and the difference not significant ($z = -1.28$, $p = 0.20$).

  Asking *why* is what turns that into a usable result, because two different failures produce it and they demand opposite remedies. If the score still *ranks* correctly and only its numbers are on the wrong scale, the fix is a two-parameter recalibration. If the score does not rank at all, no recalibration helps, because a sigmoid is monotone and preserves the ordering exactly. Measured on the deployed population, the ROC-AUC is 0.494, with a cluster bootstrap interval of [0.391, 0.601] resampling whole (ticker, day) units, since headlines sharing a company and a day share one label and the effective sample is 145 units rather than 530 rows. The interval is centred on chance. There is therefore no measurable ranking ability to preserve, and the gate is selecting at chance whatever threshold it is given. Calibration is separately and genuinely wrong, and would improve (Brier 0.260 → 0.234), but fixing it would produce a better number and an identical system.

  The most likely explanation is not that the model is broken but that it is *redundant*. Materiality among logged decisions runs at 0.626 against a training prevalence of 0.378, because decisions are only logged for headlines that already cleared the relevance and freshness filters. Those cheap upstream stages have already removed most of what the learned stage was trained to remove, leaving it little to separate. This is a general lesson about placing a learned component in a pipeline rather than a fault in the component: a model evaluated in isolation and deployed behind filters is not evaluated on the distribution it will see. The offline result of Chapter 5 stands as an evaluation-set result; the deployed gate has not earned its place on this evidence, and saying so is cheaper than defending it.

- **No human study.** Usefulness to a real investor has not yet been measured.
- **By design.** The system makes no price prediction and no trade, so profit-based questions are out of scope, not unanswered.

6.6 Future Work

The limitations map directly onto future work, summarised in Figure 6.2.

- Evaluate the *impact magnitudes* on the multi-year FNSPID corpus. The retrieval itself has already been validated there (precision@5 of 0.595, with dispersion and direction-consistency statistics); what remains is the market-adjusted impact-magnitude study over those multi-year precedents.
- Run a small human study, applying a clarity, completeness and actionability rubric to a set of alerts, to settle the open usefulness question (RQ3).
- Attack the open text gap of RQ4: richer text than headlines (article bodies, financial sentiment as a feature), the missing ticker recovered under its former symbol, and the

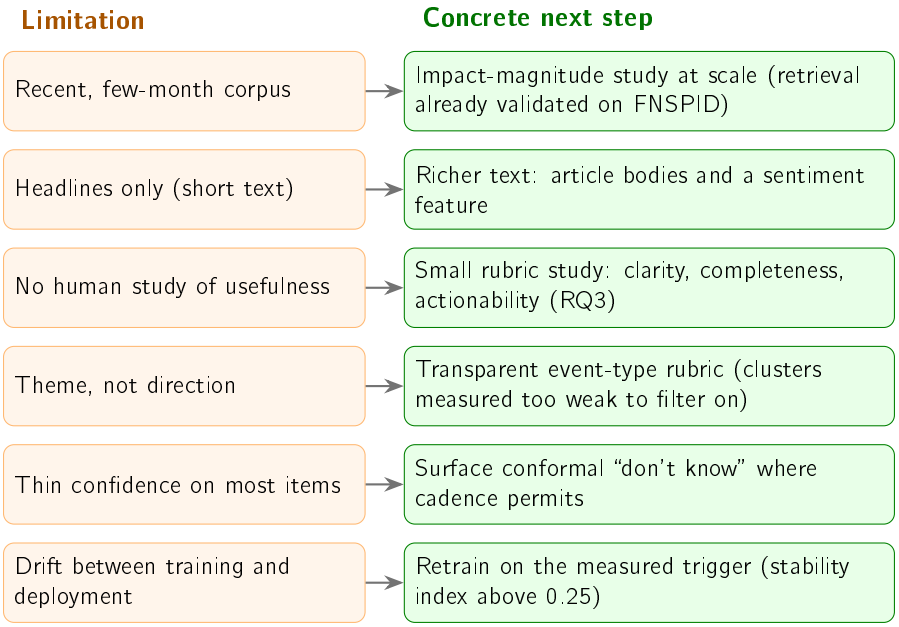

Figure 6.2: Each stated limitation maps directly onto a concrete next step. Several have already seen a first deployment-side iteration (a live knowledge base matured against observed prices, explicit directional-disagreement flags, and an intraday quote during market hours), reported as engineering follow-through whose formal evaluation remains future work.

live post-validation loop accumulating fresh labelled decisions for periodic retraining; a contextual-bandit treatment of the alert threshold is the natural learned extension.

- Close the remaining half of the news-coverage question. The share of unusual days on which *a* relevant story reached the funnel is now counted (88.5%, above), but whether the *right* story arrived is not, and no automatic proxy settles it: it needs per-day human judgement on a sample of flagged days, which is the same instrument the usefulness study requires and could be collected in the same pass.

Live deployment after the evaluation was frozen already motivated a first iteration of several of these items: the deployed system now grows a *live* knowledge base from the headlines it scans (each case matured against observed prices days later), ranks precedents with an age-decay factor while always displaying the true similarity and the age of each case, flags directional disagreement explicitly, and evaluates the day's move in progress from a real-time quote during market hours. Deployment also taught its own lesson, reported in Chapter 4: a single free price source proved a single point of failure on hosted infrastructure, and was replaced by a multi-provider fallback chain. These deployment-side iterations reuse the validated components unchanged and are reported here as engineering follow-through; their formal evaluation remains future work. One item is already *validated* rather than speculative: the extended comparison of Case Study 1 showed that an exponentially weighted volatility estimate improves the detector's proxy agreement substantially, so adopting it is a measured, one-line change waiting only on an explainability decision. Together these would turn a sound prototype into a more thoroughly validated system, without giving up the simplicity and transparency that make it defensible.

Bibliography

Aamodt, Agnar and Enric Plaza (1994). "Case-Based Reasoning: Foundational Issues, Methodological Variations, and System Approaches". In: *AI Communications* 7.1, pp. 39–59. doi: 10.3233/AIC-1994-7104.

Adadi, Amina and Mohammed Berrada (2018). "Peeking Inside the Black-Box: A Survey on Explainable Artificial Intelligence (XAI)". In: *IEEE Access* 6, pp. 52138–52160. doi: 10.1109/ACCESS.2018.2870052.

Ahmed, Mohiuddin, Abdun Naser Mahmood, and Md. Rafiqul Islam (2016). "A Survey of Anomaly Detection Techniques in Financial Domain". In: *Future Generation Computer Systems* 55, pp. 278–288. doi: 10.1016/j.future.2015.01.001.

Angelopoulos, Anastasios N. and Stephen Bates (2023). "Conformal Prediction: A Gentle Introduction". In: *Foundations and Trends in Machine Learning* 16.4, pp. 494–591. doi: 10.1561/2200000101.

Araci, Dogu (2019). "FinBERT: Financial Sentiment Analysis with Pre-trained Language Models". In: *arXiv preprint*. arXiv: 1908.10063 [cs.CL].

Bansal, Gagan et al. (2021). "Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance". In: *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. doi: 10.1145/3411764.3445717.

Barber, Brad M. and Terrance Odean (2008). "All That Glitters: The Effect of Attention and News on the Buying Behavior of Individual and Institutional Investors". In: *The Review of Financial Studies* 21.2, pp. 785–818. doi: 10.1093/rfs/hhm079.

Barberis, Nicholas and Richard Thaler (2003). "A Survey of Behavioral Finance". In: *Handbook of the Economics of Finance*. Ed. by George M. Constantinides, Milton Harris, and René M. Stulz. Vol. 1. Elsevier. Chap. 18, pp. 1053–1128. doi: 10.1016/S1574-0102(03)01027-6.

Barredo Arrieta, Alejandro et al. (2020). "Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges toward Responsible AI". In: *Information Fusion* 58, pp. 82–115. doi: 10.1016/j.inffus.2019.12.012.

Blume, Marshall E. (1971). "On the Assessment of Risk". In: *The Journal of Finance* 26.1, pp. 1–10. doi: 10.1111/j.1540-6261.1971.tb00584.x.

Bollerslev, Tim (1986). "Generalized Autoregressive Conditional Heteroskedasticity". In: *Journal of Econometrics* 31.3, pp. 307–327. doi: 10.1016/0304-4076(86)90063-1.

Breunig, Markus M. et al. (2000). "LOF: Identifying Density-Based Local Outliers". In: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pp. 93–104. doi: 10.1145/342009.335388.

Brown, Stephen J. and Jerold B. Warner (1985). "Using Daily Stock Returns: The Case of Event Studies". In: *Journal of Financial Economics* 14.1, pp. 3–31. doi: 10.1016/0304-405X(85)90042-X.

Cambridge Centre for Alternative Finance (2026). *Global AI in Financial Services Report: Adoption, Impact and Risks*. Cambridge Judge Business School, University of Cambridge. url: https://www.jbs.cam.ac.uk/faculty-research/centres/alternative-

finance/publications/2026-global-ai-in-financial-services-report/ (visited on 06/21/2026).

Cardillo, Giovanni and Helen Chiappini (2024). "Robo-advisors: A Systematic Literature Review". In: *Finance Research Letters* 62, p. 105119. doi: 10.1016/j.frl.2024.105119.

Chandola, Varun, Arindam Banerjee, and Vipin Kumar (2009). "Anomaly Detection: A Survey". In: *ACM Computing Surveys* 41.3, 15:1–15:58. doi: 10.1145/1541880.1541882.

D. Sculley et al. (2015). "Hidden Technical Debt in Machine Learning Systems". In: *Advances in Neural Information Processing Systems 28 (NIPS 2015)*. Curran Associates, Inc., pp. 2503–2511. url: https://papers.nips.cc/paper_files/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract.html.

D'Acunto, Francesco, Nagpurnanand Prabhala, and Alberto G. Rossi (2019). "The Promises and Pitfalls of Robo-Advising". In: *The Review of Financial Studies* 32.5, pp. 1983–2020. doi: 10.1093/rfs/hhz014.

Da, Zhi, Joseph Engelberg, and Pengjie Gao (2011). "In Search of Attention". In: *The Journal of Finance* 66.5, pp. 1461–1499. doi: 10.1111/j.1540-6261.2011.01679.x.

Devlin, Jacob et al. (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Association for Computational Linguistics, pp. 4171–4186. doi: 10.18653/v1/N19-1423. arXiv: 1810.04805.

Ding, Xiao et al. (2015). "Deep Learning for Event-Driven Stock Prediction". In: *Proceedings of the 24th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 2327–2333. url: https://www.ijcai.org/Abstract/15/329.

Dong, Zihan, Xinyu Fan, and Zhiyuan Peng (2024). "FNSPID: A Comprehensive Financial News Dataset in Time Series". In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*. Association for Computing Machinery, pp. 4918–4927. doi: 10.1145/3637528.3671629. arXiv: 2402.06698 [q-fin.ST].

Doshi-Velez, Finale and Been Kim (2017). "Towards a Rigorous Science of Interpretable Machine Learning". In: *arXiv preprint*. arXiv: 1702.08608 [stat.ML].

Engle, Robert F. (1982). "Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation". In: *Econometrica* 50.4, pp. 987–1007. doi: 10.2307/1912773.

Fama, Eugene F. (1970). "Efficient Capital Markets: A Review of Theory and Empirical Work". In: *The Journal of Finance* 25.2, pp. 383–417. doi: 10.2307/2325486.

Fama, Eugene F. et al. (1969). "The Adjustment of Stock Prices to New Information". In: *International Economic Review* 10.1, pp. 1–21. doi: 10.2307/2525569.

Friedman, Jerome H. (2001). "Greedy Function Approximation: A Gradient Boosting Machine". In: *The Annals of Statistics* 29.5, pp. 1189–1232. doi: 10.1214/aos/1013203451.

Gallup (2025). *What Percentage of Americans Own Stock?* url: https://news.gallup.com/poll/266807/percentage-americans-owns-stock.aspx (visited on 06/21/2026).

Gama, João et al. (2014). "A Survey on Concept Drift Adaptation". In: *ACM Computing Surveys* 46.4, pp. 1–37. doi: 10.1145/2523813.

Google (June 2026). *Our latest Google Finance upgrades, including a new app*. url: https://blog.google/products-and-platforms/products/search/google-finance-updates-june-2026/ (visited on 08/07/2026).

Guidotti, Riccardo et al. (2018). "A Survey of Methods for Explaining Black Box Models". In: *ACM Computing Surveys* 51.5, 93:1–93:42. doi: 10.1145/3236009.

Johnson, Jeff, Matthijs Douze, and Hervé Jégou (2021). "Billion-Scale Similarity Search with GPUs". In: *IEEE Transactions on Big Data* 7.3, pp. 535–547. doi: 10.1109/TBDATA.2019.2921572.

Kahneman, Daniel and Amos Tversky (1979). "Prospect Theory: An Analysis of Decision under Risk". In: *Econometrica* 47.2, pp. 263–291. doi: 10.2307/1914185.

Kearney, Colm and Sha Liu (2014). "Textual Sentiment in Finance: A Survey of Methods and Models". In: *International Review of Financial Analysis* 33, pp. 171–185. doi: 10.1016/j.irfa.2014.02.006.

Lee, John D. and Katrina A. See (2004). "Trust in Automation: Designing for Appropriate Reliance". In: *Human Factors* 46.1, pp. 50–80. doi: 10.1518/hfes.46.1.50_30392.

Lipton, Zachary C. (2018). "The Mythos of Model Interpretability". In: *Communications of the ACM* 61.10, pp. 36–43. doi: 10.1145/3233231.

Liu, Fei Tony, Kai Ming Ting, and Zhi-Hua Zhou (2008). "Isolation Forest". In: *2008 Eighth IEEE International Conference on Data Mining (ICDM)*, pp. 413–422. doi: 10.1109/ICDM.2008.17.

Loughran, Tim and Bill McDonald (2011). "When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks". In: *The Journal of Finance* 66.1, pp. 35–65. doi: 10.1111/j.1540-6261.2010.01625.x.

Lundberg, Scott M. and Su-In Lee (2017). "A Unified Approach to Interpreting Model Predictions". In: *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pp. 4765–4774. arXiv: 1705.07874.

Manning, Christopher D., Prabhakar Raghavan, and Hinrich Schütze (2008). *Introduction to Information Retrieval*. Cambridge, England: Cambridge University Press. isbn: 978-0-521-86571-5.

Mikolov, Tomas et al. (2013). "Efficient Estimation of Word Representations in Vector Space". In: *arXiv preprint*. arXiv: 1301.3781.

Miller, Tim (2019). "Explanation in Artificial Intelligence: Insights from the Social Sciences". In: *Artificial Intelligence* 267, pp. 1–38. doi: 10.1016/j.artint.2018.07.007.

Niculescu-Mizil, Alexandru and Rich Caruana (2005). "Predicting Good Probabilities with Supervised Learning". In: *Proceedings of the 22nd International Conference on Machine Learning (ICML '05)*, pp. 625–632. doi: 10.1145/1102351.1102430.

Pang, Guansong et al. (2021). "Deep Learning for Anomaly Detection: A Review". In: *ACM Computing Surveys* 54.2, 38:1–38:38. doi: 10.1145/3439950.

Pennington, Jeffrey, Richard Socher, and Christopher D. Manning (2014). "GloVe: Global Vectors for Word Representation". In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1532–1543. doi: 10.3115/v1/D14-1162.

Reimers, Nils and Iryna Gurevych (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992. doi: 10.18653/v1/D19-1410.

Ribeiro, Marco Tulio, Sameer Singh, and Carlos Guestrin (2016). ""Why Should I Trust You?": Explaining the Predictions of Any Classifier". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144. doi: 10.1145/2939672.2939778.

Robertson, Stephen and Hugo Zaragoza (2009). "The Probabilistic Relevance Framework: BM25 and Beyond". In: *Foundations and Trends in Information Retrieval* 4.1–2, pp. 1–174. doi: 10.1561/1500000019.

Robinhood Markets (Mar. 2025). *Introducing Robinhood Strategies, Robinhood Banking, and Robinhood Cortex*. url: https://robinhood.com/us/en/newsroom/introducing-strategies-banking-and-cortex/ (visited on 08/07/2026).

Rousseeuw, Peter J. (1987). "Silhouettes: a graphical aid to the interpretation and validation of cluster analysis". In: *Journal of Computational and Applied Mathematics* 20, pp. 53–65. doi: 10.1016/0377-0427(87)90125-7.

Rudin, Cynthia (2019). "Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead". In: *Nature Machine Intelligence* 1.5, pp. 206–215. doi: 10.1038/s42256-019-0048-x.

Salton, Gerard, A. Wong, and C. S. Yang (1975). "A Vector Space Model for Automatic Indexing". In: *Communications of the ACM* 18.11, pp. 613–620. doi: 10.1145/361219.361220.

Securities Industry and Financial Markets Association (SIFMA) (2025). *2025 Capital Markets Fact Book*. url: https://www.sifma.org/resources/research/fact-book/ (visited on 06/21/2026).

Tetlock, Paul C. (2007). "Giving Content to Investor Sentiment: The Role of Media in the Stock Market". In: *The Journal of Finance* 62.3, pp. 1139–1168. doi: 10.1111/j.1540-6261.2007.01232.x.

Vasicek, Oldrich A. (1973). "A Note on Using Cross-Sectional Information in Bayesian Estimation of Security Betas". In: *The Journal of Finance* 28.5, pp. 1233–1239. doi: 10.1111/j.1540-6261.1973.tb01452.x.

Vaswani, Ashish et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pp. 5998–6008. arXiv: 1706.03762.

Vinh, Nguyen Xuan, Julien Epps, and James Bailey (2010). "Information Theoretic Measures for Clusterings Comparison: Variants, Properties, Normalization and Correction for Chance". In: *Journal of Machine Learning Research* 11.95, pp. 2837–2854. url: https://www.jmlr.org/papers/v11/vinh10a.html.

Vovk, Vladimir, Alexander Gammerman, and Glenn Shafer (2005). *Algorithmic Learning in a Random World*. New York: Springer. isbn: 978-0-387-00152-4. doi: 10.1007/b106715.

Welch, Ivo (2022). "The Wisdom of the Robinhood Crowd". In: *The Journal of Finance* 77.3, pp. 1489–1527. doi: 10.1111/jofi.13128.

World Monitor (2026). *Real-time global intelligence dashboard*. url: https://worldmonitor.app (visited on 07/31/2026).

Wu, Shijie et al. (2023). "BloombergGPT: A Large Language Model for Finance". In: *arXiv preprint*. arXiv: 2303.17564 [cs.LG].

Xing, Frank Z., Erik Cambria, and Roy E. Welsch (2018). "Natural Language Based Financial Forecasting: A Survey". In: *Artificial Intelligence Review* 50.1, pp. 49–73. doi: 10.1007/s10462-017-9588-9.

Yang, Yi, Mark Christopher Siy Uy, and Allen Huang (2020). "FinBERT: A Pretrained Language Model for Financial Communications". In: *arXiv preprint*. arXiv: 2006.08097.

Appendix A

Reproducibility

This appendix gathers the material that lets the results be reproduced and inspected: the software environment, the data at each stage, and the evaluation outputs behind the numbers reported in the body.

A.1 Environment

The system is implemented in Python 3.12, chosen for the maturity of its packages for the machine-learning stack. Dependencies are pinned and frozen in a lock file so the environment can be recreated across machines. The numerical and data handling rely on established open-source libraries; sentence embeddings use the Sentence-BERT framework on a CPU build of the underlying deep-learning library, installed from a dedicated index to avoid the much larger GPU download; figures are produced with a standard plotting library. Table A.1 lists the pinned versions of the core dependencies; the complete set is frozen in a lock file alongside them.

Table A.1: Pinned versions of the core dependencies (Python 3.12).

Library	Version
numpy	2.1.3
pandas	2.2.3
matplotlib	3.11.0
scikit-learn	1.9.0
yfinance	1.4.1
sentence-transformers	5.6.0
transformers	5.12.1
torch (CPU build)	2.12.1
pytest	8.3.4
ruff	0.8.6

A.2 Code Organisation

The code is organised as one package per component, mirroring the architecture of Figure 4.1: market data, news retrieval, anomaly detection, the correlation engine (similarity and event study), the historical knowledge base, the explanation engine, and alert delivery, with a small orchestration layer wiring them into the two end-to-end flows. Pure logic is separated from input/output, and heavy or networked dependencies are loaded lazily, so the

numerical core is unit-tested without a network connection or the model stack. Secrets are read only from a local, ignored environment file and are never written to versioned files.

A.3 The Complete Pipeline on One Page

The body of the dissertation deliberately presents the system through several small, simple diagrams (architecture, data model, per-trigger flows, the life of one alert). Figure A.1 is the opposite view, kept in the appendix precisely because it is dense: the *complete* deployed pipeline on a single landscape page, with every gate and its production configuration value annotated. Each value is a disclosed deployment parameter (the evaluation of Chapter 5 freezes its own, stricter settings, as noted on the figure).

A.4 Tests and Reproducible Results

The system is covered by an automated test suite spanning the anomaly detector, the event study, similarity, the knowledge base, the news parser, the explanation engine and the evaluation modules, together with an offline end-to-end smoke test of the news trigger. Tests that require network access or heavy models are marked and excluded from the default run so that it is fast, deterministic and offline.

Every experimental result in Chapter 5 is regenerated by a deterministic, fixed-seed procedure that writes its tables and figures directly into the evaluation and figure directories, so the loop between computation and document is closed and auditable. Each of the four case studies has one such procedure: precedent retrieval and its per-sector breakdown, the anomaly firing-rate and window ablation, the statistical-versus-learned detector comparison, and the materiality-triage dataset build and training, and the worked triage decomposition, the embedding-space projection and the production funnel are produced the same way. Every headline number in the body is the value written by one of these procedures. The data-fetch step that precedes them, and the full build of the historical knowledge base, follow the same deterministic recipe.

A.5 Every Number Traced to Its Source

Each number is written by its deterministic procedure into a frozen results record, and the thesis quotes that record verbatim, so the loop between computation and document is closed and auditable. Table A.2 gathers every headline result with the value as reported and the case study it belongs to; each can be regenerated, and verified, in isolation.

The lower block of that table was produced after the results in the upper block and never alters them: each of those procedures reads the frozen artefacts, and the two that depend on the triage model reproduce its frozen test score exactly before reporting anything, halting otherwise. That check is what makes them statements about the model this dissertation reports rather than about a model that happens to be lying next to it.

Live operation. Beyond reproducibility on demand, InvestiGator operated live. A worker process scanned the watchlist on a sixty-second cycle and delivered explained alerts to a public Telegram channel, having replaced an earlier scheduled workflow that ran after each US market close; the shared, versioned alert record accumulated dozens of real alerts,

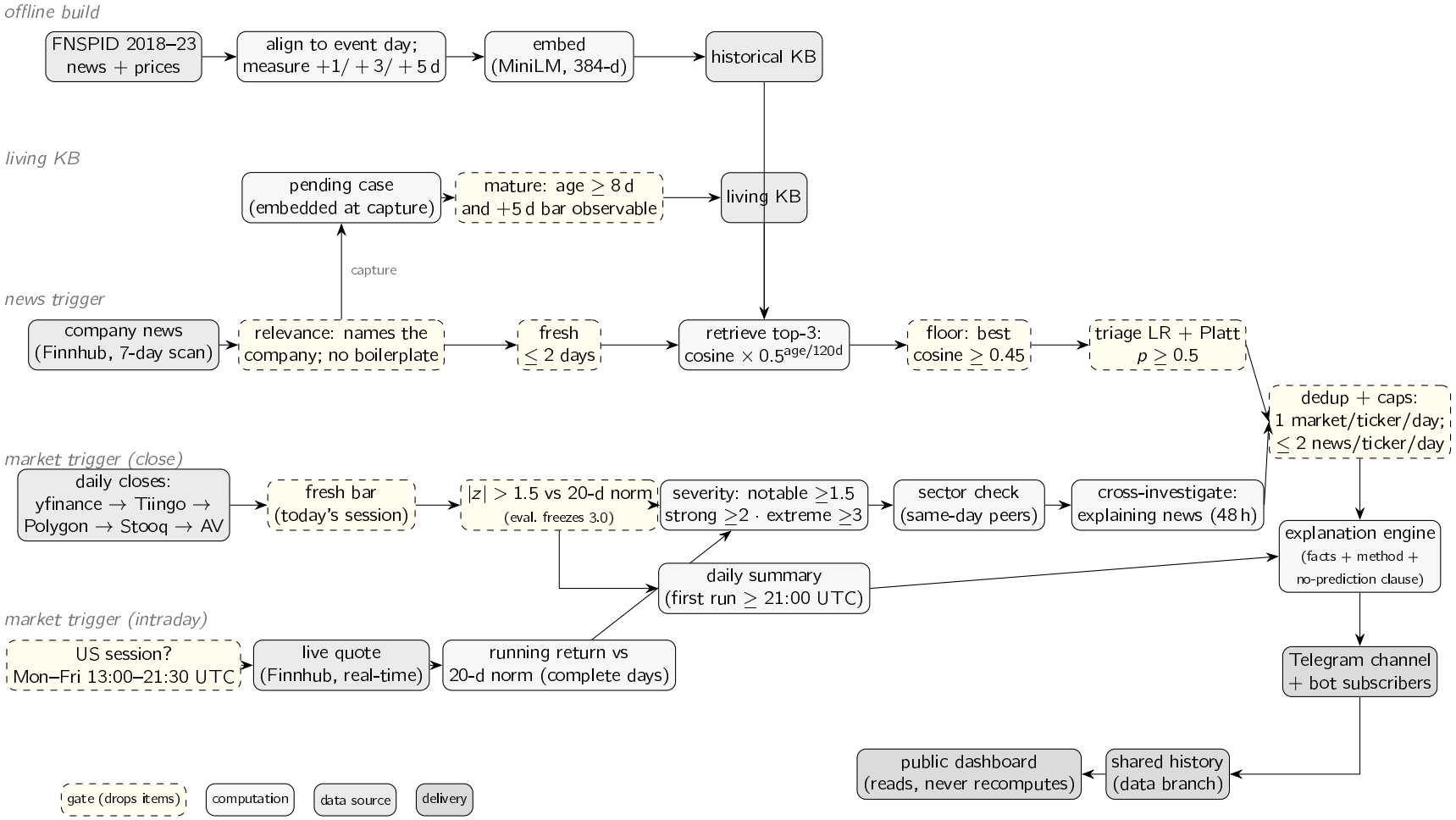

Figure A.1: The complete deployed pipeline on one page: the offline and living knowledge bases, the news trigger with its five quality gates, the close-based and intraday market triggers with the multi-source price fallback chain, and the shared outputs. Dashed boxes are gates that drop items; the annotated values (thresholds, caps, decay half-life) are the disclosed production configuration, distinct from the frozen evaluation settings of Chapter 5. The body chapters present the same system as several simpler views; this figure exists so that no stage is left implicit.

Table A.2: Every headline result, its reported value, and the study it belongs to.

Result	**Value**	**Study**
Precedent retrieval, precision@5 (MiniLM / MPNet / lexical)	0.514 / 0.538 / 0.346	CS2
Anomaly detection, F_1 (z-score vs fixed threshold)	0.516 vs 0.218	CS1
Statistical rule vs learned detectors, F_1 (z-score / IF / LOF)	0.530 / 0.269 / 0.280	CS1
EWMA volatility (validated future), F_1 vs rolling	0.664 vs 0.516	CS1
Materiality triage, PR-AUC (volatility / context / full)	0.542 / 0.538 / 0.496	CS4
Triage precision@5 per day vs alert-always	0.632 vs 0.163	CS4
Extended feature ablation, PR-AUC (context + 5 signals)	0.535 ($\Delta-0.002$)	CS4
Worked triage decision (META, 12 Jul 2026)	$p = 0.539$ (54%)	CS4
Production selectivity (headlines : alerts sent)	22 : 1	CS2
Event-type agreement, adjusted mutual information (event / ticker / sector)	0.358 / 0.188 / 0.130	CS5
Cluster purity against the rubric, versus a size-matched random assignment	0.712 vs 0.444	CS5
Conformal coverage, random split (α = 0.05/0.10/0.20)	0.951 / 0.902 / 0.803	CS6
Conformal coverage, temporal split (same α)	0.937 / 0.900 / 0.822	CS6
Definite calls at 90 % guaranteed coverage	39.5 %	CS6
Input drift, stability index (volatility / headline length / returns)	0.281 / 0.111 / $\leq$0.020	CS7
Label prevalence across the three temporal blocks	0.385 / 0.470 / 0.378	CS7
Signal fusion versus best single signal (budgets 1 / 3 / 5)	-0.0498 / -0.0317 / $+0.0045$	CS8
Derived weight on news intensity (negative)	-0.283	CS8

including the first genuine market anomaly (NVDA, 13 July 2026). The live knowledge base matured its captured headlines against the real prices that followed, and the post-validation loop labelled real past decisions with their observed outcome; what that loop measured is reported in Section 6.5 rather than here, because it qualifies a claim rather than supporting one. A large automated suite of tests, offline and deterministic, runs on every push through continuous integration. Regenerable numbers, a live deployment, and a green test suite together document a system that runs end to end and whose every reported figure can be reproduced in isolation.

A.6 Evidence Matrix

The preceding table lists results. This one lists *claims*, which is a stricter test, because a claim can outlive the evidence that once supported it. Every substantive assertion this dissertation makes is set against the evidence behind it, whether that evidence can be regenerated on demand, and what was decided about the claim as a result. Three entries record claims

that were *withdrawn or narrowed* after measurement; they are kept in the table rather than deleted, because a matrix that only lists surviving claims would not be an audit.

The reproducibility column distinguishes three cases. **Script** means a versioned procedure regenerates the figure deterministically from data that is itself regenerable. **Test** means an automated assertion fails if the claim stops holding. **Live** means the figure is measured from the running deployment and will move as more data accumulates, so the value is a snapshot with its date and the procedure, not the number, is the durable artefact.

Two observations about the table as a whole, rather than about any row in it. First, five claims were withdrawn or narrowed, and all five were withdrawn by *this work's own measurements* rather than by review. That is the intended behaviour of an evaluation designed before the experiments: it is supposed to be able to say no. Second, the rows with the weakest evidence column are also the rows this dissertation leans on least, and the strongest claim it makes, the no-lookahead guarantee, is the one enforced by a test rather than by prose. Where a claim could not be supported, it appears here as withdrawn rather than as absent.

A.7 Data and Attribution

Large data files are never versioned; they are regenerated by the data-preparation procedures, and only small samples are retained. The trained triage models are the exception by design: they are small, so the fitted bundles are kept under version control together with JSON metadata (training window, seed, threshold and metrics), which makes the deployed model itself inspectable and reproducible. The FNSPID dataset (Dong, Fan, and Peng 2024) is used under its CC BY-SA 4.0 licence, with attribution as required; the live market-data and news services are used within their free tiers.

Table A.3: Every substantive claim, its evidence, and its status. "Narrowed" and "withdrawn" entries record claims this work retracted after measuring them.

Claim	Evidence	Where	Repro.	Status
RQ1 — transparent detection				
Fires at a comparable rate across stocks of very different volatility	Firing-rate range 0.015 vs 0.344 over 15 tickers, no labels needed	§5.2	Script	Kept
Beats a fixed threshold on an extreme-move proxy	F_1 0.516 vs 0.218	§5.2	Script	Kept (proxy caveat stated)
Beats two learned detectors on identical causal features	F_1 0.530 vs 0.269 (IF) and 0.280 (LOF)	§5.2	Script	Kept
No lookahead in the detector	Window slice excludes the judged day	Listing 3.1	Test	Kept
An EWMA estimate would do better on this proxy	F_1 0.664 vs 0.516	§5.2	Script	Kept, not deployed
RQ2 — precedent retrieval				
Semantic retrieval beats lexical and trivial baselines	P@5 0.514 vs 0.346/0.240/0.126, 5 seeds	§5.3	Script	Kept
Holds at multi-year scale	P@5 0.595 on ∼80k headlines	§5.3	Script	Kept
Retrieval captures theme, not direction	Direction consistency 0.71 vs 0.69 chance floor	§5.3	Script	Kept
Relevance = sector agreement is a proxy, not ground truth	Stated; qualitative inspection per sector	§5.10	—	Limitation
RQ3 — explanation				
Alert text cannot diverge from the computation	Rendered from the computed objects; asserted per precedent	§5.4	Test	Kept
The "why" line is exact, not an approximation	Additive log-odds terms	Listing 3.4	Test	Kept
Generated language never supplies evidence	Runtime guard; red-team corpus as regression tests	§6.1	Test	Kept
Delivered-violation count is partly circular	Same checker decides and grades; stated	§6.1	—	Limitation
Explanations are *useful*	—	—	—	**Not claimed**
RQ4 — learned triage				
Training is deterministic and reproduces the stored artefacts	Frozen bundle regenerates PR-AUC, ROC-AUC, Brier and budgeted precision exactly	§5.5	Test	Kept
No lookahead in the features	Mutating the future leaves features identical and changes the label	Listing 3.2	Test	Kept
Blocks do not share label windows	Split by unique day + embargo (820 rows dropped)	Listing 3.3	Test	Kept
No text model beats the volatility baseline	PR-AUC 0.542 vs 0.496; survives cluster bootstrap, a fair re-test and a change of metric	§5.5	Script	Kept
Triage raises within-budget precision 0.163 → 0.632	Held-out block of the training corpus	§5.5	Script	**Narrowed** to evaluation set
The deployed gate selects material news	ROC-AUC 0.494, cluster CI [0.391, 0.601], on 145 (ticker, day) units	§6.5	Live	**Withdrawn**
The deployed gate controls volume	Suppresses ∼3/5 of news decisions	§4.9	Live	Kept
System and deployment				
The evaluated model and the deployed model are the same model	Cosine 0.992; 95% of retrieved neighbours shared over 503 queries	§4.9	Script	Kept
Top-3 neighbour sets identical in 20 of 23 queries	Does not reproduce; 12/23 on another sample of the	—	—	**Withdrawn**